

第 17 章: Scopes (作用域)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 上一章 (第 16 章) 介绍了函数的基本定义与调用, 本章深入 Python 最核心的机制之一——作用域 (scopes): 名字在哪里定义、如何被查找。你将学习 LEGB 查找规则、global 与 nonlocal 声明、闭包 (closure) 与工厂函数 (factory function), 以及四种跨调用保存状态 (state retention) 的方案。本章是第 18 章"参数传递"的直接前置, 也是理解后续装饰器 (第 39 章)、类与继承 (Part VI) 的基石。

17.1 开篇引言 (Chapter Intro)

英文原文

The preceding chapter introduced basic function definitions and calls. As we saw, Python's core function model is simple to use, but even simple function examples quickly lead to questions about the meaning of variables in code. This chapter moves on to present the details behind Python's scopes—the places where variables are defined and looked up. Like module files, scopes help prevent same-name clashes across a program's code: names defined in one program unit

don't interfere with names in another. As you'll learn here, the place where a name is assigned in your code is crucial to determining what the name means. You'll also find that scope usage can have a major impact on program maintenance effort; overuse of globals, for example, is a generally bad idea. On the plus side, you'll also learn that scopes can provide a way to retain state information between function calls, and they offer an alternative to classes in some roles.

中文翻译

上一章介绍了基本的函数定义与调用。正如我们所看到的，Python 的核心函数模型用起来很简单，但即使是简单的函数示例也会很快引出关于“代码中的变量到底意味着什么”的问题。本章继续深入 Python 的作用域（scopes）背后的细节——作用域就是变量被定义和查找的“地方”。和模块文件一样，作用域有助于防止同名名字（name）在整个程序的代码中相互冲突：在一个程序单元中定义的名字不会干扰另一个程序单元中的名字。你将在本章学到：名字在你的代码中被赋值的位置，是决定这个名字含义的关键。你还会发现，作用域的用法会对程序维护工作量产生重大影响；例如，过度使用全局变量（globals）通常是个坏主意。好的一面是，你还会学到：作用域提供了一种在函数调用之间保留状态信息（state information）的方式，并且在某些角色中可以替代类（class）的使用。

深度理解

·核心概念：本章要回答一个看似简单的问题——“当代码

里写了一个名字 X 时，Python 到哪儿去找它的值？"答案是：不是看"谁调用了谁"，而是看"X 在源码里被赋值的位置"。

·底层视角：Python 中几乎一切名字相关的事都发生在赋值时刻。名字不是预先声明的，而是第一次被赋值时"凭空出现"的。这决定了作用域分析是静态的（词法的，lexical）——编译器扫一眼源码就能确定每个名字属于哪个作用域，而不需要运行程序。

·为什么这样设计：作用域的本质是"隔离"——让不同的程序单元可以放心使用同名变量而不互相干扰。这和模块文件的目的一致：上一章学过的"局部变量避免名字冲突"，正是本章将要展开的机制。

·解决什么实际问题：本书的两大核心函数概念是"作用域（本章）"与"参数传递（第 18 章）"。作用域决定了"名字在哪找"，而状态保留技术（nonlocal、闭包等）让函数能"记住"调用之间的信息——这是计数器、GUI 事件处理器、装饰器等无数真实需求的地基。

·初学者误区：很多新手以为"函数里能用全局变量 = 可以随便改全局变量"，或者认为作用域由调用关系决定。本书反复强调：作用域完全由源码中赋值的位置决定，与谁调用谁无关；并且"读"全局变量不需要声明，但"改"全局变量必须写 global。

17.2 Python 作用域基础 (Python Scopes Basics)

英文原文

Now that we're starting to write our own functions, we need to get more formal about what names mean in Python. When we use a name in a program, Python creates, changes, or looks up the name in what is known as a namespace—a place where names live. And when we talk about names in relation to code, namespaces correspond to scopes: the location of a name's assignment in your source code determines the scope of the name's visibility in your program. Just about everything related to names, including scope classification, happens at assignment time in Python. As we've seen, names in Python spring into existence when they are first assigned values, and they must be assigned before they are used. Because names are not declared ahead of time, Python uses the location of the assignment of a name to associate it with (i.e., bind it to) a particular namespace and scope. In other words, the place where you assign a name in your source code determines the namespace it will live in, and hence its scope of visibility. Besides packaging code for reuse, functions add an extra namespace layer to your programs to minimize the potential for collisions among variables of the same name—by default, all names assigned inside a function are associated with that function's namespace, and no other. This rule means that:- Names assigned inside a def can be seen only

by the code within that `def`. You cannot even refer to such names from outside the function.- Names assigned inside a `def` do not clash with variables outside the `def`, even if the same names are used elsewhere. A name `X` assigned outside a given `def` (i.e., in a different `def` or at the top level of a module file) is a completely different variable from a name `X` assigned inside that `def`.- Names assigned in a `lambda` work the same way, though assignment in their expressions can happen only for arguments and `:=` named assignments, and is much less common than in `def`. In all cases, the scope of a variable (where it can be used) is wholly determined by where it is assigned in your source code, and has nothing to do with which functions call which. In fact, as you'll learn in this chapter, variables may be assigned in three different places, corresponding to three different scopes:- If a variable is assigned inside a `def` or `lambda`, it is local to that function.- If a variable is assigned in an enclosing `def` or `lambda`, it is nonlocal to nested functions.- If a variable is assigned outside all `defs` and `lambdas`, it is global to the entire file. We call this lexical scoping because variable scopes are determined entirely by the locations of the variables in the source code of your program files, not by function calls. Hence, a simple visual inspection is enough to make the call. For example, in the following module file, the `X = 99` assignment creates a global variable named `X` (visible everywhere in this

file), but the `X = 88` assignment creates a local variable `X` (visible only within the `def` statement). If we refer to `X` inside the function, we'll get the function's `X`, not the global: `python X = 99 # Global (module) scope`
`X def func(): X = 88 # Local (function) scope X: a different variable` Even though both variables are named `X`, their scopes make them different. The net effect is that function scopes help to avoid name clashes in your programs and help to make functions more self-contained program units—their code need not be concerned with names used elsewhere.

中文翻译

既然我们开始编写自己的函数了，就需要更正式地理解 Python 中名字 (name) 的含义。当程序中使用一个名字时，Python 会在一个称为命名空间 (namespace) ——也就是“名字居住的地方”——中创建、修改或查找这个名字。而当我们在代码语境下谈论名字时，命名空间就对应着作用域 (scope)：你的源代码中名字被赋值的位置，决定了这个名字在程序中的可见范围。在 Python 中，几乎所有与名字相关的事情——包括作用域的分类——都发生在赋值时刻。正如我们所见，Python 中的名字在第一次被赋值时才“凭空出现”，并且必须先赋值才能使用。由于名字不需要预先声明，Python 就利用名字被赋值的位置，把它绑定 (bind) 到某个特定的命名空间和作用域上。换句话说，你在源代码中赋值一个名字的位置，决定了它居住的命名空间，从而决定了它的可见作用域。除了把代码打包复用之外，函数还为你的程序增加了一层命名空间，以最大限度地降低同

名变量之间发生冲突的可能性——默认情况下，函数内部被赋值的所有名字都与该函数的命名空间关联，绝不与其他命名空间关联。这条规则意味着：

- 在 `def` 内部被赋值的名字，只有该 `def` 内部的代码能看到。你甚至无法从函数外部引用这些名字。
- 在 `def` 内部被赋值的名字，不会与 `def` 外部的变量冲突，即使其他地方使用了相同的名字。在某个 `def` 外部（例如在另一个 `def` 中，或模块文件顶层）赋值的名字 `X`，与在该 `def` 内部赋值的名字 `X` 是完全不同的变量。
- 在 `lambda` 中被赋值的名字遵循同样的规则，不过 `lambda` 表达式中的赋值只能发生在参数和 `:=` 命名赋值（named assignment）上，而且远不如在 `def` 中常见。在所有情况下，变量的作用域（它在哪里可以被使用）完全由它在源代码中被赋值的位置决定，与“哪个函数调用哪个函数”毫无关系。事实上，你将在本章学到：变量可能在三个不同的位置被赋值，分别对应三种不同的作用域：

- 如果变量在 `def` 或 `lambda` 内部被赋值，它对该函数而言是局部（local）的。
- 如果变量在某个外围的 `def` 或 `lambda` 中被赋值，它对嵌套函数而言是非局部（nonlocal）的。
- 如果变量在所有 `def` 和 `lambda` 之外被赋值，它对该文件整体而言是全局（global）的。我们称之为词法作用域（lexical scoping），因为变量的作用域完全由变量在程序文件源代码中的位置决定，而不是由函数调用决定。因此，只需简单地看一眼源码就能作出判断。例如，在下面的模块文件中，`X = 99` 赋值创建了一个名为 `X` 的全局变量（在此文件内处处可见），而 `X = 88` 赋值创建了一个局部变量 `X`（只在 `def` 语句内部可见）。如果我们在函数内部引用 `X`，得到的将是函数自己的 `X`，而不是全局的那个：

```
python X = 99 # 全局（模块）作用域中的 X
def func(): X = 88 # 局部（函数
```

作用域中的 X：一个不同的变量尽管两个变量都叫 X，但它们的作用域使它们彼此不同。净效果是：函数作用域有助于避免程序中的名字冲突，并让函数成为更自包含（self-contained）的程序单元——它们的代码不必担心其他地方用过的名字。

深度理解

·核心概念：一个名字的"身份"由三件事确定——名字本身、赋值位置（决定作用域）、它引用的对象。两个同名变量只要作用域不同，就是完全不同的变量。

·底层实现：在 CPython 中，每个作用域在编译期被识别出来。模块代码编译后全局名字存储在模块字典（module.dict）中；函数代码编译时，所有函数内部被赋值的名字被标记为 local，对应的字节码指令是 STOREFAST（存入函数自己的"快速局部变量槽"），而引用外部作用域的名字用 LOADGLOBAL/LOADDEREF 等指令。函数调用时会创建栈帧（frame），局部变量槽就存在栈帧里——这就是"每次调用都有独立局部命名空间"的物理基础。

·为什么这样设计：把"赋值位置"当作分类依据，是因为 Python 不需要声明语句。C/Java 要写 `int x;` 声明，Python 没有；为了让编译器仍然能确定一个名字属于哪层作用域，就必须用"赋值出现的位置"这个唯一可靠的信息来源。设计目标是：静态可判定、无需运行程序就能看懂。

·解决什么实际问题：大型程序中不同函数都可能想用 `i`、`count`、`result` 这样的通用名字。如果没有作用域隔离，这些名字会互相踩踏。作用域让函数成为"自包含"的软件单

元——写函数时不用关心别人用了什么名字。

·初学者误区：①误以为 lambda 里的名字与 def 不同——其实规则完全一样；②误以为"作用域由调用关系决定"——f1 调用 f2，f2 里的 X 仍然是 f2 自己的局部变量；③误以为"能在函数里读到全局变量 = 全局变量可以被随便改"——读与改的规则不同（改需要声明），这是本章后半部分的重点。

代码分析

```
X = 99          # X
def func():
    X = 88      # X
```

解释器如何处理：当 Python 编译 func 函数体时，它发现 X = 88 出现在函数内部，于是把 X 标记为 func 的局部变量（编译期决定，运行前就定了）。此后函数体内任何对 X 的引用，都会直接访问栈帧中的局部槽，而不是模块字典。用 dis 反汇编可以看到 func 里是 STOREFAST X；模块顶层则是 STORENAME X。所以 func() 执行完后，func 内的 X 被销毁（栈帧回收），模块的 X 依旧是 99。

设计思想：这就是"名字必须先赋值才能用"与"赋值位置决定作用域"两条规则的合流——源码里画一条竖线，竖线右边的所有赋值都是局部的，这是用眼睛就能检查的规则，成本最低、最不易出错。

17.3 作用域概述 (Scopes Overview)

英文原文

Before we started writing functions, none of the code we wrote was nested in a `def`, so the names we used either lived in a module or were built-ins predefined by Python like `open` and `zip`. This includes code typed at the REPL: the interactive prompt is technically a module named `main` that prints results and doesn't correspond to a real file; in all other ways, though, it's like the top level of a module file. Functions, though, provide nested namespaces (scopes) that localize the names they use, such that names inside a function won't clash with those outside it (in a module or another function). Specifically, functions define a local scope and modules define a global scope with the following properties:- The enclosing module is a global scope. Each module is a global scope—that is, a namespace in which variables created by assignment at the top level of the module file live. Global variables become attributes of a module object to the outside world after imports but can also be used as simple variables within the module file itself.- The global scope spans a single file only. Don't be fooled by the word "global" here—names at the top level of a file are global to code within that single file only. There is no notion of an all-encompassing global scope that correspond

s to user code in Python (built-ins are truly global, but not user defined). Instead, names are partitioned into modules, and you must always import a module explicitly if you want to be able to use the names its file defines. When you hear "global" in Python, think "module." - Assigned names are local unless declared global or nonlocal. By default, all the names assigned inside a function definition are put in its local scope—the namespace associated with the function itself. To assign a name that lives at the top level of the module enclosing a function, declare it in a global statement inside the function. To assign a name that lives in an enclosing def instead, declare it in a nonlocal statement. Because lambda bodies are limited to expressions, these two statements are just for def. - All other names are enclosing function locals, globals, or built-ins. Names not assigned a value in the function definition are assumed to be either locals defined in a physically enclosing function, globals that live in the enclosing module's namespace, or built-ins in the predefined built-ins module that Python provides. Enclosing scopes can arise for any def and lambda combo, though def can't be coded in lambda. - Each call to a function creates a new local scope. Every time you call a function, you create a new local scope—that is, a namespace in which the names created inside that function will live, barring global or nonlocal statements. You can think of each def statement and lambda expressi

on as defining a new local scope, but the local scope actually corresponds to a function call, for reasons the next bullet explains. - Per-call scopes matter in recursion and closures. It's crucial that each active call receive its own copy of the function's local variables when using recursion, an advanced technique that allows functions to loop by calling themselves, and noted briefly in Chapter 9 for comparisons. Recursion is useful in functions we write as well, to process structures whose shapes can't be predicted ahead of time; we'll explore it more fully in Chapter 19. Per-call scopes are also required for the closure functions covered ahead here: each call gets a fresh packet of the function's locals, which can remember state-information objects to be used later. Beyond those basics, there are three subtleties worth underscoring. First, as noted, code typed at the Python interactive prompt lives in a module, too, and follows the normal scope rules: names assigned there are global variables, accessible to the entire interactive session. You'll learn more about modules in the next part of this book. Second, also bear in mind that any type of assignment classifies a name as local or global, based on where the assignment appears. This includes `=` statements and `:=` expressions; module names in `import`; function and argument names in `def`; names in class covered later; and so on. If you assign a name in any way within a function, it will be local to that function by default. This include

s its arguments listed in its header, but also names in its body's code. Third, in-place changes to objects do not classify names as locals; only actual name assignments do. For instance, if the name `L` is assigned to a list at the top level of a module, a statement `L = X` within a function will classify `L` as a local, but `L.append(X)` will not. In the latter case, we are changing the list object that `L` references, not `L` itself—hence, `L` is found in the global scope as usual, and Python happily modifies it without requiring a global declaration. As usual, it helps to keep the distinction between names and objects clear: changing an object is not an assignment to a name.

中文翻译

在我们开始编写函数之前，所写的代码没有哪部分是嵌套在 `def` 里的，因此我们使用的名字要么存在于模块中，要么是 Python 预定义的内置 (built-in) 名字，比如 `open` 和 `zip`。这包括在 REPL 中键入的代码：交互提示符 (interactive prompt) 在技术上是一个名为 `main` 的模块，它会打印结果，并且不对应任何真实文件；但在其他所有方面，它都等同于模块文件的顶层。不过，函数提供了嵌套的命名空间（作用域），把函数使用的名字局部化，使得函数内部的名字不会与函数外部的名字（在模块或另一个函数中）冲突。具体来说，函数定义了一个局部作用域 (local scope)，模块定义了一个全局作用域 (global scope)，它们具有以下性质：- 外围模块就是全局作用域。每个模块都是一个全局作用域——也就是说，在模块文件顶层通过赋值创建的

变量就居住在这个命名空间中。导入之后，全局变量对外部世界而言会变成模块对象的属性（attribute），但在模块文件内部，它们也可以当作普通变量使用。- 全局作用域只覆盖单个文件。别被这里的"global（全局）"一词骗了——文件顶层的名字只对该单个文件内的代码是"全局"的。Python 中不存在一个包罗万象、对应所有用户代码的全局作用域（内置名字才是真正全局的，但它们不是用户定义的）。相反，名字被划分到各个模块中，而且你必须显式 import 一个模块，才能使用它的文件所定义的名字。在 Python 里听到 "global"，就请想成 "module（模块）"。- 被赋值的名字默认为局部（local），除非声明为 global 或 nonlocal。默认情况下，函数定义内部被赋值的所有名字都被放入它的局部作用域——即与函数本身关联的命名空间。要赋值一个位于函数外围的模块顶层的名字，需要在函数内部用 global 语句声明它。要赋值一个位于外围 def 中的名字，则用 nonlocal 语句声明。由于 lambda 的函数体只限于表达式，这两个语句只能用于 def。- 其余所有名字都来自外围函数的局部变量、全局变量或内置名字。在函数定义中没有被赋值的名字，被认为是以下三者之一：在物理上外围函数中定义的局部变量、位于外围模块命名空间中的全局变量，或 Python 提供的预定义 built-ins（内置）模块中的内置名字。任何 def 与 lambda 的组合都可能产生外围作用域，不过 lambda 里不能写 def。- 每次调用函数都会创建新的局部作用域。每当你调用一个函数，就创建了一个新的局部作用域——也就是函数内部创建的名字将要居住的命名空间（除非有 global 或 nonlocal 语句干扰）。你可以把每个 def 语句和 lambda 表达式理解为定义了一个新的局部作用域，但局部作用域实际上对应一次函数调用，原因

见下一条。 - 每次调用的独立作用域在递归 (recursion) 与闭包 (closure) 中至关重要。使用递归时，每个活跃调用必须得到函数局部变量各自独立的副本——递归是一种让函数通过调用自身来实现循环的高级技术，第 9 章在比较时简单提过。递归在我们自己写的函数中同样有用，可用于处理形状无法事先预测的结构；我们将在第 19 章更完整地探索它。本章后面的闭包函数也需要每次调用的独立作用域：每次调用都会得到函数局部变量的一包全新数据，这些数据可以记住状态信息对象，供以后使用。在这些基础之上，还有三个值得强调的微妙之处。第一，如前所述，在 Python 交互提示符下键入的代码也居住在一个模块中，并遵循正常的作用域规则：在那里赋值的名字是全局变量，整个交互会话都能访问。你将在本书下一部分学到更多关于模块的知识。第二，请记住：任何形式的赋值都会根据赋值出现的位置，把一个名字归类为局部或全局。这包括 `=` 语句和 `:` `=` 表达式；`import` 中的模块名；`def` 中的函数名与参数名；后面要讲的 `class` 中的名字；等等。只要你在函数内部以任何方式给一个名字赋值，默认它就会成为该函数的局部变量。这既包括函数头部列出的参数，也包括函数体代码中的名字。第三，对对象的就地修改 (in-place change) 不会把名字归类为局部变量；只有真正的名字赋值才会。例如，如果名字 `L` 在模块顶层被赋值为一个列表，那么函数内的语句 `L = X` 会把 `L` 归类为局部变量，但 `L.append(X)` 不会。在后者中，我们改变的是 `L` 所引用的列表对象，而不是 `L` 本身——因此 `L` 会照常在全局作用域中被找到，Python 会愉快地修改它，而无需 `global` 声明。和往常一样，区分“名字”与“对象”很有帮助：改变一个对象并不是对一个名字的赋值。

深度理解

·核心概念：作用域体系是"三加一"结构——def/lambda 的局部作用域、模块的全局作用域、Python 预置的 built ins 作用域，外加"每次调用新建局部作用域"的动态特性。注意"global"的实际含义是"module (模块)"，每个文件一个，互不相通，想用别的文件的名字必须 import 。

·底层实现：CPython 中一次函数调用 = 压入一个栈帧 (frame object)。栈帧持有局部变量槽 (flocals)、代码对象引用、字节码计数器等。递归时每层递归各压一个栈帧，同名局部变量互不干扰；递归返回时逐层弹出栈帧并回收。这就是"每次调用都有新局部作用域"的物理实现。

·为什么这样设计：把作用域与"调用"而不是"定义"绑定，是为了支持递归（每层调用需要独立的状态）和闭包（后续每次调用工厂函数都要产生独立的"记忆包"）。如果把局部作用域绑定在"定义"上，递归和闭包都无法工作。

·解决什么实际问题：①"交互式 REPL 也是一个模块"解释了为什么在 REPL 里赋值的变量在会话中一直存在；②"任何赋值形式都归类名字"意味着 for i in ...、import、def、class 的名字都可能被归类——特别是 for 循环变量不会局部化（第 8 章提过），它留在外层作用域；③"就地修改不算赋值"直接引出后文"用可变对象绕过 nonlocal"的技巧（本意是让你区分名字与对象）。

·初学者误区：①以为 global 意味着"整个程序都能用"——实际上只是"整个模块文件"；②以为函数参数是某种特

殊名字——参数就是局部变量（通过赋值传入）；③以为 `L.append(x)` 在函数里写了就得声明 `global L`——不需要，因为没给 `L` 这个名字赋值。

·生活例子：作用域就像办公室的档案柜：每个函数（办公室）有自己上锁的抽屉（局部作用域），每个模块（部门）有部门共享柜（全局作用域），而 `builtins` 是公司总部的公共资源（打印机、空调），谁都能用。函数调用结束 = 抽屉销毁；递归 = 每层递归都是新办公室。

17.4 名字解析：LEGB 规则 (Name Resolution: The LEGB Rule)

英文原文

If the prior section sounds complicated, it really boils down to three simple rules. Within a `def` statement:- Name assignments create or change local names by default.- Name references search at most four scopes: local, then enclosing functions (if any), then global, then built-in.- Names declared in global and nonlocal statements map assigned names to enclosing module and function scopes, respectively. The same goes for `lambda`, except for the last bullet: it doesn't support statements. In other words, all names assigned inside a function `def` statement or `lambda` expression are locals by default. Functions can freely use names assigned in syntactically enclosing functions and the gl

obal scope, but they must declare such nonlocals and globals in order to change them. Python's name-resolution scheme is usually called the LEGB rule, after the names of the scopes it searches:- When you use an unqualified name (not after a ".") inside a function, Python searches up to four scopes—the local (L) scope of the function itself, then the local scopes of any enclosing (E) defs and lambdas, then the global (G) scope of the surrounding module, and finally the built-in (B) scope—and stops at the first place the name is found. If the name is not found during this search, Python reports an error.- When you assign a name inside a function (instead of just referring to it in an expression), Python always creates or changes the name in the assigner's local scope, unless it's declared to be global or nonlocal there.- When you assign a name outside any function (i.e., at the top level of a module file, or at the interactive prompt), the local and global scope are one and the same—the module's namespace. Because names must be assigned before they can be used (as we saw in Chapter 6), there are no automatic components in this model: assignments always determine name scopes unambiguously. Figure 17-1 illustrates Python's four scopes and LEGB rule. Note that the second scope lookup layer, E—the scopes of enclosing defs or lambdas—can technically correspond to more than one lookup level. This layer only comes into play when you nest functions within functions for r

asons you'll meet ahead, and is enhanced by the `nonlocal` statement. ¹ Figure 17-1. The LEGB scope lookup rule for names Also keep in mind that these rules apply only to simple variable names (e.g., `name`). In Parts V and VI, you'll see that qualified attribute names (e.g., `object.name`) live in particular objects and follow a completely different set of lookup rules than those covered here. References to attribute names following periods (.) search one or more objects, not scopes, and in fact may invoke something called inheritance in Python's OOP model; more on this in Part VI.

中文翻译

如果上一节听起来很复杂，其实它完全可以归结为三条简单规则。在 `def` 语句内部：- 名字赋值默认创建或修改局部名字（local name）。- 名字引用最多搜索四个作用域：局部（local），然后是外围函数（若有），再是全局（global），最后是内置（built-in）。- 在 `global` 和 `nonlocal` 语句中声明的名字，会把被赋值的名字分别映射到外围的模块作用域和函数作用域。`lambda` 也遵循同样规则，只是最后一条除外：它不支持语句。换句话说，所有在函数内部的 `def` 语句或 `lambda` 表达式中被赋值的名字，默认都是局部变量。函数可以自由使用在语法上外围的函数和全局作用域中赋值的名字，但要修改这些 `nonlocal` 和 `global` 名字，则必须声明。Python 的名字解析方案通常被称为 LEGB 规则，取自它搜索的各作用域名字的首字母：- 当你在函数内部使用一个非限定（unqualified）名字（后面没有跟 ".")

时，Python 最多搜索四个作用域——函数自身的局部（L, local）作用域，然后是任何外围（E, enclosing）def 和 lambda 的局部作用域，再是外围模块的全局（G, global）作用域，最后是内置（B, built-in）作用域——并在第一个找到该名字的地方停止。如果搜索过程中没有找到该名字，Python 会报告错误。

- 当你在函数内部（而不是仅仅在表达式中引用它）赋值一个名字时，Python 总是会在赋值者自己的局部作用域中创建或修改这个名字，除非它在那里被声明为 global 或 nonlocal。
- 当你在任何函数之外赋值一个名字（即在模块文件顶层，或在交互提示符下）时，局部作用域与全局作用域合二为一——都是模块的命名空间。

因为名字必须先赋值才能使用（如第 6 章所见），这个模型中没有自动成分：赋值总能明确无误地决定名字的作用域。图 17-1 展示了 Python 的四个作用域与 LEGB 规则。注意第二个查找层 E——外围 def 或 lambda 的作用域——在技术上可能对应不止一层查找。这一层只有在你出于后面将要讲到的原因在函数中嵌套函数时才发挥作用，并且由 nonlocal 语句增强。

图 17-1. 名字的 LEGB 作用域查找规则还要记住：这些规则只适用于简单的变量名字（例如 name）。在第 V 部分和第 VI 部分你会看到，限定的（qualified）属性名字（例如 object.name）居住在特定的对象中，遵循与这里完全不同的查找规则。跟在句点（.）后面的属性名引用搜索的是一个或多个对象，而不是作用域，事实上在 Python 的 OOP 模型中它可能触发所谓的继承（inheritance）机制；第 VI 部分会有更多介绍。

深度理解

·核心概念：LEGB 是一个"从里往外逐层翻找"的查找链——

—Local (当前函数) → Enclosing (一层层外围函数)
→ Global (模块) → Built-in (builtins 模块)。找到即停, 找不到就抛 `NameError`。而赋值是"单向定位": 默认永远写进当前函数的局部作用域。

·底层实现: CPython 编译器在编译每个函数时, 扫描函数体里的每个名字并分类: 赋值过的 → local (字节码 `STOREFAST/LOADFAST`); 只读且外围函数有赋值 → cell / free variable (`LOADDEREF`); 只读且在模块层 → global (`LOADGLOBAL`); 都不在 → 按 builtins 查找 (`LOADGLOBAL` 的 fallback)。注意: 作用域分类是编译期完成的——"函数里有没有写过 `X = ...`"在编译时就有定论, 与运行时机无关。这也解释了著名的 `UnboundLocalError`: 函数里某处写了 `X = ...`, 则整个函数内 `X` 都是局部变量, 函数开头"读" `X` 也会报错, 而不是去读全局的 `X`。

·为什么这样设计: LEGB 让"读"免费 (任意层都能读), 让"写"默认就近 (写到当前函数)。这保证了: 写函数时不用担心覆盖别人; 读时又能自然复用全局/内置的名字。一句话: 读外放, 写内聚。

·解决什么实际问题: 你写的每个 `print()` 能直接使用, 是因为 LEGB 最后会找到 B 层的 builtins; 你在函数里想读模块级配置常量, 无需声明; 但若想在函数里修改模块级变量, 就必须 `global`——这种"要明说才给改"的设计刻意提高做"错事"的成本。

·初学者误区: ①把"读"与"写"混为一谈——能读到全局不等于能改全局; ②以为 LEGB 的 E 层"一跳而过"——嵌套

多层函数时 E 层是多层的，逐层由内向外查找；③以为"点号" (object.name) 也走 LEGB——属性查找走对象继承链，跟作用域无关；④忘记"名字必须先赋值"——所以永远不会出现"隐式创建"的魔法，行为总是确定。

17.5 预览：Python 的其他作用域 (Preview: Other Python scopes)

英文原文

Though obscure at this point in the book, there are technically three more scopes in Python—temporary loop variables in comprehensions, exception reference variables in try handlers, and local scopes in class statements. The first two of these are special cases that rarely impact real code, and the third falls under the LEGB umbrella rule. Importantly, most statement blocks and other constructs do not localize the names used within them. This includes loop variables in for loop statements, and the targets of `:=` named assignment, which works the same as all others with regard to scopes. There are, however, some boundary cases whose variables are not available to (and do not clash with) surrounding code, and which involve topics covered in full elsewhere in this book:

- Comprehension loop variables: The variable (or variables) `X` in `[X for X in I]` used to refer to the current iteration item in comprehension expressions. Because they might clash with

other names and reflect internal state in generators, such names are local to the expression itself in all comprehension forms: list, dictionary, set, and generator. By contrast, for loop statements never localize their loop variable (e.g., `X in for X in l:`) to the statement block as noted, despite the similarity. Moreover, names assigned by `:=` within a comprehension do leak out to the enclosing scope, per the info and demos in Chapter 20.

- Exception variables: The variable `X` in `except E as X` used to reference the raised exception in a try statement handler. Because this might defer garbage collection's memory recovery, such variables are local to that except block, and in fact are removed from the containing scope when the block is exited (even if you've used them earlier in your code!). You'll learn all about try statements in Chapter 34.
- Named assignments in lambda: The variable `X` in `X := Y` used in assignment expressions. In terms of scopes, the `:=` works the same as an unnested `=` assignment statement in general. Keep in mind, though, that a lambda function expression creates a new local function scope just like a `def` statement. Hence, any names assigned by a `:=` nested in a lambda are local variables that can be used within the body of the lambda itself, but are not available to code outside the lambda expression. That said, this is likely a rare coding pattern in most code (and pushes the envelope on complexity). For a refresher on `:=`, see Chapter 11. Most of these contexts au

gment the LEGB rule, rather than modifying it. Loop variables assigned in a comprehension, for example, are simply bound to a further nested and special-case local scope; all other names referenced within these expressions follow the usual LEGB lookup rules. It's also worth noting that the class statement you'll meet in Part VI creates a new local scope, too, for the names assigned inside the top level of its block. As for `def`, names assigned inside a class don't clash with names elsewhere, and follow the LEGB lookup rule, where the class block is the L level. As for imported modules, these class-local names also morph into class object attributes after the class statements ends. For all practical purposes, classes are a local scope (despite some rare border cases that likely qualify as glitches, and can be safely omitted here). Unlike functions, though, class names are not created per call: class object calls generate instances, which inherit names assigned in the class and record per-object state as per-instance attributes of their own. As you'll also learn in Chapter 29, although the LEGB rule is used to resolve names used in both the top level of a class itself as well as the top level of method functions nested within it, classes themselves are skipped by scope lookups—their names must be fetched as object attributes. Hence, because Python searches enclosing functions for referenced names, but not enclosing classes, the LEGB rule sketched in Figure 17-1 still applies to OOP code.

中文翻译

虽然在本书目前的阶段这些还比较冷门，但严格来说 Python 还有三种作用域——推导式（comprehension）中的临时循环变量、try 处理器中的异常引用变量，以及 class 语句中的局部作用域。前两种是很少影响真实代码的特例，第三种则属于 LEGB 规则的总框架之下。重要的是：大多数语句块和其他构造不会把其中使用的名字局部化。这包括 for 循环语句中的循环变量，以及 `:=` 命名赋值的目标——它在作用域方面与其它所有赋值表现一致。不过，确实存在一些边界情形，其中的变量对周围代码不可见（也不会与之冲突），它们涉及本书其他章节详细讨论的主题：

- 推导式的循环变量：`[X for X in I]` 中的变量 `X` 用于指代推导式表达式中当前的迭代项。因为它们可能与其他名字冲突，并反映生成器（generator）的内部状态，所以这类名字在所有推导式形式中都是表达式自身的局部变量：列表、字典、集合和生成器推导式皆如此。相比之下，for 循环语句永远不会把循环变量（例如 `for X in I:` 中的 `X`）局部化到语句块内——尽管两者看似相似。此外，根据第 20 章的信息与演示，推导式内部由 `:=` 赋值的名字确实会泄漏到外围作用域。
- 异常变量：`except E as X` 中的变量 `X` 用于在 try 语句处理器中引用被抛出的异常。因为保留它可能会推迟垃圾回收（garbage collection）的内存回收，这类变量是该 except 块内部的局部变量，而且事实上当退出该块时会被从所在作用域中移除（即使你早就在代码里用过它！）。你将在第 34 章全面学习 try 语句。
- lambda 中的命名赋值：赋值表达式（assignment expression）中 `X := Y` 的变量 `X`。就作用域而言，`:=` 与一般的非嵌套 `=` 赋值语句行为相同。但

要记住：lambda 函数表达式与 def 语句一样，会创建新的局部函数作用域。因此，嵌套在 lambda 中由 := 赋值的任何名字，都是可以在 lambda 函数体内部使用的局部变量，但对 lambda 表达式之外的代码不可见。话虽如此，这在大多数代码中可能是罕见的写法（而且把复杂度推到了极限）。需要复习 := 的话，参见第 11 章。上述大多数情形都是对 LEGB 规则的增强（augment），而不是修改。例如，推导式中赋值的循环变量只是被绑定到一个更深层的、特殊处理的局部作用域；这些表达式中引用的其它所有名字仍遵循通常的 LEGB 查找规则。同样值得注意的是：你将在第 VI 部分遇到的 class 语句也会为其块顶层内部赋值的名字创建一个新的局部作用域。和 def 一样，class 内部赋值的名字不会与其它地方的名字冲突，并遵循 LEGB 查找规则——此时 class 块就是 L 层。与导入的模块类似，这些类局部名字在 class 语句结束后也会转变为类对象的属性。从一切实用目的来看，类就是一个局部作用域（尽管存在一些罕见、大概算得上缺陷的边界情形，此处可以放心忽略）。不过与函数不同的是，class 名字不是按调用创建的：调用类对象会生成实例（instance），实例继承在 class 中赋值的名字，并把每个对象的各自状态记录为自己的实例属性。正如你在第 29 章还会学到的：虽然 LEGB 规则既用于解析类自身顶层使用的名字，也用于解析嵌套在类中的方法函数顶层使用的名字，但类本身会被作用域查找跳过——它们的名字必须作为对象属性来获取。因此，由于 Python 会搜索外围函数来找被引用的名字，却不会搜索外围类，图 17-1 勾勒的 LEGB 规则依然适用于 OOP 代码。

深度理解

·核心概念：LEGB 是"总规则"，三种特殊作用域是"例外条款"：推导式循环变量（局部化）、except as 异常变量（局部化且退出即删除）、类块（局部作用域但被查找跳过）。其余语句块——for、while、if、with——都不局部化名字。

·底层实现：推导式的循环变量在 CPython 3.x 中被编译进独立的代码对象（code object），拥有自己的作用域；except as X 变量在块结束时被解释器执行 DELETENAME X 显式删除（这正是为了不拖住异常对象的内存回收）。类块则被编译为一个特殊的代码对象（LOADBUILDClass 相关的构建流程），其名字表存入类命名空间 dict。

·为什么这样设计：推导式局部化循环变量，是为了让推导式内部的临时变量不污染外层代码（Python 2 中会泄漏，是著名痛点）；异常变量删除是为了尽早释放大对象内存（异常对象可能携带巨大的 traceback）；类被查找跳过，是因为类内部名字的意义本来就是"属性"，需要通过对象访问，避免隐式混乱。

·解决什么实际问题：写 GUI 回调或生成器时，循环变量是否泄漏直接影响行为；处理大异常时，except as 变量的自动删除能避免意外的内存滞留——这些细节在真实项目中会"莫名其妙"地冒出来，知道规则才能解释现象。

·初学者误区：①以为 for 循环变量也局部化——不，for i in range(3) 之后 i 在外层还活着，值为 2；②以为 exce

pt as 变量可以随便长期使用——块结束它就没了；③以为类内部可以像函数那样引用外围函数的名字——类被跳过，方法里要用类属性得写 `self.x` 或 `ClassName.x`。

17.6 作用域示例 (Scopes Examples)

英文原文

But enough theory! Let's step through a live example that demos scope ideas. Suppose we wrote the code in Example 17-1 and saved it in a module file named `scopes101.py`. Example 17-1. `scopes101.py`

```
python # Global scope X = 99 # X and func assigned
in module: global def func(Y): # Y and Z assigned in f
unction: locals # Local scope Z = X + Y # X is a global
when referenced here
```

```
return Z func(1) # func in module: result=100 (not pri
nted)
```

To see this example's result, import and call its function; the file's global `func` is a module attribute to imports (be sure to run this in the same folder as the file if it matters for imports in your REPL, as introduced in Chapter 3):

```
python >>> import scopes101 >>> scopes101.func(
1) 100
```

This module and the function it contains use a number of names to do their business. Applying Python's s

cope rules, we can classify these names as follows: Global names: `X`, `func X` is global because it's assigned at the top level of the module file; it can be referenced inside the function as a simple unqualified variable without being declared global. `func` is global for the same reason; the `def` statement makes a function object and assigns it to the name `func` at the top level of the module. Local names: `Y`, `Z` `Y` and `Z` are local to the function (and exist only while the function runs) because they are both assigned values in the function definition: `Z` by virtue of the `=` statement, and `Y` because arguments are always passed by assignment. Hence both are assigned during a function call, and both are locals. The underlying rationale for this name-segregation scheme is that local variables serve as temporary names that you need only while a function is running. For instance, in the preceding example, the argument `Y` and the addition result `Z` exist only inside the function; these names don't interfere with the enclosing module's namespace—or any other function, for that matter. In fact, local variables are removed from memory when the function call exits, and objects they reference may be garbage-collected if not referenced elsewhere. This is an automatic internal step, but it helps minimize memory requirements. The local/global distinction also makes functions easier to understand, as most of the names a function uses appear in the function itself, not at an arbitrarily distant place

in a module file. Also, because you can be sure that local names will not be changed by some remote function in your program, they tend to make programs easier to debug and modify. In the following sort of code, for instance, each function's variable cannot be changed—or even seen—anywhere else:

```
python def func1(): X = 'hack' # This X...
```

```
def func2(): X = 'code' # ...is not the same as this X
```

The net effect helps make functions self-contained units of software, by design.

中文翻译

理论够了！让我们用一个现场示例来演示作用域思想。假设我们编写了示例 17-1 中的代码，并把它保存为名为 `scopes101.py` 的模块文件。示例 17-1. `scopes101.py`

```
python
# 全局作用域X = 99 # X 和 func 在模块中赋值：全局
def func(Y): # Y 和 Z 在函数中赋值：局部
# 局部作用域
Z = X + Y # 在此处引用 X 时，X 是全局的
return Z
func(1) # 模块中的 func：结果=100（未打印）
```

想看这个示例的结果，可以 `import` 并调用它的函数；文件中的全局 `func` 对导入者来说是模块属性（如果在你的 REPL 中 `import` 位置很重要，请确保在与文件相同的文件夹中运行，正如第 3 章介绍的那样）：

```
python >>> import scopes101
>>> scopes101.func(1)
100
```

这个模块及其包含的函数为了完成工作使用了多个名字。套用 Python 的作用域规则，我们可以把这些名字分类如下：

- 全局名字：X、func

X 是全局的，因为它在模块文件顶层被赋值；在函数内部可以把它作为简单的非限定变量直接引用，无需声明为 `global`。func 是全局的

也是同理：def 语句创建了一个函数对象，并在模块顶层把它赋值给名字 func。局部名字：Y、Z Y 和 Z 是函数的局部变量（只在函数运行期间存在），因为它们在函数定义中都被赋了值：Z 是通过 = 语句赋值的，Y 则是因为参数总是通过赋值传递的。因此两者都在函数调用期间被赋值，都属于局部变量。这种名字隔离方案的底层逻辑是：局部变量充当临时名字，只在函数运行期间需要。例如在上面的示例中，参数 Y 和加法结果 Z 只存在于函数内部；这些名字不会干扰外围模块的命名空间——也不会干扰任何其它函数。事实上，函数调用退出时局部变量会从内存中移除，它们引用的对象如果没有被其它地方引用，就可能被垃圾回收（garbage-collected）。这是一个自动的内部步骤，但它有助于把内存需求降到最低。局部/全局的区分还让函数更容易理解，因为函数使用的大多数名字都出现在函数本身之中，而不是位于模块文件中某个随意遥远的角落。此外，由于你可以确信局部名字不会被程序中某个遥远的函数修改，它们往往让程序更容易调试和修改。例如在下面这类代码中，每个函数的变量在任何其它地方都无法被修改——甚至无法被看到：python def func1(): X = 'hack' # 这个 X... def func2(): X = 'code' # ...不是那个 X 净效果是：按设计，函数成为了自包含的软件单元。

深度理解

- 核心概念：一次完整的“名字分类演练”——把模块文件里的每个名字按赋值位置归类。X、func 是全局（模块顶层赋值）；Y（参数）、Z（函数体内赋值）是局部。
- 底层实现：func(1) 被调用时，解释器为这次调用分配栈帧，Y 和 Z 写入栈帧的局部槽。函数返回时栈帧被弹出，

局部变量随之消失；Z 引用的整数 100 若没有其他引用，引用计数归零即被回收。而 X、func 一直活在模块字典里，除非程序结束或显式删除。

·为什么这样设计：局部变量是“用完即走”的临时工，全局变量是“常驻岗位”——把临时数据放栈帧（速度快、自动回收），把长期数据放模块字典（生命周期长），这是“哪里需要往哪放”的资源管理智慧。

·解决什么实际问题：你可以确信 func1 里的 X 不会因为某个角落的函数运行而变成别的值——这种“确定性”让调试和并行修改代码成为可能；也解释了为什么“局部变量是默认选择”：它天然安全的。

·初学者误区：①以为 func(1) 那行会让 REPL 打印 100——不，模块顶层调用函数的结果若没赋给变量或没 print，就不会显示；②以为参数 Y 是“特殊变量”——它跟函数体内赋值的 Z 地位完全相同；③以为局部变量消失 = 对象立刻销毁——只是“可能”被垃圾回收，若对象还被外部引用（比如被返回），它仍然活着（这正是闭包的前奏）。

·生活例子：局部变量像考试草稿纸（用完就扔），全局变量像家里的工具箱（常年备着）——草稿纸写坏了不会影响工具箱里的东西，反过来工具箱里的扳手你可以随时拿来用（读全局），但想往工具箱里放新东西得打声招呼（global 声明）。

17.7 内置作用域 (The Built-in Scope)

英文原文

We've been talking about the built-in scope in the abstract, but it's a bit simpler than you may think. Really, the built-in scope is just a built-in module called `builtins`, but you have to import `builtins` to query built-ins because the name `builtins` is not itself a built-in... Yes, seriously! The built-in scope is implemented as a standard-library module named `builtins`, but that name itself is not placed in the built-in scope, so you have to import it in order to inspect it. Once you do, you can run a `dir` call to see which names are predefined (run this on your own for full fidelity: it can vary across Python versions):

```
python >>> import builtins >>> dir(builtins) ['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BaseExceptionGroup', 'BlockingIOError', 'BrokenPipeError', 'BufferError',
```

... many more names omitted: 158 total in 3.12

```
...'ord', 'pow', 'print', 'property', 'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars', 'zip']
```

The names in this list constitute the built-in scope in Python. Roughly the first half are built-in exceptions, and the second half are built-in functions. Also in this

list are the special names None, True, False, and Ellipsis we met earlier. Because Python automatically searches this module last in its LEGB lookup (it's B), you get all the names in this list "for free." That is, you can use them without importing any modules. Thus, there are really two ways to refer to a built-in function—by taking advantage of the LEGB rule, or by manually importing the builtins module:

```
python >>> zip # The normal way, per LEGB's B <class'zip'>
>>> import builtins # The hard way: for customizations
>>> builtins.zip <class'zip'>
>>> zip is builtins.zip # Same object, different lookup routes
True
```

The second of these approaches, though more to type, can be useful in advanced roles you'll meet in the sidebar "Why You Will Care: Customizing open". Redefining built-in names: For better or worse Careful readers might notice that because the LEGB lookup procedure takes the first occurrence of a name that it finds, names in the local scope may override variables of the same name in both the global and built-in scopes, and global names may override built-ins. A function can, for instance, create a local variable called open by assigning to it:

```
python def hider(): open = 'text' # Local variable, hides built-in here (L before B) ... open('data.txt') # Error:
this no longer opens files in this scope!
```

However, this will hide the built-in function called `open` that lives in the built-in (outer) scope, such that the name `open` will no longer work within the function to open files—it's now a string, not the opener function. This is sometimes called "shadowing" a built-in. This isn't a problem if you don't need to open files in this function, but triggers an error if you attempt to open through this name in this scope; your `open` is no longer the built-in `open`. This can even occur more simply at the interactive prompt, which works as a global, implicit-module scope:

```
python >>> open = 99 # Assign in global scope, hides built-in here too
```

That being said, there is nothing inherently wrong with using a built-in name for variables of your own, as long as you don't need the original built-in version. After all, if these were truly off limits, we would need to memorize the entire built-in names list and treat all its names as reserved. With some 150 usable names in this module in Python 3.12, that would be far too restrictive and daunting:

```
python >>> len(dir(builtins)), len([x for x in dir(builtins) if not x.startswith('_')]) (158, 150)
```

In fact, there are times in advanced programming where you may really want to replace a built-in name by redefining it in your code—to define a custom `open` that augments access attempts, for instance (again, more in the sidebar at the end of the chapter). Except

ion: the special names `None`, `True`, and `False` in the built-in scope are also treated as reserved words today, so they can no longer be reassigned (fun though it once was!). All other built-in names, though, are fair game. Nevertheless, redefining a built-in name is often a bug, and a nasty one at that, because Python will not issue a warning message about it. You may find tools on the web that can warn you of such mistakes, but knowledge may be your best defense on this point: don't redefine a built-in name you need. If you accidentally reassign a built-in name at the interactive prompt this way, though, you can either restart your session or run a `del name` statement to remove the redefinition from your scope, thereby restoring the original in the built-in scope per LEGB. Note that functions can similarly hide global variables of the same name with locals, but this is more broadly useful, and in fact is much of the point of local scopes—because they minimize the potential for name clashes, your functions are self-contained namespaces:

```
python X = 88 # Global X
def func(): X = 99 # Local X:
    hides global, but we want this here
func()
```

```
print(X) # Prints 88: unchanged
```

Here, the assignment within the function creates a local `X` that is a completely different variable from the global `X` in the module outside the function. As a consequence, though, there is no way to change a name outside a function without adding a global or nonlocal declaration to the de

f; the next section takes up the former. NOTE More built-in tongue twisters: Technically, the name `builtins` is preset in most global scopes, including the interactive session, to reference the module known as `builtins`, so you can often use `builtins` without an `import`, but cannot run an `import` on the name `builtins` itself—it's a preset variable, not a module's name. Thus, `builtins is builtins` is `True` after you `import builtins`. The upshot is that we can often inspect the built-in scope by simply running `dir(builtins)` sans imports, but are advised to use `builtins` for real work and customization. Who said documenting this stuff was easy?

中文翻译

我们一直在抽象地谈论内置作用域，但它其实比你想象的要简单。实际上，内置作用域只是一个名为 `builtins` 的内置模块，但你要先 `import builtins` 才能查询内置名字，因为 `builtins` 这个名字本身并不是内置的.....是的，说真的！内置作用域是以一个名为 `builtins` 的标准库模块实现的，但这个名字本身并没有被放进内置作用域，所以你必须导入它才能查看。导入之后，你可以运行一次 `dir` 调用来看看哪些名字是预定义的（为了完整起见请自行运行：它会随 Python 版本不同而变化）：

```
python >>> import builtins >>> dir(builtins) ['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BaseExceptionGroup', 'BlockingIOError', 'BrokenPipeError', 'BufferError', ...省略更多名字: 3.12 中共 158 个... 'ord', 'pow', 'print', 'property', 'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice',
```

'sorted','staticmethod','str','sum','super','tuple','type','vars','zip'] 这个列表中的名字构成了 Python 的内置作用域。大约前一半是内置异常，后一半是内置函数。列表中还有我们之前见过的特殊名字 None、True、False 和 Ellipsis。因为 Python 在 LEGB 查找中总是最后搜索这个模块（它是 B 层），列表里的所有名字你都可以"免费"使用——也就是说，不用导入任何模块就能使用它们。因此，引用一个内置函数实际上有两条路：利用 LEGB 规则，或者手动导入 builtins 模块：python >>> zip # 常规方式，靠 LEGB 的 B 层<class'zip'> >>> import builtins # 费劲的方式：用于定制>>> builtins.zip <class'zip'> >>> zip is builtins.zip # 同一个对象，不同的查找路径True 第二种方式虽然要多打几个字，但在"为什么你会关心：定制 open"这个侧边栏里你会遇到的高级角色中很有用。重定义内置名字：好还是坏（For better or worse）细心的读者可能已经注意到：由于 LEGB 查找过程取它找到的名字的第一个出现位置，局部作用域中的名字可能覆盖全局和内置作用域中的同名变量，全局名字也可能覆盖内置名字。例如，函数可以通过赋值创建名为 open 的局部变量：python def hider(): open = 'text' # 局部变量，在这里遮蔽了内置（L 在 B 之前）... open('data.txt') # 错误：在这个作用域里它不再能打开文件了！然而，这会遮蔽（shadow）居住在（外层）内置作用域中的那个叫 open 的内置函数，导致函数内 open 这个名字不再能用来打开文件——它现在是个字符串，而不是打开文件的函数。这有时被称为"遮蔽内置名字"。如果你在这个函数中不需要打开文件，那就没问题；但如果你试图通过这个名字打开文件，就会触发错误——你的 open 不再是内置的 open 了。这种情况甚至在交互提示符下也能

更简单地发生，因为交互提示符是一个全局的、隐式模块作用域：`python >>> open = 99` # 在全局作用域赋值，在这里也遮蔽了内置变量。尽管如此，只要不需要原来的内置版本，用内置名字作为你自己的变量名并没有任何本质上的错误。毕竟，如果这些名字真的被禁用了，我们就得把整个内置名字列表背下来，并把所有名字当作保留字。Python 3.12 的这个模块里有大约 150 个可用名字，那限制就太过分、也太吓人了：`python >>> len(dir(builtins)), len([x for x in dir(builtins) if not x.startswith('_')])` (158, 150) 事实上，在高级编程中有些时候你可能真的想通过在自己的代码中重定义来替换内置名字——比如定义一个增强访问尝试的自定义 `open`（本章末尾的侧边栏还有更多）。例外情况：内置作用域中的特殊名字 `None`、`True` 和 `False` 如今也被当作保留字处理，所以它们不能再被重新赋值（虽然这在当年曾经很有趣！）。除此之外的所有内置名字，都是允许使用的。尽管如此，重定义内置名字通常是个 bug，而且是个很讨厌的 bug，因为 Python 不会就此发出任何警告。你也许能在网上找到一些能警告这类错误的工具，但在这一点上，知识可能是你最好的防线：不要重定义你需要的内置名字。不过，如果你在交互提示符下这样意外地重新赋值了一个内置名字，你可以重启会话，或者运行 `del name` 语句从你的作用域中移除这个重定义，从而按照 LEGB 规则恢复内置作用域中的原版。注意：函数也可以用局部变量遮蔽同名的全局变量，但这在更大范围内是有用的，事实上这正是局部作用域存在的很大一部分意义——因为局部作用域把名字冲突的可能性降到最低，你的函数就是自包含的命名空间：`python X = 88` # 全局 X `def func(): X = 99` # 局部 X：遮蔽了全局，但我们这里就需要它 `func()` `print(X)` # 打印

88: 未受影响这里，函数内的赋值创建了一个局部 X，它与函数外模块里的全局 X 是完全不同的变量。但由此带来的后果是：如果不给 def 加上 global 或 nonlocal 声明，就没有办法修改函数之外的名字；下一节讨论前者。注意更多内置绕口令：从技术上说，builtins 这个名字在大多数全局作用域中（包括交互会话）都被预设为引用名为 builtins 的模块，所以你常常可以不用 import 就直接使用 builtins，但不能对 builtins 这个名字本身执行 import——它是一个预设变量，不是模块名。因此，导入 builtins 之后，builtins is builtins 为 True。结论是：我们常常可以不用任何 import 就直接运行 dir(builtins) 来检查内置作用域，但真正的工作和定制建议使用 builtins。谁说记录这些东西是容易的？

深度理解

- 核心概念：内置作用域 = builtins 模块（一个真实的、可导入的标准库模块）。LEGB 的 B 层是“最后兜底”的免费资源池。同名遮蔽（shadowing）是 LEGB“找到即停”的必然副产品。
- 底层实现：CPython 中每个模块的 builtins 变量指向 builtins 模块（模块内直接可用）；编译代码对象时，如果名字既不是局部、也没有外围绑定、也不是模块全局，字节码指令 LOADGLOBAL 在模块字典里找不到时，会退而查询 builtins 模块字典（PyEvalBuiltinsFromGlobals）。zip is builtins.zip 为 True 证明两条路径到达的是同一个函数对象。
- 为什么这样设计：把“你不需要 import 就能用”的名字集

中放在一个显式模块里，是为了：(1) 让语言自身可定制——改 `builtins.open` 就能全局替换文件打开行为（侧边栏的 `makeopen` 正是这么做的）；(2) 让 `None/True/False` 之外的名字不占用保留字名额——150 个名字全保留会让语言不堪忍受。

·解决什么实际问题：调试时用 `dir(builtins)` 查"这个名字是不是 Python 自带的"；审计代码时警惕函数内部对 `open`、`list`、`type` 等的遮蔽；`builtins` 让工具代码（如 `REPL` 增强、调试器）无需 `import` 即可拿到内置命名空间。

·初学者误区：①以为 `builtins` 能 `import`——它是预设变量，`import builtins` 会报错；②以为内置名字是保留字——只有 `None/True/False/Ellipsis`（及关键字）不可重新赋值，其他都可以但应谨慎；③以为遮蔽只影响"这一个函数"——在交互提示符全局作用域赋值 `open = 99` 会影响整个会话，重启或 `del open` 才能恢复。

代码分析

```
def hider():
    open = 'text'                # L B
    ...
    open('data.txt')            #
```

解释器如何处理：编译 `hider` 时，解释器发现 `open` 被赋值 (`open = 'text'`)，于是 `open` 被标记为 `hider` 的局部变量。之后函数体内任何 `open(...)` 调用都在局部作用域里先命中 `open`——它是一个字符串，`'text'('data.txt')` 会抛出 `TypeError:'str' object is not callable`。整个 `LEGB` 查找链根本走不到 `B` 层。同理 `X = 99` 的版本：`func`

内 X 是局部，func() 后全局 X 仍为 88。

设计思想：遮蔽规则是"就近原则"的自然结果，它既允许合法的局部遮蔽（你的函数可以放心用 X、list 等名字），也把误用成本（静默失效）留给了程序员——Python 选择信任你，但你要记住"别重定义你需要的内置名字"。

17.8 global 语句 (The global Statement)

英文原文

The global statement and its nonlocal cousin are the only declaration statements in Python that are actually used by Python (for contrast, see Chapter 6's discussion of unused type hinting). They are not type or size declarations, though; they are namespace declarations. The global statement, for instance, tells Python that a function plans to change one or more global names—that is, names that live in the enclosing module's scope (namespace). We've talked about global in passing already. Here's a summary:

- Global names are variables assigned at the top level of the enclosing module file.
- Global names must be declared only if they are assigned within a function.
- Global names may be referenced within a function without being declared, per the LEGB rule. In other words, global allows us to change names that live outside a def at the top level of a module file. As you'll see later, the nonlocal statement is almost identical but applies to names in

an enclosing def's local scope, rather than names in the enclosing module. The global statement, usable in def but not lambda, simply consists of the reserved word global, followed by one or more names separated by commas. All the listed names will be mapped to the enclosing module's scope when assigned or referenced within the function body. For instance, the following is a takeoff on the preceding example:

```
python X = 88 # Global X
def func():
    global X
    X = 99 # Global X: outside def func()
```

print(X) # Prints 99

We've added a global declaration to the example here, such that the X inside the def now refers to the X outside the def; they are the same variable this time, so changing X inside the function changes the X outside it. Here is a slightly more involved example of global at work:

```
python y, z = 1, 2 # Global variables in module

def allglobal():
    global x # Declare globals assigned x
    = y + z # No need to declare y or z: LEGB rule
```

Here, x, y, and z are all globals inside the function allglobal. Names y and z are global because they aren't assigned in the function, and x is global because it was listed in a global statement to map it to the module's scope explicitly. Without the global here, x would be considered local by virtue of the assignment. Notice that y and z are not declared global; Python's LEG

B lookup rule finds them in the module (G) automatically. Also notice that `x` does not even exist in the enclosing module before the function runs; in this case, the first assignment in the function creates `x` in the module. All of which works when needed, but you really should try to avoid using globals like this whenever possible—as the next section will explain.

中文翻译

`global` 语句和它的表亲 `nonlocal` 是 Python 中仅有的两个真正被 Python 使用的声明语句（作为对照，参见第 6 章关于未使用的类型标注（type hinting）的讨论）。不过，它们不是类型或大小声明，而是命名空间声明（namespace declaration）。例如，`global` 语句告诉 Python：这个函数打算修改一个或多个全局名字——也就是居住在外围模块作用域（命名空间）中的名字。我们已经顺带提到过 `global`。这里做一个总结：

- 全局名字是在外围模块文件顶层赋值的变量。
- 全局名字只有在函数内部被赋值时才需要声明。
- 按照 LEGB 规则，全局名字在函数内可以不经声明就直接引用。换句话说，`global` 允许我们修改位于 `def` 之外、模块文件顶层的名字。稍后你会看到，`nonlocal` 语句几乎完全类似，但它作用于外围 `def` 局部作用域中的名字，而不是外围模块中的名字。

`global` 语句可用于 `def` 但不能用于 `lambda`，它很简单：保留字 `global` 后跟一个或多个用逗号分隔的名字。所有列出的名字在函数体内被赋值或引用时，都会被映射到外围模块的作用域。例如，下面是对前面例子的改编：

```
python X = 88 # 全局 X
def func():
    global X
    X = 99 # 全局 X: 位于 def 之外
func()
print(X) #
```

打印 99 我们在这里给示例加上了 `global` 声明，于是 `def` 内部的 `X` 现在指的就是 `def` 外部的 `X`；这次它们是同一个变量，所以在函数内修改 `X` 也会修改函数外的 `X`。下面是一个稍复杂一点的 `global` 实战示例：`python y, z = 1, 2 # 模块中的全局变量`
`def allglobal(): global x # 声明被赋值的全局`
`x = y + z # 无需声明 y 或 z: LEGB 规则在这里，x、y、z 在函数 allglobal 内部都是全局的。y 和 z 是全局的，因为它们在函数中未被赋值；x 是全局的，因为它被列在 global 语句中，被显式映射到模块作用域。如果这里没有 global，x 就会因为赋值而被视为局部变量。注意 y 和 z 没有声明为 global；Python 的 LEGB 查找规则会自动在模块（G）中找到它们。还要注意：函数运行之前，外围模块中甚至还不存在 x；这种情况下，函数中的第一次赋值会在模块中创建 x。这些机制在需要时都能正常工作，但你应该尽量在一切可能的情况下避免这样使用全局变量——下一节会解释原因。`

深度理解

·核心概念：`global` 是"改名换姓"的声明——把函数内的赋值操作"重定向"到模块命名空间。它是 Python 仅有的两条声明语句之一（另一条是 `nonlocal`），作用是声明命名空间归属，与类型/大小无关。

·底层实现：编译函数体时，出现在 `global` 语句里的名字被排除出"局部变量"集合，赋值的字节码指令从 `STOREF` AST 变为 `STOREGLOBAL`（引用则为 `LOADGLOBAL`）。运行时该名字的读写直接作用于模块字典。注意：`global x` 并不要求 `x` 已存在于模块中——运行时第一次赋值会

创建它（这与 `nonlocal` 不同，`nonlocal` 要求外围已有绑定）。

·为什么这样设计：默认"写进当前函数"是安全的；要写外部作用域必须显式声明——这是在语法层面"提高做错事的成本"，让程序员清楚地知道自己越过了边界。同时"读"仍然免费，保证日常复用全局配置/常量不繁琐。

·解决什么实际问题：模块级配置项（如 `DEBUG`、连接串）需要在某个函数里统一更新时用 `global`；跨函数共享计数器等。但本书强烈建议：能用参数传递就用参数传递，全局是"最后手段"。

·初学者误区：①忘记写 `global` 就直接 `X = ...`——结果创建了一个局部 `X`，全局毫无变化（无声 bug）；②以为 `global` 需要在函数外也声明——不用，声明只在函数内出现；③以为 `global x` 要求 `x` 预先存在——不要求，赋值会创建它；④在 `lambda` 里写 `global`——非法，`lambda` 函数体只能是表达式。

代码分析

```
y, z = 1, 2          #
def all_global():
    global x          #
    x = y + z         # y zLEGB
```

解释器如何处理：编译 `allglobal` 时，解释器看到 `global x`，把 `x` 标记为"模块级"名字；`y`、`z` 只被引用、未被赋值，且函数内没有更近的绑定，所以走 `LEGB`：L（函数局部）没有 → E（无外围函数）→ G（模块）找到 `y=1, z=2` → 计算 `1+2=3` → 指令 `STOREGLOBAL x` 把 3 写入模块

字典，x 成为模块的新全局变量。调用 `allglobal()` 后，模块中就有了 `x == 3`。

设计思想：三行代码演示了三条规则的配合：赋值位置（x 在函数内）→ 被 `global` 改写归属；引用（y、z）→ 靠 LEGB 自动向上找；"声明的是要改写的名字，而不是要读的名字"——读写分离，声明只针对写入。

17.9 程序设计：尽量减少全局变量 (Program Design: Minimize Global Variables)

英文原文

Functions in general, and global variables in particular, raise some larger design questions. How, for example, should functions communicate? Although some answers will become more apparent when you begin writing larger functions of your own, a few guidelines up front might spare you from problems later. In general, functions should rely on arguments and return values instead of globals, but this may not at all be obvious to newcomers to programming. By default, names assigned in functions are locals, so if you want to change names outside functions you have to write extra code (e.g., global statements). This is deliberate—you have to say more to do the potentially "wrong" thing. Although there are times when globals are useful (and even required), variables assigned in a `def` a

re local by default because that is normally the best policy. Changing globals can lead to well-known software-engineering problems: because the variables' values are dependent on the order of calls to arbitrarily distant functions, programs can become difficult to debug, or understand at all. Consider this module file, for example, which is presumably imported and used elsewhere:

```
python X = 99
def func1():
    global X
    # Change global X when called
    X = 88
def func2():
    global X
    # But so does this, and when?
    X = 77
```

Now, imagine that it is your job to modify or reuse this code. What will the value of X be here? Really, that question has no meaning unless it's qualified with a point of reference in time—the value of X is timing-dependent, as it depends on which function was called last (something we can't tell from this file alone). The net effect is that to understand this code, you have to trace the flow of control through the entire program. Hence, if you need to reuse or modify the code, you have to keep the entire program in your head all at once. In fact, you can't really use one of these functions without bringing along the other. They are dependent on—that is, coupled with—the global variable. And that is the problem with globals: they generally make code more difficult to understand and reuse than code consisting of self-contained functions that rely on locals. On the other hand, short of using tools like the nested scope closures covered ahead or OOP with classes covered

red later, global variables are the most basic way to retain shared state information—information that a function needs to remember for use the next time it is called. Local variables disappear when the function returns, but globals do not. As you'll see later, other techniques can achieve this, too, and allow for multiple copies of the retained information; but they are more complex than pushing values out to the global scope for retention in simple cases where this applies. Moreover, some programs designate a single module to collect shared globals; as long as this is expected, it is not as harmful. Programs that use multithreading for parallel processing also commonly depend on global variables—they become shared memory between functions running in parallel threads, and so act as a communication device.² For now, though, and especially if you are relatively new to programming, avoid the temptation to use globals whenever you can—they tend to make programs difficult to understand and reuse, and won't work for cases where one copy of saved data is not enough. That's why they are not the default in Python. Try to communicate with passed-in arguments and return values instead. Six months from now, both you and your coworkers may be glad you did.

中文翻译

函数总体而言，全局变量尤其如此，会引出一些更大的设计问题。例如，函数之间应该如何通信？虽然当你开始编写自己的更大的函数时，一些答案会变得更加明显，但几条先行的指导原则也许能让你免于日后的问题。一般来说，函数应该依赖参数（arguments）和返回值（return values）而不是全局变量，但这对于编程新手来说可能一点也不明显。默认情况下，函数内赋值的名字是局部的，所以如果你想修改函数之外的名字，就必须编写额外的代码（例如 global 语句）。这是刻意为之——要做“可能错误”的事，你必须多说话。虽然有些时候全局变量是有用的（甚至是必需的），但 def 中赋值的变量默认为局部，正是因为这通常是最佳策略。修改全局变量会带来众所周知的软件工程问题：因为变量的值取决于对任意远处函数的调用顺序，程序会变得难以调试，甚至难以理解。例如，考虑下面这个模块文件（它大概会被别处导入并使用）：

```
python X = 99
def func1():
    global X # 被调用时修改全局 X
X = 88
def func2():
    global X # 但它也会修改，而且是在什么时候？
X = 77
```

现在，想象一下你的任务就是修改或复用这段代码。这里的 X 会是什么值？说真的，如果不带时间参考点，这个问题毫无意义——X 的值是“与时机相关的”，它取决于哪个函数最后被调用（这一点单凭这个文件无法判断）。净效果是：要理解这段代码，你必须追踪整个程序的控制流。因此，如果你需要复用或修改这段代码，就必须把整个程序同时装进脑子里。事实上，你无法只使用其中一个函数而不牵扯到另一个。它们都依赖——也就是与——那个全局变量耦合（coupled）。这就是全局变量的麻烦所在：与由依赖局部的自包含函数构成的代码相比，它们通常让代码更难理解和复用。另一方面，除了使用后面要讲的嵌套作用域闭包或后面要讲的

基于类的 OOP 之外，全局变量是保留共享状态信息的最基本方式——状态信息就是函数需要在下次调用时记住的信息。局部变量在函数返回时消失，全局变量则不会。稍后你会看到，其它技术也能做到这一点，而且允许保留信息的多个副本；但在适用的简单场景中，它们比"把值推到全局作用域去保留"更复杂。此外，有些程序会指定一个专门的模块来收集共享全局变量；只要这是被预期的行为，危害就没那么大。使用多线程（multithreading）做并行处理的程序也通常依赖全局变量——全局变量成为并行线程中运行的函数之间的共享内存，从而充当通信工具。不过就目前而言，尤其如果你相对来说是编程新手，请尽量抵制使用全局变量的诱惑——它们往往让程序难以理解和复用，而且在"保存数据的单个副本不够用"的情况下也无法工作。这正是它们不是 Python 默认选择的原因。试着改用传入的参数和返回值来通信吧。六个月后，你和你的同事可能都会庆幸当初这么做。

深度理解

·核心概念：这是一节"设计课"——功能上可行不等于设计上合理。全局变量让函数的可理解性、可复用性、可测试性全面下降，其根源是隐式耦合：函数的行为依赖全局状态，而全局状态依赖调用顺序。

·底层实现：纯语言机制层面并没有任何东西阻止你滥用全局变量——解释器不会警告。问题出在"人的认知"层面：模块编译后，X 只是一个字典条目，谁最后一次 STORE GLOBAL 谁就定值；而你无法从单个函数看出"最后一次"发生在哪。

- 为什么这样设计：Python 的默认选择（函数内赋值即局部）本身就是设计表态：把"需要显式声明才能改变外部"作为防线。书里那句"you have to say more to do the potentially'wrong' thing"（要做可能错误的事，必须说更多）就是设计哲学。
- 解决什么实际问题：大项目里最痛苦的调试场景之一就是"某个全局变量在某个角落被改掉了"。改用参数传入/返回值传出后，每个函数都是"输入→输出"的黑盒，可以独立测试、独立复用。
- 初学者误区：①以为"少打字"等于"好设计"——为图省事用全局，省下的字最后会在调试时加倍还回来；②以为"只要不写 global 就安全"——引用全局做"读"虽免声明，但 `X = X + 1` 这种读写混合就会立刻踩到 `UnboundLocalError`；③以为状态保留必须用全局——本章后半部分（non local、函数属性、类）给出了更优的多副本方案。
- 生活例子：全局变量就像办公室的公共冰箱——谁都能往里面放东西、换东西，最后没人知道那盒酸奶是谁的、什么时候过期（值取决于"最后一次是谁动过"）。自包含函数则像自己的便当盒：吃什么自己决定，出问题也找得到人。

17.10 程序设计：尽量减少跨文件修改（Program Design: Minimize Cross-File Changes）

英文原文

While we're on the subject of globals, here's another related design note: although we can change global variables in another file directly, we usually shouldn't. Module files were introduced in Chapter 3 and are covered in depth in the next part of this book, but their basics are simple. To demo their relationship to scopes, consider these two module files:

```
python # first.py
X = 99 # This code doesn't know about second.py
python # second.py
import first
print(first.X) # OK: references a name in another file
first.X = 88 # But changing it can be too subtle and implicit
```

The first defines a variable `X`, which the second prints and then changes by assignment. Notice that we must import the first module into the second to get to its variable at all—as we've learned, each module is a self-contained namespace (package of variables), and we must import one module to see inside it from another. That's the main purpose of modules: by segregating variables on a per-file basis, they avoid name collisions across files, in much the same way that local variables avoid name clashes across functions. Really, though, in terms of this chapter's topic, the global scope of a module file becomes the attribute namespace of the module object once it is imported—importers automatically have access to all of the file's global variables, because a file's global scope morphs into an object's attribute nam

espace when it is imported. After importing the first module, the second module prints its variable and then assigns it a new value. Referencing the module's variable to print it is fine—this is how modules are linked together into a larger system normally. The problem with the assignment to `first.X`, however, is that it is far too implicit: whoever's charged with maintaining or reusing the first module probably has no clue that some arbitrarily far-removed module on the import chain can change `X` at runtime. In fact, the second module may be in a different folder, and so difficult to notice at all. Although such cross-file variable changes are always possible in Python, they are usually much more subtle than you will want. Again, this sets up too strong a coupling between two components—because the files are both dependent on the value of the variable `X`, it's difficult to understand or reuse one file without the other. Such implicit cross-file dependencies can lead to inflexible code at best, and surprising bugs at worst. Here again, and generally speaking, don't do that—the better way to communicate across file boundaries is to call functions, passing in arguments and getting back return values. In this specific case, we would probably be better off coding an accessor function to manage the change:

```
python # first.py
X = 99
def setX(new): # Accessors make external changes explicit
    global X # And can manage access in a single place
    X = new
# second.py
import first
first.setX(8
```

8) # Call the function instead of changing directly This requires more code and may seem like a trivial change, but it makes a huge difference in terms of readability and maintainability—when a person reading the first module by itself sees a function, that person will know that it is a point of interface and will expect the change to the X. In other words, it removes the element of surprise that is rarely a good thing in software projects. Although we cannot prevent cross-file changes from happening (sans obscure hacks that we'll omit here), common sense dictates that they should be minimized unless widely known across the program .

中文翻译

既然说到全局变量，这里还有一条相关的设计说明：虽然我们可以直接修改另一个文件中的全局变量，但通常不应该这样做。模块文件在第 3 章引入，将在本书下一部分深入讲解，但它的基础很简单。为了演示它与作用域的关系，考虑这两个模块文件：
python # first.py X = 99 # 这段代码并不知道 second.py 的存在
second.py import first print (first.X) # 可以：引用另一个文件中的名字
first.X = 88 # 但修改它就太隐晦、太隐式了
第一个文件定义了变量 X，第二个文件先打印它，再通过赋值修改它。注意：我们必须把第一个模块导入到第二个模块中，才能碰到它的变量——正如我们学到的，每个模块都是自包含的命名空间（变量包），要从另一个模块看到它内部，就必须导入它。这正是模块的主要目的：通过按文件隔离变量，避免跨文件的变量冲突。

突，方式与局部变量避免跨函数名字冲突如出一辙。不过，就本章主题而言，真正重要的是：模块文件的全局作用域在导入后变成了模块对象的属性命名空间——导入者自动就能访问该文件的所有全局变量，因为文件被导入时，其全局作用域就蜕变成了对象的属性命名空间。导入第一个模块后，第二个模块打印它的变量，然后给它赋新值。引用模块的变量来打印它完全没问题——这是模块链接成更大系统的常规方式。但问题在于对 `first.X` 的赋值：它太过隐式了——负责维护或复用第一个模块的人大概毫不知情，导入链上某个任意遥远的模块竟能在运行时修改 `X`。事实上，第二个模块可能位于不同的文件夹，让人根本难以察觉。虽然这类跨文件变量修改在 Python 中总是可能的，但它们通常比你想要的隐晦得多。这同样会在两个组件之间建立起过强的耦合——因为两个文件都依赖变量 `X` 的值，脱离其中一个就难以理解或复用另一个。这类隐式跨文件依赖，往好里说会导致代码不灵活，往坏里说会导致令人惊讶的 bug。这里依然是一般规律：别那样做——跨文件边界通信的更好方式是调用函数、传入参数并拿回返回值。在这个具体场景中，更好的做法大概是编写一个访问器函数（accessor function）来管理修改：

```
python # first.py X = 99 def setX(new): # 访问器让外部修改显式化
global X # 并且可以在单一位置管理访问
X = new # second.py import first first.setX(88) # 调用函数，而不是直接修改
这需要更多代码，看起来似乎只是无关紧要的小改动，但它对可读性和可维护性的影响是巨大的——当一个人单独阅读第一个模块并看到一个函数时，他会知道那是一个接口（interface）点，并预期到 X 会被修改。换句话说，它消除了“惊喜”因素——而惊喜在软件项目中很少是好事。虽然我们无法阻止跨文件修
```


改的发生（除非采用我们在此略过的晦涩 hack），但常识告诉我们：除非整个程序广为人知，否则这类修改应该尽量少。

深度理解

·核心概念：跨文件修改（`first.X = 88`）技术上可行，设计上要避免。模块作用域 = 模块对象属性命名空间，这是“作用域”与“属性”第一次接轨；`first.X` 走的是属性查找（对象），不是作用域查找。

·底层实现：导入模块 = 执行该模块文件 → 其全局字典变成 `sys.modules['first']`（模块对象）的 dict。`first.X = 88` 就是往这个字典里写条目，其他任何持此模块引用的代码都能看到变化——包括运行中的其他线程。没有运行时保护机制，全靠约定。

·为什么这样设计：Python 刻意允许这种操作，因为动态语言的灵活性有时必须保留（热补丁、测试替身等）。但语言允许 ≠ 鼓励；Lutz 给出的方案是用访问器函数把修改变成显式接口，让“谁在改、何时改、为什么改”变得可追踪。

·解决什么实际问题：大型系统中“隐性依赖”是最难排查的问题之一。把跨文件写入收敛到函数（`setX`），配合日志或断点，就能在修改发生时被定位；直接改属性则无处设防。

·初学者误区：①以为 `from first import X` 后再 `X = 88` 就能改 `first` 的 `X`——不能！那只是复制了一份引用并重新绑定，`first.X` 原封不动（这是经典大坑）；②以为 `imp`

ort 是"复制代码"——是执行并缓存模块对象，first.X 是实时别名；③以为只有 = 才算修改——first.L.append(...) 也是跨文件修改（只是作用于对象内部，通常可接受）

17.11 访问全局的其他方式 (Other Ways to Access Globals)

英文原文

Interestingly, because global-scope variables morph into the attributes of a loaded module object, we can emulate the global statement by importing the enclosing module and assigning to its attributes, as in the module file in Example 17-2. Code in this file accesses its enclosing module, first by importing it, and then by indexing the sys.modules dictionary that records all loaded modules and is used in advanced roles (there's more on this dictionary in Part V).

Example 17-2. thismod.py

```
python "Change a global three ways"
```

```
var = 99 # Global variable == module attribute
def local():
    var = 0 # Change local var
def glob1():
    global var # Declare global (normal) var
    var += 1 # Change global var
```

```
def glob2():
    var = 0 # Change local var
    import thismod # Import myself
    thismod.var += 1 # Change global
```

var

```
def glob3(): var = 0 # Change local var
import sys # Import system table
thismod = sys.modules['thismod'] # Get module object (or use name)
thismod.var += 1 # Change global var
```

```
def test(): print(var) local(); glob1(); glob2(); glob3()
print(var)
```

When we import and call this module's test to invoke its other functions, this adds 3 to the global variable var—only its first function, local, does not impact the global:

```
python >> import thismod >> thismod.test() 99 102
>> thismod.var 102
```

All these global-access techniques work, and illustrate the equivalence of the global scope to module attributes, but they're noticeably more work than using the global statement to make your intentions explicit.

As we've seen, global allows us to easily change names in a module outside a function. It has a close relative named nonlocal that can be used to change names in enclosing functions, too—but to understand how that can be useful, we first need to explore function nesting in general, the topic we turn to next.

中文翻译

有趣的是，因为全局作用域中的变量会蜕变成已加载模块对象的属性，我们可以通过导入外围模块并对其属性赋值来模

拟 `global` 语句，就像示例 17-2 中的模块文件那样。该文件中的代码访问自己的外围模块：先导入它，再对记录所有已加载模块的 `sys.modules` 字典做索引（关于这个字典，第 V 部分还有更多内容）。示例 17-2. `thismod.py`

```
python "用三种方式修改全局变量"
var = 99 # 全局变量 == 模块属性
def local():
    var = 0 # 修改局部 var
def glob1():
    global var # 声明 global (常规方式)
    var += 1 # 修改全局 var
def glob2():
    var = 0 # 修改局部 var
import thismod # 导入自己
thismod.var += 1 # 修改全局 var
def glob3():
    var = 0 # 修改局部 var
import sys # 导入系统表
thismod = sys.modules['thismod'] # 获取模块对象 (或使用 name)
thismod.var += 1 # 修改全局 var
def test():
    print(var)
    local()
    glob1()
    glob2()
    glob3()
    print(var)

当我们导入这个模块并调用它的 test 来调用其他函数时，全局变量 var 会被加上 3——只有它的第一个函数 local 不影响全局：
python >>> import thismod
>>> thismod.test()
99
102
>>> thismod.var
102
```

所有这些访问全局的技巧都能工作，并演示了全局作用域与模块属性之间的等价性，但它们显然比用 `global` 语句显式表达意图要麻烦得多。正如我们所见，`global` 让我们能轻松修改函数之外、模块中的名字。它有一个近亲叫 `nonlocal`，也可以用来修改外围函数中的名字——但要理解它为何有用，我们首先需要从总体上探索函数嵌套，这正是下一节的主题。

深度理解

·核心概念：三条“殊途同归”的全局访问路径：(1) `global` 声明（正常方式）；(2) 导入自身模块后 `thismod.var += 1`；(3) 通过 `sys.modules` 字典拿到模块对象再改属性。

三者都等价于写模块字典，证明了"全局作用域 == 模块属性命名空间"。

·底层实现：`import thismod` 会先在 `sys.modules` 中查找——模块已在缓存中，不会重复执行文件，直接返回现有模块对象。`sys.modules['thismod']` 只是换了一条路拿同一个对象。`var += 1` 是 `var = var + 1` 的复合形式：读旧值（从模块属性）、算新值、写回模块字典。注意 `glob2/glob3` 函数体里先 `var = 0`——这是必须的：如果不给 `var` 赋值，`var` 就是全局的（读到 99 再 +1 也是 100，看起来也对）；但一旦赋值，`var` 成为局部，`thismod.var` 才需要显式路径。这就是"任何赋值都归类名字"的活教材。

·为什么这样设计：`sys.modules` 的存在使"运行时获取任意已加载模块"成为可能——这是插件系统、动态重新加载（`importlib.reload`）、测试注入的基础设施。等价性本身也说明作用域规则与对象模型是统一的。

·解决什么实际问题：当你想"无 `global` 关键字修改模块变量"（例如第三方库限制、或代码生成场景）时，属性路径是逃生通道；理解 `sys.modules` 也有助于排查"为什么改了一个文件，另一个文件里没生效"的问题。

·初学者误区：①以为 `import thismod` 会重新执行模块文件——不会，模块只执行一次，之后查缓存；②以为这些技巧是"更高级"的写法——恰恰相反，Lutz 说它们"比 `global` 麻烦得多"，显式声明才是最佳实践；③分不清 `var = 0` 在函数里的效果——它把 `var` 变成局部变量，这正是这些例子必须先写 `var = 0` 的原因。

代码分析

```
"""
var = 99                                # ==

def local():
    var = 0                             # var

def glob1():
    global var                           # global
    var += 1                             # var

def glob2():
    var = 0                             # var
    import thismod                       #
    thismod.var += 1                     # var

def glob3():
    var = 0                             # var
    import sys                           #
    thismod = sys.modules['thismod']    # __name__
    thismod.var += 1                     # var

def test():
    print(var)
    local(); glob1(); glob2(); glob3()
    print(var)
```

解释器如何处理：test() 调用时：print(var) 走 LEGB 找到全局 var=99；local() 在自己的栈帧里创建局部 var=0 后返回（全局不变）；glob1() 中 global var 使 var += 1 编译为 LOADGLOBAL + BINARYOP + + STOREGLOBAL，全局变为 100；glob2() 中 import thismod 从 sys.modules 取回本模块对象，thismod.var += 1 是属性读写（LOADATTR/STOREATTR），全局变为 101；glo

b3() 同理从 `sys.modules['thismod']` 索引得到同一对象，变为 102。最终 `print(var)` 输出 102。

设计思想：`var = 0` 在 `glob2/glob3` 中不是可有可无的——它故意把局部 `var` 变成“替身”，强迫你走属性路径。这展示了作用域规则的“刚性”：一旦赋值，编译器就把名字钉死在局部，任何想改全局的意图都必须通过显式通道表达。

17.12 嵌套函数与作用域 (Nested Functions and Scopes)

英文原文

So far, this chapter has largely omitted one part of Python's scope rules on purpose, because it's relatively uncommon to encounter it in practice. However, it's time to take a deeper look at layer E in the LEGB look up rule. This layer was added during Python 2.X's reign. It takes the form of the local scopes of any and all enclosing function's local scopes. Enclosing scopes are sometimes also called statically nested scopes. Really, the nesting is a lexical one—nested scopes correspond to physically and syntactically nested code structures in your program's source code text.

中文翻译

到目前为止，本章刻意略过了 Python 作用域规则的一部分，因为它在实践中相对少见。不过，现在该深入看看 LEGB 查找规则中的 E 层了。这一层是在 Python 2.X 时代加入的。它的形态是：所有外围函数局部作用域（一层或多层）的局部作用域。外围作用域有时也被称为静态嵌套作用域（statically nested scopes）。其实，这种嵌套是词法（lexical）上的——嵌套作用域对应着你程序源代码文本中物理上、语法上嵌套的代码结构。

深度理解

- 核心概念：E 层是"函数套函数"带来的额外查找层。它叫"静态嵌套"或"词法嵌套"，是因为判定依据是源码文本的物理位置，而不是运行时调用栈。
- 底层实现：CPython 把被内层函数引用的外层局部变量提升为单元胞变量（cell variable）：外层函数的局部槽中存放一个 cell 对象（一层间接），内层函数通过 cell 读取。字节码层面：外层用 STOREDEREF/LOADDEREF，内层用 LOADDEREF 或 COPYFREEVARS。这个 cell 是"闭包机制"的物理载体，本章后面的 closure 属性就是它的暴露接口。
- 为什么这样设计：Python 2.1 之前没有 E 层（只有 LGB），嵌套函数想用外层变量得靠参数默认值"手动捕获"（17.21 节会细讲）。加入 E 层后，编译器自动完成捕获——这就是"默认值方案成为历史遗留"的原因。

·解决什么实际问题：闭包、工厂函数、装饰器（第 39 章）全部依赖 E 层。没有它，"函数返回另一个函数并记住环境"这种函数式编程的核心模式就无法成立。

·初学者误区：①以为 E 层与调用栈有关——完全无关，f1 调用完再调用 f2，f2 照样能找到 f1 的变量（靠 cell 存活）；②以为"词法"是难懂的黑话——它就是"看源码文本就能确定"的意思。

17.13 嵌套作用域概述 (Nested Scopes Overview)

英文原文

With the addition of nested function scopes, variable lookup rules become slightly more complex. Within a function:- A name reference (X) looks for the name X first in the current local scope (function); then in the local scopes of any lexically enclosing functions in your source code, from inner to outer; then in the current global scope (the module file); and finally in the built-in scope (which we've seen means the built-in module builtins). global declarations in a def make the search begin in the global (module file) scope instead.- A name assignment (e.g., X = value) creates or changes the name X in the current local scope, by default. If X is declared global within the function, the assignment creates or changes the name X in the enclosing

ing module's scope instead. If, on the other hand, `X` is declared nonlocal within a `def` function, the assignment changes the name `X` in the local scope of the closest enclosing function that assigns the same name. Notice that the global declaration still maps variables to the enclosing module. When nested functions are present, variables in enclosing functions may be referenced, but require nonlocal declarations to be changed. Also note that, unlike `global`, `nonlocal` does not create a name in an enclosing `def` if it's not assigned there; `nonlocal` makes sense only if the variables it names are also assigned by an enclosing `def`—and is an error to use otherwise. This section is primarily concerned with the first bullet in the preceding list: nested scope references. We'll explore nested assignments and `nonlocal` in a moment, but first we need to grok function nesting in general.

中文翻译

随着嵌套函数作用域的加入，变量查找规则变得稍微复杂一些。在函数内部：- 名字引用（`X`）会先在当前局部作用域（函数）中查找 `X`；然后在你源代码中所有词法上外围的函数局部作用域中查找，从内到外；再在当前全局作用域（模块文件）中查找；最后在内置作用域（我们已经知道它指内置模块 `builtins`）中查找。`def` 中的 `global` 声明会让查找改从全局（模块文件）作用域开始。- 名字赋值（例如 `X = value`）默认在当前局部作用域创建或修改名字 `X`。如果 `X` 在函数中被声明为 `global`，赋值就改在外围模块作用域中

创建或修改 X。另一方面，如果 X 在某个 def 函数中被声明为 nonlocal，赋值就修改离得最近的外围函数中赋值过该名字的局部作用域中的 X。注意 global 声明仍然把变量映射到外围模块。当存在嵌套函数时，外围函数中的变量可以被引用，但要修改它们就需要 nonlocal 声明。还要注意：与 global 不同，nonlocal 不会在外围 def 中不存在该名字赋值时创建它；nonlocal 只有在它列出的名字也由外围 def 赋值时才有意义——否则使用它就是错误。本节主要关注上面列表中的第一条：嵌套作用域的引用。我们稍后会探索嵌套赋值与 nonlocal，但首先需要从整体上理解函数嵌套。

深度理解

- 核心概念：嵌套后的规则是"引用逐层向上找 ($L \rightarrow E \rightarrow G \rightarrow B$)，赋值默认就近写 (L)，global 跳到 G ，nonlocal 跳到最近的 E 层同名绑定"。读多层、写一层。
- 底层实现：编译器对每个名字做绑定分析：若函数体有赋值且未声明 $\rightarrow$ local；若只引用、外围函数有赋值 $\rightarrow$ free variable (cell)；nonlocal 列表中的名字 $\rightarrow$ 标记为"来自外围"，赋值为 STOREDEREF。nonlocal 的"必须有外围绑定"是在编译期检查的 (no binding for nonlocal ... found 是 SyntaxError，不是运行时错误)。
- 为什么这样设计：global 允许"模块中不存在的名字由函数创建" (前面 allglobal 见过)，但 nonlocal 刻意不允许——因为外围函数有多层，名字该建在哪一层有歧义；强行规定"必须已存在"就把歧义消灭在编译期。

·解决什么实际问题：nonlocal 声明让你能安全地修改闭包状态，而不用把状态挪到模块级（避免全局污染）或退回可变对象 hack。

·初学者误区：①把 nonlocal 当 global 用——nonlocal state 要求 state 必须在外围函数里赋值过，否则 Syntax Error；②以为 nonlocal 是“声明一个变量”——它只声明“改哪个已有的”，不创建新名字；③忘记 E 层是多层的——nonlocal 会绑定到“最近的”那层，不是最外层。

17.14 嵌套作用域示例 (Nested Scopes Examples)

英文原文

Let's turn to code to illustrate some of the preceding points. Here is what an enclosing function scope looks like (type this into a script file or at the interactive prompt to run it live):

```
python X = 99 # Global scope
name: not used
def f1():
    X = 88 # Enclosing
    def local def f2():
        print(X + 1) # Reference made in nested
    def f2() f1() # Prints 89: enclosing def local + 1
```

First off, this is legal Python code: the def is simply an executable statement, which can appear anywhere any other statement can—including nested in another def. Here, the nested def runs while a call to the function f1 is running; it makes a function and assigns it to the name f2, a local variable within f1's local scope. In a sense

, f2 is a temporary function that lives only during the execution of (and is visible only to code in) the enclosing f1. As such, its name won't clash with any other part of the program—which is one reason to nest it this way. But notice what happens inside f2: when it uses the variable X, it refers to the X that lives in the enclosing f1 function's local scope. Because functions can access names in all physically enclosing functions, the X in f2 is automatically mapped to the X in f1, by the LEGB lookup rule's level E. In fact, this enclosing scope lookup works even if the enclosing function's call has already ended. For example, the following code defines a function that makes and returns another function, and introduces a common nesting role: python

```
def f1(): X = 88
def f2(): print(X + 1) # Remembers X in enclosing def scope
return f2 # Return f2 but don't call it
action = f1() # Make, return function
action() # Call it now: prints 89
```

In this code, the call to action is really running the function we named f2 when f1 ran. This works naturally because functions are objects in Python like everything else and can be passed back as return values from other functions. More subtly, f2 "remembers" the enclosing scope's X in f1—even though f1 is no longer active when f2 is run. Though abstract here, this memory behavior turns out to be one of the main reasons to nest functions, and warrants a closer look in the next section.

中文翻译

让我们用代码来说明前面的某些要点。下面就是外围函数作用域的样子（把它键入脚本文件或交互提示符中现场运行）

```
: python X = 99 # 全局作用域名字：不会用到def f1(): X = 88 # 外围 def 的局部def f2(): print(X + 1) # 在嵌套 def 中作出的引用f2() f1() # 打印 89: 外围 def 的局部 + 1
```

首先，这是合法的 Python 代码：def 就是一个可执行语句，可以出现在任何其他语句能出现的地方——包括嵌套在另一个 def 中。在这里，嵌套的 def 是在对函数 f1 的一次调用运行期间执行的；它创建一个函数并把它赋值给名字 f2——f1 局部作用域中的一个局部变量。从某种意义上说，f2 是一个临时函数，只在外围 f1 执行期间存在（并且只对其中的代码可见）。因此，它的名字不会与程序任何其他部分冲突——这正是要这样嵌套的原因之一。但请注意 f2 内部发生了什么：当它使用变量 X 时，它引用的是居住在外围 f1 函数局部作用域中的那个 X。因为函数可以访问所有物理上外围的函数中的名字，所以 f2 中的 X 会被 LEGB 查找规则的 E 层自动映射到 f1 中的 X。事实上，即使外围函数的调用已经结束，这种外围作用域查找依然有效。例如，下面的代码定义了一个函数，它制造并返回另一个函数，由此引入一种常见的嵌套角色：

```
python def f1(): X = 88 def f2(): print(X + 1) # 记住外围 def 作用域中的 X return f2 # 返回 f2 但不调用它action = f1() # 制造、返回函数action() # 现在调用它：打印 89
```

在这段代码中，对 action 的调用实际上是在运行 f1 执行期间我们命名为 f2 的那个函数。这能自然工作，是因为在 Python 中函数与其他一切一样都是对象，可以作为返回值从其他函数传回来。更微妙的是

， f2 "记得"外围作用域中 f1 的 X——即使运行 f2 时 f1 早已不再活跃。虽然这里还很抽象，但这种"记忆"行为正是嵌套函数的主要理由之一，值得在下一节仔细研究。

深度理解

·核心概念：两个示例揭示两个关键事实：①嵌套 def 只是一个可执行语句，运行时才创建函数对象；②内层函数引用外层变量靠 E 层自动映射，且外层调用结束后映射依然有效——这就是"记忆 (memory)"行为的起点。

·底层实现：当编译 f1 时，编译器发现 X 被内层 f2 引用，于是把 f1 的 X 存储到 cell 对象中 (STOREDEREF)，f2 的代码对象记录一个 free variable (自由变量)，通过 LOADDEREF 读取。f1() 调用结束时，f1 的栈帧被回收，但 cell 对象不随之消亡——只要 f2 函数对象还活着 (被 action 引用)，cell 里的 88 就还在。f2.closure 元组里就装着这个 cell。这就是"函数是对象、可被返回"与"环境被记住"的物理基础。

·为什么这样设计：函数式编程里"闭包"是核心能力：生成器式的延迟行为、事件回调、配置化工厂都依赖"函数自带环境"。Python 选择让"函数对象 + 环境 cell"打包存在，而不是像 C 语言那样函数指针只是裸地址。

·解决什么实际问题：GUI 回调 (每个按钮记住自己的处理状态)、延迟计算、柯里化 (部分参数固化) ——"返回的函数带着一份快照环境"。

·初学者误区：①以为 action() 调用的是"副本"——不是，它就是 f2 函数对象本身；②以为 f1 结束 = X 被销毁

——栈帧变量名没了，但 cell 里的对象仍被 f2 持有；③ 以为必须显式"传递"环境——Python 自动完成，nonlocal 只是让你能修改它。

代码分析

```
def f1():  
    X = 88  
    def f2():  
        print(X + 1)    # def X  
    return f2           # f2  
action = f1()           #  
action()                # 89
```

解释器如何处理：① f1() 调用：执行 X = 88，因为 X 是 cell 变量，指令是 STOREDEREF——88 被存入 cell 对象；② 执行嵌套 def f2：此刻才创建 f2 函数对象，它的 closure 被填上对那个 cell 的引用（MAKEFUNCTION 指令会捕获自由变量）；③ return f2：函数对象被赋给 action，f1 的栈帧弹出——但 cell 被 f2.closure 持有，88 存活；④ action()：执行 f2 的字节码，LOADDEREF X 从 cell 取出 88，+1 后打印 89。全程没有任何"全局"参与——全局的 X = 99 从未被读到。

设计思想：return f2 而不调用 f2()，是"工厂"模式的最简雏形：外层函数生产内层函数，内层函数自带环境。下次 f1() 再调用，会创建另一个全新的 cell 与 f2——互不干扰，这是"每次调用新局部作用域"规则的延续。

17.15 闭包与工厂函数 (Closures and Factory Functions)

英文原文

Depending on whom you ask, the preceding example's behavior is sometimes called a closure or a factory function—the former describing a functional programming technique, and the latter denoting a design pattern. In code, a factory function creates and returns a function that has a stateful closure, but the two terms are intertwined. Whatever the label, the function object in question remembers values in enclosing scopes regardless of whether those scopes are still active for a call. In effect, it has attached packets of memory (a.k.a. state retention), which are made anew for each copy of the nested function created, and often provide a simple alternative to classes in this role. Closures (a.k.a. factory functions) are sometimes used by programs that need to generate event handlers on the fly in response to conditions at runtime. For instance, imagine a GUI that must define actions according to user inputs that cannot be anticipated when the GUI is built and vary per action. In such cases, we need a function that creates and returns another function, with information (i.e., state) that differs per function made. To demo this live in simplified terms, consider the following function, typed at the interactive prompt:

```
python >>> def maker(N): ... def action(X): ... return
X N # action retains N from enclosing scope ... return
action This defines an outer function that simply mak
es and returns a nested function, without calling it—
maker makes action, but simply returns action withou
t running it. If we then call the outer function:
```

```
python >>> f = maker(2) # Pass 2 to argument N >>
> f <function maker.<locals>.action at 0x101ae7ba0
> What we get back is a reference to the nested func
tion built—the one created and assigned to action w
hen the nested def runs. If we now call what we got b
ack from the outer function:
```

```
python >>> f(3) # Pass 3 to action's X 9 # And maker
's N remembers 2: 3 2
```

```
>>> f(4) # Pass 4 to X, N is still 2: 4 2 16
```

We invoke the nested function—the one called `action` within `maker`. In other words, we're calling the nested function that `maker` created and passed back. Perhaps the most unusual part of this, though, is that the nested function remembers integer 2, the value of the variable `N` in `maker`, even though `maker` has returned and exited by the time we call `action`. In effect, `N` from the enclosing local scope is retained as state information attached to the generated `action`, which is why we get back its argument squared whenever it is later called. Just as important, if we now call the outer function again, we get back a new nested function

with different state information attached. In the following, we compute the argument cubed instead of squared when calling the new function, but the original still squares as before:

```
python >>> g = maker(3) # g remembers 3, f remembers 2
>>> g(4) # 4 3 64
>>> f(4) # 4 2 16
```

This works because each call to a closure/factory function like this gets its own set of state information. In our demo, the function we assign to name `g` remembers 3, and `f` remembers 2, because each has its own state information retained by the variable `N` in `maker`. In a GUI, `N` might be a username, email address, or other per-function state. This is all literal inside Python. Nested functions have an attached closure storage area for the enclosing scope names they use. It's available as the `closure` attribute of such functions (and fetching `cellcontents` after indexing this tuple yields a state item), but we don't need to understand Python internals to use closures in our code. This is also a somewhat advanced technique that you may not see commonly in most code, and may be more popular among programmers with backgrounds in functional programming languages. On the other hand, enclosing scopes are often employed by the lambda function-creation expressions introduced in the prior chapter—because lambda is an expression, it is almost always

nested in a def. For example, a lambda can serve in place of a def in our demo, and also relies on enclosing scope references to retain state—like N in the following variation:

```
python >>> def maker(N): ... return lambda X: X N # I
lambda functions retain state too >>> h, i = maker(2)
, maker(3) # Make two closure functions >>> h('Py'),
i('Py') # Run lambdas: ('Py' 2) and ('Py' 3) ('PyPy','PyPyPy')
```

For a more tangible example of closures at work, see the sidebar "Why You Will Care: Customizing open". It uses similar techniques to store information for later use in an enclosing scope. As you'll also see after the next section's closer, closures become more useful when their state becomes changeable with nonlocal.

中文翻译

取决于你问谁，前面示例的行为有时被称为闭包（closure）或工厂函数（factory function）——前者描述的是一种函数式编程（functional programming）技术，后者指的是一种设计模式（design pattern）。在代码中，工厂函数创建并返回一个带有有状态闭包的函数，但这两个术语是交织在一起的。无论标签是什么，这个函数对象都会记住外围作用域中的值，不管这些作用域对一次调用是否仍然活跃。实际上，它带有附着于自身的记忆包（又称状态保留（state retention）），每创建一个嵌套函数副本就重新制作一份，并且在这一角色中常常是类的简单替代方案。闭包（即工厂函数）有时会被需要"在运行时根据条件临时生成事件

处理器"的程序使用。例如，设想一个 GUI 必须根据用户输入来定义动作——这些输入在 GUI 构建时无法预知，而且每个动作各不相同。在这种情况下，我们需要一个"创建并返回另一个函数"的函数，每个被创建的函数携带不同的信息（即状态）。为了用简化术语现场演示，考虑下面这个在交互提示符下键入的函数：

```
python >>> def maker(N): ...  
def action(X): ... return X N # action 保留外围作用域中的 N ... return action 这定义了一个外层函数，它只是制造并返回一个嵌套函数，而不调用它——maker 制造 action，但只是返回 action 而不运行它。然后如果我们调用这个外层函数：  
python >>> f = maker(2) # 把 2 传给参数 N  
>>> f <function maker.<locals>.action at 0x101ae7ba0> 我们拿回的是对构建出来的嵌套函数的引用——也就是嵌套 def 执行时创建并赋给 action 的那个函数。现在如果我们调用从外层函数拿回的返回值：  
python >>> f(3) # 把 3 传给 action 的 X 9 # 而 maker 的 N 记住的是 2：3  
2 >>> f(4) # 把 4 传给 X，N 仍然是 2：4 2 16 我们调用的是嵌套函数——即 maker 内部那个叫 action 的函数。换句话说，我们调用的是 maker 创建并传回来的嵌套函数。不过，这里最不寻常的一点也许是：嵌套函数记住了整数 2——maker 中变量 N 的值——即使在我们调用 action 时 maker 早已返回并退出。实际上，来自外围局部作用域的 N 作为状态信息附着在生成的 action 上被保留下来，这正是为什么之后每次调用它，我们都能拿回其参数的平方。同样重要的是：如果我们再次调用外层函数，我们拿回的是一个带有不同状态信息的新嵌套函数。下面，调用新函数时我们计算的是参数的立方而不是平方，而原来的函数仍然像以前一样算平方：  
python >>> g = maker(3) # g 记住 3
```

，f 记住 2 >>> g(4) # 4 3 64 >>> f(4) # 4 2 16 这之所以有效，是因为像这样的闭包/工厂函数的每次调用都得到它自己的那套状态信息。在我们的演示中，赋给名字 g 的函数记住 3，f 记住 2——因为 maker 中的变量 N 为每个函数各自保留了自己的状态信息。在 GUI 中，N 可能是用户名、电子邮件地址或其他按函数区分的状态。这一切在 Python 内部都是字面真实的。嵌套函数带有一个附着的闭包存储区，用于存放它们使用的外围作用域名字。它可以通过这类函数的 closure 属性获得（对这个元组做索引后，再取 cellcontents 就能得到一个状态项），但我们在代码中使用闭包并不需要理解 Python 内部机制。这也是一项相当高级的技术，在大多数代码中你可能并不常见，它在有函数式编程语言背景的程序员中可能更流行。另一方面，外围作用域经常被上一章引入的 lambda 函数创建表达式利用——因为 lambda 是表达式，它几乎总是嵌套在 def 中。例如，在我们的演示中，lambda 可以代替 def，同样依赖外围作用域引用来保留状态——比如下面这个变体中的 N：python >>> def maker(N): ... return lambda X: X N # lambda 函数同样保留状态>>> h, i = maker(2), maker(3) # 制造两个闭包函数>>> h('Py'), i('Py') # 运行 lambda: ('Py' 2) 和 ('Py' 3) ('PyPy', 'PyPyPy') 想看到闭包实际工作的更具体示例，请看侧边栏“为什么你会关心：定制 open”。它用类似的技术在外围作用域中存储信息供以后使用。你还会在下一节末尾看到：当闭包的状态可以通过 nonlocal 被修改时，闭包会变得更加有用。

深度理解

·核心概念：闭包 = 函数对象 + 被捕获的外围环境 (cell

集合)。工厂函数是"生产闭包的函数"。每次调用工厂都产生一个携带独立状态的新函数——这是与类的核心区别：类实例要写一堆样板，闭包几行就够。

·底层实现： $f = \text{maker}(2)$ 时发生了什么？①执行 maker 的栈帧， N (cell 变量) 存入 cell 对象，值为 2；②MAKEFUNCTION 创建 action 函数对象，其 $\text{closure} = (\text{cell},)$ ，tuple 长度 1 正好对应 X 之外唯一的自由变量 N ；③ $f(3)$ 执行时 LOADDEREF N 从 cell 取 2，与参数 $X=3$ 相乘/乘方。验证： $f.\text{closure}[0].\text{cellcontents}$ 返回 2。而 $\text{maker}(3)$ 生成的是全新的 cell (新帧)，所以 $g.\text{closure}[0].\text{cellcontents}$ 是 3，与 f 互不影响。

·为什么这样设计："每次调用新作用域" (17.3 节) 与"函数是对象" (第 16 章) 两条规则的组合自然产生闭包——无需任何额外语法。让闭包成为一等公民，函数式编程 (map/filter 式风格、回调、装饰器) 才成立。

·解决什么实际问题：GUI 为每个按钮生成回调 (每个回调记住自己的按钮编号)；按用户身份生成日志函数；延迟执行 (现在配置、将来调用)。“按函数定制行为”用闭包比类更轻量。

·初学者误区：①以为闭包是"高深黑魔法"——它就是"函数记住了自己出生时的环境变量"；②以为 f 和 g 共享 N ——不共享，各带各的 cell；③把闭包与"拷贝"混为一谈——cell 里存的是引用 (对象本身)，如果捕获的是可变对象，各处看到的是同一对象 (下一节的 nonlocal 例子会放大这一点)。

·生活例子：闭包像"定制模具"——maker(2) 造出"平方模具"，maker(3) 造出"立方模具"，每副模具自带的参数焊死在模具里，互不干扰。

代码分析

```
>>> def maker(N):
...     def action(X):
...         return X ** N      # action  N
...     return action
>>> f = maker(2)               # 2  N
>>> f(3)                       # 3  action  X
9                              # maker  N  23 ** 2
>>> g = maker(3)               # g  3f  2
>>> g(4)                       # 4 ** 3
64
>>> f(4)                       # 4 ** 2
16
```

解释器如何处理：核心机制分四步。①编译：编译器检测到 action 引用了 N 而 N 在 maker 中赋值，于是把 N 定为 cell 变量（action 代码对象中出现 N 的自由变量条目）；②maker(2)：栈帧里 STOREDEREF N（存入 cell），MAKEFUNCTION 生成 action 并把 cell 填入 closure，函数对象返回赋给 f；③f(3)：action 的字节码 LOADFAST X（取参数 3）、LOADDEREF N（从 cell 取 2）、BINARYOP 得 9；④maker(3)：全新栈帧、全新 cell（值为 3），g 与 f 各握一个 cell，互不干扰。用 dis 可验证 action 里没有 LOADGLOBAL——N 的获取完全绕过了 LEGB 的 G 层。

设计思想：maker 是"参数固化器"：把 N 这个配置从"每

次调用都要传"变成"制造时就焊死"。GUI 里一个按钮一个 maker(id) 调用, 事件回调自动记住 id——这就是"按运行时条件生成行为"的工业级做法。

17.16 任意深度的作用域嵌套 (Arbitrary Scope Nesting)

英文原文

Finally, this tour of nested function scopes would be remiss if it didn't point out that scopes may nest arbitrarily, though only enclosing functions (not classes, described in Part VI) are searched when names are referenced: python >>> def f1(): ... x = 99 ... def f2(): ... def f3(): ... print(x) # Found in f1's local scope! ... f3() .. f2() >>> f1() 99 This does work: Python will search the local scopes of all enclosing defs, from inner to outer, after the referencing function's own local scope and before the module's global scope or built-ins. However, this sort of code is even less likely to crop up in practice. Per coding aphorism, flat is better than nested, and this still holds generally true even with nested scope closures in the toolbox. Except in limited contexts, your life (and the lives of your fellow travelers in the software realm) will generally be better if you minimize nesting in function definitions.

中文翻译

最后，如果不说清楚“作用域可以任意深度嵌套”这一点，这次嵌套函数作用域之旅就不完整——不过要注意，引用名字时只搜索外围函数（不包括第 VI 部分描述的类）：

```
python >>> def f1(): ... x = 99 ... def f2(): ... def f3(): ... print(x) # 在 f1 的局部作用域里找到了! ... f3() ... f2() >>> f1() 99
```

这确实有效：Python 会在引用函数自己的局部作用域之后、模块全局作用域或内置之前，从内到外搜索所有外围 def 的局部作用域。不过，这类代码在实践中出现的可能性更低。按照编码格言，扁平优于嵌套（flat is better than nested）——即使工具箱里有了嵌套作用域闭包，这条原则也大体依然成立。除非在有限的情境下，否则尽量减少函数定义中的嵌套，对你（以及软件世界中与你同行的人）一般都更好。

深度理解

·核心概念：E 层不是一层，而是“从内到外的所有层”——f3 直接命中 f1 的 x，中间的 f2 可以没有 x。查找顺序永远是“逐层外推，找到即停”。

·底层实现：CPython 用连续 cell 链实现多层捕获：f1 的 x 是 cell；f2 的代码对象把它作为自由变量再提升为 cell；f3 的代码对象也把它列为自由变量。closure 元组层层传递。字节码层面，跨多层的读取仍是 LOADDEREF（cell 是一次间接，不随层数增加而变慢）。

·为什么这样设计：嵌套是“词法结构”的自然结果——def 可以出现在任何语句能出现的地方，多层嵌套顺理成章。

但 Lutz 随即亮出《Python 之禅》的格言："flat is better than nested (扁平优于嵌套)"——能用就简单用，嵌套是手段不是目的。

·解决什么实际问题：三层以上的嵌套闭包在实际代码中几乎只出现在装饰器工厂、状态机等少数场景；多数时候直接返回外层结果即可，正如 17.21 节"扁平化替代方案"所示。

·初学者误区：①以为闭包只能捕获"一层"外围——可以捕获任意层；②以为嵌套越多越"高级"——恰恰相反，过度嵌套是代码坏味道；③在类里嵌函数时以为能捕获类作用域——类会被查找跳过（17.5 节），方法里要用类变量必须 `ClassName.x` 或 `self.x`。

17.17 nonlocal 语句 (The nonlocal Statement)

英文原文

In the prior section, we saw how nested functions can reference variables in an enclosing function's scope, even if that function has already returned. As suggested earlier, we can also change such enclosing scope variables, as long as we declare them in nonlocal statements. With this statement, nested functions gain both read and write access to names in enclosing functions. This makes nested scope closures more useful, by making state changeable. The nonlocal statement,

usable in `def` but not `lambda`, is similar in both form and role to `global`, covered earlier. Like `global`, `nonlocal` declares that a name (or names) will be changed in an enclosing scope. Unlike `global`, though, `nonlocal` applies to a name in an enclosing function's scope, not to the global module scope outside all functions. Also unlike `global`, `nonlocal` names must exist in an enclosing function's scope—they are mapped only to enclosing functions and cannot be created by a first assignment in a nested `def` that uses `nonlocal`. In other words, `nonlocal` both allows assignment to names in enclosing function scopes, and limits scope lookups for such names to enclosing functions. The net effect is a direct and reliable implementation of changeable state information, for contexts that do not desire or need to use classes or other stateful tools.

中文翻译

在上一节，我们看到了嵌套函数如何引用外围函数作用域中的变量——即使该函数已经返回。正如之前暗示的，我们也可以修改这类外围作用域变量，只要在 `nonlocal` 语句中声明它们。有了这个语句，嵌套函数就获得了对外围函数中名字的读和写双重访问权。通过让状态可变，这使得嵌套作用域闭包更加有用。`nonlocal` 语句可用于 `def` 但不能用于 `lambda`，它在形式和角色上都与前面讲过的 `global` 相似。和 `global` 一样，`nonlocal` 声明一个（或多个）名字将在外围作用域中被修改。但与 `global` 不同的是，`nonlocal` 作用于外围函数作用域中的名字，而不是所有函数之外的全局

模块作用域。还有一点与 global 不同：nonlocal 的名字必须存在于外围函数的作用域中——它们只映射到外围函数，不能由使用 nonlocal 的嵌套 def 中的第一次赋值来创建。换句话说，nonlocal 既允许对外围函数作用域中的名字赋值，又限制这类名字的作用域查找仅限外围函数。净效果是：为不需要（或没必要）使用类或其他有状态工具の場合，提供了一种直接而可靠的可变状态信息实现。

深度理解

- 核心概念：nonlocal 是"可写的闭包"开关——没有它，内层函数对外层变量只能读不能写；有了它，读写都行。global 管模块、nonlocal 管外围函数，且 nonlocal 的名字必须预先存在于外围 def（不能凭空创建）。
- 底层实现：nonlocal state 使内层函数体里 `state += 1` 编译为 `LOADDEREF + STOREDEREF`——读写同一个 cell。这就是"改变的是 cell 里的值"，与外层函数共享同一存储。注意 `state += 1` 是"读-算-写"，三步都作用于同一 cell，闭包状态因此可被持久修改。
- 为什么这样设计：Python 2 时代闭包状态只能"读"，想写要 hack（可变对象、函数属性）。nonlocal（Python 3）让"可变闭包状态"成为一等语法，同时保留"显式声明"的安全原则——不改就不声明。
- 解决什么实际问题：计数器、累加器、缓存（memoization 的朴素版）、状态机——"函数带着自己的状态，每次调用更新它"，且状态天然私有（不像全局被所有人共享）。

·初学者误区：①不写 `nonlocal` 就 `state += 1`——`state` 因赋值被判为内层局部，但 `+=` 又要先读，于是 `UnboundLocalError`（本章马上演示）；②写了 `nonlocal state` 但外层没赋值 `state`——编译期直接 `SyntaxError`；③试图用 `nonlocal` 指向全局/内置名字——同样 `SyntaxError`。

17.18 `nonlocal` 基础 (nonlocal Basics)

英文原文

The `nonlocal` statement has meaning only inside a function, and is an error to use elsewhere. More specifically, it applies and is usable only when `def` is nested in another `def`: because the body of a lambda allows just an expression, it supports neither `nonlocal` nor nested `def` statements. When used within a `def`, the `nonlocal` statement seals the fate of references—much like the `global` statement, `nonlocal` causes searches for the names listed in the statement to begin in the enclosing `defs`' scopes, not in the local scope of the declaring function. That is, `nonlocal` also means "skip my local scope entirely." In fact, the names listed in a `nonlocal` must be assigned in an enclosing `def`, and never refer to names in the global or built-in scopes. Importantly, the addition of `nonlocal` does not alter name reference scope rules in general; they still work as before, per the LEGB rule described earlier. The `nonlocal`

statement simply serves to allow names in enclosing scopes to be changed rather than just referenced. The global and nonlocal statements, though, tighten up and even restrict the lookup rules within a function for the names that they list: - global makes scope lookup begin in the enclosing module's scope and allows names there to be assigned. Scope lookup continues on to the built-in scope if the name does not exist in the module, but assignments to global names always create or change them in the module's scope. - nonlocal restricts scope lookup to just enclosing defs, requires that names exist there, and allows them to be reassigned. Scope lookup does not continue on to the global or built-in scopes.

中文翻译

nonlocal 语句只在函数内部有意义，在其他地方使用就是错误。更具体地说，它只在 def 嵌套于另一个 def 中时适用、可用：因为 lambda 的函数体只允许表达式，它既不支持 nonlocal，也不支持嵌套的 def 语句。在 def 内使用时，nonlocal 语句“封死了”引用的去处——和 global 语句很像：nonlocal 使语句中列出的名字的搜索从外围 def 的作用域开始，而不是从声明它的函数自己的局部作用域开始。也就是说，nonlocal 也意味着“完全跳过我的局部作用域”。事实上，nonlocal 列出的名字必须在外围 def 中被赋值，并且绝不指向全局或内置作用域中的名字。重要的是，nonlocal 的加入并不会改变一般的名字引用作用域规则；它们照旧按前面描述的 LEGB 规则工作。nonlocal 语句只

是让外围作用域中的名字可以被修改，而不仅仅是引用。不过，`global` 和 `nonlocal` 语句确实会针对它们列出的名字，在函数内部收紧甚至限制查找规则：- `global` 让作用域查找从外围模块的作用域开始，并允许其中的名字被赋值。如果模块中不存在该名字，查找会继续到内置作用域，但对全局名字的赋值总是在模块作用域中创建或修改它们。- `nonlocal` 把作用域查找限制为仅限外围 `def`，要求名字存在于那里，并允许它们被重新赋值。查找不会继续到全局或内置作用域。

深度理解

·核心概念：两条声明语句是"查找起点重定向器"：`global` 把起点拨到模块层（找不到还能顺路看 `builtins`，但写入只在模块层）；`nonlocal` 把起点和终点都钉死在外围 `def` 层（绝不外溢到模块/`builtins`）。`nonlocal` = 我的局部不算 + 只能在围城里的 `def` 中找。

·底层实现：编译器处理 `nonlocal` 列表时，把每个名字解析为"某个外围函数代码对象中的 `cell` 或自由变量"，绑定关系在编译期确定（所以"外层没赋值"会在编译时报 `SyntaxError`）。运行时 `LOADDEREF/STOREDEREF` 直达对应 `cell`。

·为什么这样设计：把 `nonlocal` 查找圈死在外围 `def`，是为了消除歧义——如果允许外溢到模块，编译器就无法确定该在哪一层创建或修改名字。宁可报错，也不猜。这与 `global`（永远只有一个模块层，无歧义）形成对照。

·解决什么实际问题：状态的作用域纪律——"闭包的状态只属于它的函数家族，不泄漏到全局"；调试时你不用满程

序找"谁改了它"。

·初学者误区：①以为 `nonlocal` 是 `global` 的弱化版——两者语义完全不同（一个指模块、一个指外围函数）；②以为 `lambda` 里能写 `nonlocal`——不能，`lambda` 体只是表达式；③以为 `nonlocal` 能读/写 `builtins` 名字（如 `nonlocal len`）——`SyntaxError`，`builtins` 不在外围 `def` 的绑定集中。

17.19 `nonlocal` 实战 (`nonlocal in Action`)

英文原文

Let's move on to examples to make this more concrete. Simple references to enclosing `def` scopes work as we've already seen—in the following, `outer` builds and returns the function `inner` to be called later, and the state reference in `inner` maps the local scope of `outer` using the normal scope LEGB lookup rules:

```
python >> def outer(start):
... state = start ... def inner(label): ... print(label, state)
... # Remembers state in enclosing scope ... return inner
>> F = outer(0) >> F('code') # State is the same on e
very inner run code 0 >> F('hack') hack 0
```

Changing a name in an enclosing `def`'s scope, however, is not allowed by default:

```
python >> def outer(start):
```

```
... state = start ... def inner(label): ... state += 1 # Can
not change by default ... print(label, state) ... return in
ner >> F = outer(0) >> F('code') UnboundLocalError:
cannot access local variable 'state'...
```

Now, if we declare state in the outer scope as nonlocal within inner, we get to both reference and change it inside the nested function. Again, this works even though outer has returned and exited by the time we call the returned inner function through the name F:

```
python >> def outer(start):
```

```
... state = start # Each call gets its own state ... def inn
er(label): ... nonlocal state # <= Remembers state in e
nclosing scope ... state += 1 # Allowed to change it if
nonlocal ... print(label, state) ... return inner >> F = o
uter(0) >> F('code') # Increments state on each call c
ode 1 >> F('hack') hack 2
```

As usual with enclosing scope references, we can call the outer factory (closure) function multiple times to get multiple copies of its state in memory. The state object in the enclosing scope is essentially attached to the new inner function object returned; each call makes a new, distinct state object, such that updating one function's state won't impact the other. The following continues the prior listing's interaction:

```
python >> G = outer(24) # Make a new outer that st
arts at 24 >> G('code') code 25 >> G('hack') # G's sta
te information updated to 26 hack 26 >> F('more') #
```

But F's state is where it left off: at 2 more 3 # Each call has different state information

As you can see, Python's nonlocals are much more functional than "static" function locals typical in some other languages: in a closure function, nonlocals are per-call, multiple copy data.

中文翻译

让我们继续看示例，让这一点更具体。对外围 def 作用域的简单引用和我们之前看到的一样工作——下面，outer 构建并返回稍后要调用的函数 inner，inner 中的 state 引用按正常的 LEGB 查找规则映射到 outer 的局部作用域：python >>> def outer(start): ... state = start ... def inner(label): ... print(label, state) # 记住外围作用域中的 state return inner >>> F = outer(0) >>> F('code') # 每次运行 inner 时 state 都一样code 0 >>> F('hack') hack 0 然而，默认情况下不允许修改外围 def 作用域中的名字：python >>> def outer(start): ... state = start ... def inner(label): ... state += 1 # 默认不允许修改... print(label, state) ... return inner >>> F = outer(0) >>> F('code') UnboundLocalError: cannot access local variable 'state' ... 现在，如果我们在 inner 中把 outer 作用域的 state 声明为 nonlocal，我们就能在嵌套函数内部既引用又修改它。再次强调：即使在我们通过名字 F 调用返回的 inner 函数时 outer 早已返回并退出，这依然有效：python >>> def outer(start): ... state = start # 每次调用都有自己的 state ... def inner(label): ... nonlocal state # 记住外围作用域中的 state ... state += 1 # 声明 nonlocal

后允许修改... `print(label, state) ... return inner` >>> `F = outer(0)` >>> `F('code')` # 每次调用都会递增 state code 1 >>> `F('hack')` hack 2 与外围作用域引用通常的情况一样，我们可以多次调用 outer 工厂（闭包）函数，在内存中得到它状态的多份副本。外围作用域中的 state 对象本质上附着在返回的新 inner 函数对象上；每次调用都会制作一个新的、独立的 state 对象，因此更新一个函数的状态不会影响另一个。下面继续前面列出的交互：python >>> `G = outer(24)` # 制作一个从 24 开始的新 outer >>> `G('code')` code 25 >>> `G('hack')` # G 的状态信息更新为 26 hack 26 >>> `F('more')` # 但 F 的状态停在原处：2 more 3 # 每个调用有不同的状态信息如你所见，Python 的 nonlocal 比一些其他语言中典型的 "static（静态）" 函数局部变量功能强大得多：在闭包函数中，nonlocal 是每次调用独立、多副本的数据。

深度理解

·核心概念：三个递进示例展示：读（LEGB 自动）→ 写报错（UnboundLocalError）→ 声明 nonlocal 后读写皆可。最后点明：nonlocal 状态是“每次调用工厂 = 一份新副本”，与 C 语言 static 变量（全程序一份）形成鲜明对比。

·底层实现：第二个示例的报错很有教学价值——`state += 1` 使 state 被编译为 inner 的局部变量（因为出现赋值），但 `+=` 需要先读，而局部变量尚未赋值，于是 LOAD FAST state 在栈帧中找不到绑定，抛 UnboundLocalError（在 Python 3.11+ 中错误信息提示 "cannot access l

ocal variable")。声明 nonlocal 后，state 被编译为 LOADDEREF/STOREDEREF，读写都作用于 cell——outer 虽然退场，cell 仍在 inner.closure 里存活。

·为什么这样设计：UnboundLocalError 是"就近赋值"原则的必然代价——编译器宁可报错也不去猜"你是不是想改外围的"。而 nonlocal 给了显式通道。这保持了"赋值即归属"的单一、可预测规则。

·解决什么实际问题：这是计数器、调用次数统计、缓存状态等"有状态函数"的标准写法；相比全局变量，状态天然隔离（F 与 G 互不干扰），相比类，代码更少。

·初学者误区：①以为 state += 1 的报错是"Python 的 bug"——这是规则（赋值→局部）与用法（想改外围）冲突的提示；②以为 nonlocal 必须写在函数体最开头——可以写，但出现在任何位置都会作用于整个函数体的该名字；③以为多个 outer 调用共享状态——每个 inner 持有自己的 cell，F 和 G 的计数互相独立。

·生活例子：F 和 G 像两台独立计数的自动售货机（每台自己数自己卖了几瓶），nonlocal 是给售货机加装"累计计数器"的许可；而 C 的 static 更像"全楼共用的总电表"，谁都用同一份。

代码分析

```

>>> def outer(start):
...     state = start                # state
...     def inner(label):
...         nonlocal state          # <= state
...         state += 1              # nonlocal
...         print(label, state)
...     return inner
>>> F = outer(0)
>>> F('code')                      # state
code 1
>>> F('hack')
hack 2

```

解释器如何处理：①编译 `outer`：state 被标记为 cell（被内层引用），`inner` 的代码对象声明自由变量 `state`；②`outer(0)`：新栈帧，`STOREDEREF state` 把 0 存入新 cell，创建 `inner` 函数对象并把 cell 挂进 closure，返回；③`F('code')`：`inner` 字节码执行 `LOADDEREF state`（读 cell 得 0）、`BINARYOP +`、`STOREDEREF state`（写回 cell 得 1），打印 `code 1`；④`F('hack')`：同样的三条指令再次执行，cell 中现在从 1 变 2。`outer` 的栈帧早已销毁，但这三步全部发生在活的 cell 上——这就是“外层退场、状态长存”的全部秘密。

设计思想：`nonlocal` 让“闭包=状态包”从只读升级为可读写，闭包因此成为真正的“轻量对象”——有属性（cell 里的状态）、有方法（函数本体），而无需类语法。这为装饰器（第 39 章）等高级模式铺路。

17.20 nonlocal 边界情形 (nonlocal Boundary Cases)

英文原文

Though useful, nonlocals come with some subtleties to be aware of. First, unlike the global statement, non local names really must be assigned in an enclosing def's scope—you cannot create them dynamically by assigning them anew in a nested function. The enclosing function's assignment can appear either before or after the nested function, but you'll get an error if it's missing.

In fact, nonlocals are checked for an enclosing function assignment at function definition time, before either an enclosing or nested function is ever called:

```
python >> def outer(start):
... def inner(label): ... nonlocal state # Nonlocals must
... exist in an enclosing def ... state = 0 ... print(label, sta
... te) ... return inner SyntaxError: no binding for nonlocal
'state' found >> def outer(start):
... def inner(label): ... global state # But globals don't
... have to exist when declared ... state = 0 ... print(label,
... state) ... return inner >> F = outer(0) >> F('glob') glo
b 0 >> state # Created by the assignment in nested inner 0
```

Second, `nonlocal` restricts the scope lookup to just enclosing defs; nonlocals are not looked up in the enclosing module's global scope or the built-in scope outside all defs, even if they are already there:

```
python >> state = 0 >> def outer():
```

```
... def inner(): ... nonlocal state # Must be located in a
def, not the module ... state += 1 # Use global for gl
obals, not nonlocal ... return inner SyntaxError: no bin
ding for nonlocal'chapter' found >> def outer():
```

```
... def inner(): ... nonlocal ord # Ditto for built-ins (an
d immediate calls) ... print(ord) ... inner() SyntaxError:
no binding for nonlocal'ord' found
```

These restrictions make sense once you realize that Python would not otherwise generally know in which enclosing scope to create a brand-new name. In the prior listing, should `state` be assigned in `outer`, or the module outside? Because this is ambiguous, Python must resolve nonlocals at function creation time, not function call time.

Finally, this chapter has been careful to say that nonlocal names must be assigned in an enclosing function, not the enclosing function. This can matter when function nesting runs deep: a name listed in `nonlocal` can technically appear anywhere in the hierarchy of enclosing functions—not just one level up—and the closest appearance is used.

Even so, because such code seems unlikely to crop up in practice (and probably qualifies as cruel and unusual punishment), we'll forgo a formal demo here, but see "Arbitrary Scope Nesting" for most of the morass.

中文翻译

虽然很有用，但 `nonlocal` 也有一些需要注意的微妙之处。首先，与 `global` 语句不同，`nonlocal` 的名字真的必须在外围 `def` 的作用域中被赋值——你不能在嵌套函数中重新赋值来动态创建它们。外围函数的赋值可以出现在嵌套函数之前或之后，但如果缺失就会报错。事实上，`nonlocal` 会在函数定义时刻（而不是任何外围或嵌套函数被调用之前）就被检查是否有外围函数的赋值：

```
python >>> def outer(start): ... def inner(label): ... nonlocal state # nonlocal 必须存在于外围 def 中... state = 0 ... print(label, state) ... return inner
SyntaxError: no binding for nonlocal'state' found
>>> def outer(start): ... def inner(label): ... global state # 但 global 声明时不必已存在... state = 0 ... print(label, state) ... return inner
>>> F = outer(0)
>>> F('glob')
glob 0
>>> state # 由嵌套 inner 中的赋值创建
0
第二，nonlocal 把作用域查找仅限制在外围 def 中；即使名字已经存在于外围模块的全局作用域或所有 def 之外的内置作用域中，nonlocal 也不会去那里查找：
```

```
python >>> state = 0
>>> def outer(): ... def inner(): ... nonlocal state # 必须位于某个 def 中，而不是模块中... state += 1 # 全局用 global，而不是 nonlocal ... return inner
SyntaxError: no binding for nonlocal'chapter' found
>
```

>> def outer(): ... def inner(): ... nonlocal ord # 内置亦然 (包括立即调用的情况) ... print(ord) ... inner() Syntax Error: no binding for nonlocal'ord' found 一旦你意识到: 否则 Python 通常无法知道该在哪个外围作用域中创建一个全新的名字, 这些限制就说得通了。在前面那个列表中, state 应该赋值在 outer 中, 还是外面的模块中? 因为这有歧义, Python 必须在函数创建时刻 (而不是函数调用时刻) 解析 nonlocal。最后, 本章一直小心翼翼地表述为: nonlocal 名字必须赋值在某个外围函数中, 而不是那一个外围函数中。当函数嵌套很深时这一点很重要: nonlocal 列出的名字在技术上可以出现在外围函数层级中的任意位置——不仅仅是一层之上——并且使用最近的那处出现。即便如此, 因为这类代码在实践中似乎不太可能出现 (而且大概算得上"残忍而不寻常的惩罚"), 我们这里就不做正式演示了, 迷宫的大致情况参见"任意深度的作用域嵌套"。

深度理解

·核心概念: nonlocal 的三条边界规则: ①必须在外围 def 中已有绑定 (否则 SyntaxError, 且在编译期检查); ②只在"围城" (外围 def) 里找, 模块/内置作用域不算数; ③绑定到"最近的"那层外围绑定, 而非必须紧邻一层。

·底层实现: nonlocal 的绑定解析发生在编译期 (compile() 阶段): 编译器从内层函数开始, 逐层向外扫描代码对象, 找第一个包含该名字绑定的函数作用域, 把它对应的 cell 引用写入内层代码对象。找不到就立刻抛 SyntaxError: no binding for nonlocal'state' found——程序还没运行就报错了。而 global 则不需要任何存在性检查 (

模块作用域必然存在，名字可以后创建）。

·为什么这样设计：global 的目标层是唯一的（模块），"创建新名字"没有歧义；nonlocal 的目标层可能是多层函数中的任意一层，若允许"创建"，编译器无法确定该建在哪层——所以强制"已存在"，把歧义变成编译期错误。这是"宁可早报错，不可晚乱猜"的典型。

·解决什么实际问题：这套严格规则保证了闭包状态绑定的确定性——你永远知道 nonlocal x 指的是哪一层函数里的 x，排查问题时不必猜测。

·初学者误区：①以为 nonlocal 可以像 global 一样"声明一个将来才存在的变量"——不行，那是它俩的核心差异；②以为模块级有 state = 0 就能让 nonlocal state 通过——不能，nonlocal 只看外围 def；③把报错当成运行时错误——它是编译期 SyntaxError，脚本根本不会启动。

代码分析

```
>>> def outer(start):
...     def inner(label):
...         nonlocal state          # nonlocal def
...         state = 0
...         print(label, state)
...     return inner
SyntaxError: no binding for nonlocal 'state' found
```

解释器如何处理：当编译 inner 函数体时，编译器处理 nonlocal state：它需要在外围函数（outer）的作用域中找到 state 的绑定（binding）。扫描 outer 的作用域：参数 start 是绑定，但 state 不是——outer 的函数体里

没有任何赋值 `state`。于是编译直接失败，抛出 `SyntaxError`，整个过程发生在任何代码执行之前。对比之下，`global state` 不需要检查：模块命名空间天然存在，`state` 可以等运行时第一次赋值才出现（第二个示例中 `F('glob')` 调用后，模块里就有了 `state == 0`）。

设计思想：编译期检查是 Python 的一贯哲学——能在编译时发现的问题，绝不留到运行时。`nonlocal` 的“存在性检查”正是一项免费的安全网：它保证了闭包状态的绑定在程序启动前就完全确定，也杜绝了“名字该在哪一层创建”的歧义。

17.21 状态保留方案 (State-Retention Options)

英文原文

Given the extra complexity of nested functions, you might wonder why they're worth the fuss. Although it's difficult to see in our small examples, state information becomes essential in many programs. While functions can return results, their local variables won't normally retain other values that must live on between calls. Moreover, many applications require such values to differ per context of use. Broadly speaking, there are multiple ways for Python functions to retain state between calls. These include the global variables and enclosing scope references we've already met, but als

o class-instance attributes and function attributes. To close out this chapter, let's review these options to see how they stack up.

中文翻译

考虑到嵌套函数带来的额外复杂度，你可能想知道它们为什么值得大费周章。虽然在我们的小示例中很难看出来，但状态信息在许多程序中是必不可少的。函数虽然可以返回结果，但它们的局部变量通常不会保留那些必须存活于多次调用之间的其他值。而且，许多应用要求这些值随使用情境的不同而不同。概括地说，Python 函数在调用之间保留状态有多种方式。这包括我们已经见过的全局变量和外围作用域引用，也包括类实例属性和函数属性。为了给本章收尾，让我们回顾这些方案，看看它们各自的优劣。

深度理解

·核心概念：本章最后的大总结——四种"跨调用状态保留"

方案横向对比：nonlocal、global、函数属性、类属性。

评判维度：可否修改、是否每份独立（per-call）、是否显式、是否私密。

·底层实现：四种方案背后是四种存储位置：cell 对象（nonlocal）、模块字典（global）、函数对象自身的属性字典 dict（函数属性）、实例对象的属性字典（类实例）。本质都是"把数据挂在某个长期存活的对象上"。

·为什么这样设计：函数"用完即走"的天性（局部变量随栈帧销毁）与"记住状态"的需求天然矛盾，Python 给出多条出路，让程序员按需取舍：要私密轻量选 nonlocal，要

显式可外查选函数属性，要体系化选类。

·解决什么实际问题：这是"无类编程"（函数式风格）与 OOP 的分岔口——选对了，代码又短又清晰；选错了（比如全用 global），就会出现本章前面警告的耦合问题。

·初学者误区：以为"只有一种正确做法"——没有，按"是否需要多副本、是否需要外部可见、代码量预算"来选；也容易忽视"函数属性"这个冷门但优雅的方案（很多老 Python 代码用它，读得懂是本事）。

17.22 nonlocal: 可修改、每调用独立、LEGB (Nonlocals: Changeable, Per-Call, LEGB)

英文原文

First off, the following code is a recap from the prior section, repeated here for compare and contrast. As we've seen, its nonlocal allows state in an enclosing scope to be saved and modified. Each call to outer makes a self-contained package of changeable information, whose names and objects do not clash with any other part of the program:

```
python >>> def outer(start):
...     state = start # Each call gets its own state
...     def inner(label):
...         nonlocal state # Remembers state in enclosing scope
...         state += 1 # Allowed to change it if nonlocal
...         print(label, state)
...     return inner
>>> F = outer(0) # State to be saved in enclosing scope
>>> F('nonlocal1') # State visible within closure only nonl
```

ocal1 1 >>> F('nonlocal2') nonlocal2 2 >>> F.state
AttributeError:'function' object has no attribute'state'
We need to declare variables nonlocal only if they must be changed (other enclosing scope name references are automatically retained as usual per LEGB), and nonlocal names are not visible outside the enclosing function. While this scheme works well, the next three sections present some alternatives. Some of the code in these sections uses tools we haven't covered yet and is intended partially as preview, but we'll keep the examples simple here so that you can compare and contrast along the way.

中文翻译

首先，下面的代码是上一节的回顾，为了比较对照而在此重复。正如我们所见，它的 nonlocal 允许外围作用域中的状态被保存和修改。每次调用 outer 都会制作一个自包含的可变信息包，其名字和对象不会与程序任何其他部分冲突：

```
python >>> def outer(start): ... state = start # 每次调用都有自己的一份 state ... def inner(label): ... nonlocal state # 记住外围作用域中的 state ... state += 1 # 声明 nonlocal 后允许修改... print(label, state) ... return inner
>>> F = outer(0) # 要保存到外围作用域的状态>>> F('nonlocal1') # 状态只在闭包内可见nonlocal1 1 >>> F('nonlocal2') nonlocal2 2 >>> F.state
AttributeError:'function' object has no attribute'state'
```

我们只需要在必须修改变量时声明 nonlocal（其他外围作用域名字引用照常按 LEGB 自动保留），而且 nonlocal 名字在外围函数之外不

可见。虽然这个方案工作得很好，但接下来三节会介绍一些替代方案。这些节里的某些代码使用了我们还没学过的工具，部分目的是预览，但这里我们会让示例保持简单，方便你一路比较对照。

深度理解

- 核心概念：nonlocal 方案的招牌特性：可修改（changeable）+ 每调用独立（per-call）+ 天然私密。F.state 报 AttributeError 正是“私密性”的证明——状态藏在闭包里，外部无从直接读写。

- 底层实现：state 住在 cell 里，cell 挂在 inner.closure[0]。函数对象没有名为 state 的属性（dict 里没有），所以 F.state 走属性查找失败抛 AttributeError——除非你访问 F.closure[0].cellcontents 这条“暗道”。

- 为什么这样设计：私密性不是 bug 而是特性——状态被封装在“函数家族”内部，外部世界无法意外修改，这正是封装（encapsulation）的轻量实现。

- 解决什么实际问题：当状态不该被外部看到、又需要每份独立时，nonlocal 是最优解。比如“每个按钮独立计数”的场景，外部根本不该碰那个计数。

- 初学者误区：①以为 F.state 是合法的访问方式——不是，那是函数属性（17.24 节）的做法；②以为 nonlocal 状态是“全局可见”的——恰恰相反，它只对闭包函数内部可见。

17.23 全局变量：可修改但共享 (Globals: Changeable but Shared)

英文原文

One common suggestion for achieving state retention without nonlocal is to simply move saved info out to the global scope (the enclosing module):

```
python >> def outer(start):  
... global state # Move it out to the module to change it ...  
state = start # global allows changes in module scope ...  
def inner(label): ... global state ... state += 1 ...  
print(label, state) ... return inner  
>> F = outer(0)  
>> F('global1') # Each call increments shared global state  
global1 1  
>> F('global2') global2 2  
>> state # State accessible as global outside defs 2
```

This works in this case, but it requires global declarations in both functions and is prone to name collisions in the global scope (what if "state" is already being used for something else?). A more subtle problem is that it only allows for a single shared copy of the state information in the module scope—if we call outer again, we'll wind up resetting the module's state variable, such that prior calls will see their state overwritten:

```
python >> G = outer(24) # Resets state's single copy in global scope  
>> G('global3') global3 25  
>> F('glob
```

al4') # But F's counter has been overwritten! global4
26

As shown earlier, when you use nonlocal and nested function closures instead of global, each call to outer remembers its own unique copy of the state object. F or per-call roles, globals don't fit the bill.

中文翻译

一种常见的、不用 nonlocal 实现状态保留的建议是：干脆把保存的信息搬到全局作用域（外围模块）：

```
python >>>
def outer(start): ... global state # 把它移到模块中以便修改...
state = start # global 允许在模块作用域中修改...
def inner(label): ... global state ... state += 1 ... print(label, state) ...
return inner >>> F = outer(0) >>> F('global1') # 每次调用都递增共享的全局状态
global1 1 >>> F('global2') global2 2 >>> state # 在 def 之外也可作为全局访问
2 这个方案在这种情形下确实有效，但它需要两个函数里都写 global 声明，而且容易在全局作用域中发生名字冲突（如果"state"已经被别的东西占用了呢？）。一个更微妙的问题是：它只允许模块作用域中单一共享副本的状态信息——如果我们再次调用 outer，就会重置模块的 state 变量，导致先前调用生成的函数看到自己的 state 被覆盖：
```

```
python >>> G = outer(24) # 重置了全局作用域中 state 的唯一副本
>>> G('global3') global3 25 >>> F('global4') # 但 F 的计数器已被覆盖！
global4 26 正如前面所示，当你用 nonlocal 和嵌套函数闭包而不是 global 时，每次调用 outer 都会记住 state 对象自己独一无二的副本。对于"每调用独立"的角色，全局变量并不合适。
```

深度理解

·核心概念：global 方案是"把状态搬出函数、寄存在模块字典"。它有三宗罪：①要写两处 global 声明；②容易与模块里其他名字冲突；③只有一份副本——G = outer(24) 会覆盖 F 的状态，两个闭包互相踩踏。

·底层实现：state 是模块字典里的一个条目。outer(0) 写一次 (STOREGLOBAL)，inner 每次 LOADGLOBAL/STOREGLOBAL 读写同一条目。outer(24) 再次执行时把同一条目改成 24——F 的下一次调用读到的自然是 24，然后变 25、26。没有"每份独立"可言，因为存储位置只有一个。

·为什么这样设计：模块字典是"进程级共享内存"——单副本是它的天然属性。这既是优点（多线程通信、全局配置），也是缺点（多副本需求无法满足）。书里 17.9 节已警告过耦合问题，这里是它的技术后果。

·解决什么实际问题：全局状态适合"整个程序只有一个"的数据（总开关、连接池、单一计数器）；一旦出现"每个实例各一份"的需求，立即切换到 nonlocal/函数属性/类。

·初学者误区：①以为 global 方案与 nonlocal 方案"差不多"——差多了，一个是共享一份、一个是各持一份；②在多层函数里忘写某一处 global 导致"看起来改了、其实没改"（赋给了局部）；③以为"反正只有一个计数器"没关系——当第二个调用点出现时，bug 就来了。

17.24 函数属性：可修改、每调用独立、显式 (Function Attributes: Changeable, Per-Call, Explicit)

英文原文

As another state-retention option, we can often achieve the same effect as nonlocals with function attributes—user-defined names attached to functions explicitly. When you attach user-defined attributes to nested functions generated by factory functions, they can also serve as per-call, multiple copy, and writeable state, just like nonlocal scope closures.

Such user-defined attribute names won't clash with names Python creates itself, and as for nonlocal, need be used only for state variables that must be changed; other scope references are retained and work normally.

Because factory functions make a new function on each call anyhow, this does not require extra objects—the new function's attributes become per-call state in much the same way as nonlocals, and are similarly associated with the new function in memory. In contrast, function attributes are perhaps less magical than scopes, and allow state variables to be accessed outside the nested function.

With `nonlocal`, state variables can be seen directly only within the nested `def`; with attributes, state access is a simple function-attribute fetch.

Here's a mutation of our example based on this technique—it replaces a `nonlocal` with an attribute attached to the nested function. This scheme may not seem as intuitive to some at first glance; you must access state explicitly through the function's name instead of as simple variables, and must initialize after the nested `def`. Still, it allows state to be accessed externally, saves a line by eliminating a `nonlocal` declaration, and makes state usage more explicit:

```
python >> def outer(start):  
... def inner(label): ... inner.state += 1 # Change object  
t attr, not inner itself ... print(label, inner.state) # inner  
is in the enclosing scope ... inner.state = start # Initialize  
state after inner defined ... return inner >> F = outer(0)  
>> F('attr1') # F is an inner with state attached attr1 1  
>> F('attr2') attr2 2 >> F.state # Can access state outside  
defs too 2
```

Because each call to the outer function produces a new nested function object, this scheme supports multiple-copy, per-call changeable data just like `nonlocal` closures—a usage mode that global variables cannot provide:

```
python >> G = outer(24) # G has own state, doesn't  
overwrite F's >> G('attr3') attr3 25 >> F('attr4') # F's
```

state varies attr4 3 >> F.state # State is accessible and per-call 3 >> G.state 25 >> F is G # Different function objects False

Fine points: this code relies on the fact that the function name `inner` is a local variable in the outer scope enclosing `inner`; as such, it can be referenced freely inside `inner` per LEGB's E.

This code also relies on the fact that changing an object in place is not an assignment to a name; when it increments `inner.state`, it is changing part of the object `inner` references, not the name `inner` itself (much like the `L.append` call we saw earlier). Because we're not really assigning a name in the enclosing scope, no `nonlocal` declaration is required.

We'll explore function attributes further in Chapter 19. Importantly, you'll see there that Python uses naming conventions that ensure that the arbitrary names you assign as function attributes won't clash with names related to internal implementation, making the namespace equivalent to a user scope. At the end of the day, function attributes both predate `nonlocal` and provide similar utility, making the latter technically redundant in some roles.

NOTE State with enclosing scope mutables: On a related note, it's also possible to change a mutable object like a list in the enclosing scope without declaring its name `nonlocal`. The following, for example, implem

ents changeable per-call state information, and largely works the same as the preceding version (though it does not support access to state info from outside the functions):

```
python def outer(start):  
    def inner(label):  
        state[0] += 1  
    # Clever hack or dark magic?
```

```
    print(label, state[0]) # Leverage in-place mutable change  
    state = [start]  
    return inner
```

This exploits the mutability of lists, and like function attributes, relies on the fact that in-place object changes do not classify a name as local. This is perhaps more obscure than either function attributes or nonlocal, though—it's a technique that predates others, and seems to lie today somewhere on the spectrum from arcane hack to dated workaround.

You're probably better off using named function attributes than lists and numeric offsets this way (but you can't control what others code).

中文翻译

作为另一种状态保留方案，我们常常可以用函数属性（function attributes）达到与 nonlocal 相同的效果——也就是显式地给函数附加用户自定义名字。当你给工厂函数生成的嵌套函数附加用户自定义属性时，它们同样可以作为每调用独立、多副本、可写入的状态，就像 nonlocal 作用域闭包一样。这类用户自定义属性名不会与 Python 自己创建的名字冲突，而且和 nonlocal 一样，只需要用于那些必须被修

改的状态变量；其他作用域引用照常被保留并正常工作。因为工厂函数反正每次调用都会制作一个新函数，所以这不需要额外的对象——新函数的属性以与 `nonlocal` 基本相同的方式成为每调用独立的状态，并同样在内存中与新函数关联。相比之下，函数属性可能比作用域少一点魔法感，而且允许状态变量在嵌套函数之外被访问。用 `nonlocal` 时，状态变量只能直接出现在嵌套 `def` 内部；用属性时，状态访问就是简单的函数属性获取。下面是我们示例基于这一技术的变体——它用附加在嵌套函数上的属性替换了 `nonlocal`。这个方案对某些人来说乍一看可能不太直观：你必须通过函数的名字显式访问状态，而不是把它当简单变量用，而且必须在嵌套 `def` 之后初始化。不过，它允许状态在外部被访问，省掉一行 `nonlocal` 声明，而且让状态的用法更加显式：

```
python >>> def outer(start): ... def inner(label): ... i
nner.state += 1 # 修改对象属性，而不是 inner 这个名字
... print(label, inner.state) # inner 在外围作用域中... inn
er.state = start # 在 inner 定义之后初始化状态... return
inner >>> F = outer(0) >>> F('attr1') # F 是一个带着
状态属性的 inner attr1 1 >>> F('attr2') attr2 2 >>> F.s
tate # 在 def 之外也可以访问状态2 因为每次调用外层函
数都会产生一个新的嵌套函数对象，这个方案像 nonlocal
闭包一样支持多副本、每调用独立、可修改的数据——这
是全局变量无法提供的用法模式：python >>> G = outer
(24) # G 有自己的状态，不覆盖 F 的 >>> G('attr3') attr3
25 >>> F('attr4') # F 的状态独立变化 attr4 3 >>> F.stat
e # 状态可访问且每调用独立3 >>> G.state 25 >>> F is
G # 不同的函数对象 False 细节要点：这段代码依赖这样一个事实——函数名 inner 是外围 outer 作用域中的一个局
```


部变量；因此，按照 LEGB 的 E 层，它可以在 inner 内部被自由引用。这段代码还依赖这样一个事实——就地修改对象不是对名字的赋值；当它执行 `inner.state += 1` 时，修改的是 inner 所引用的对象的一部分，而不是 inner 这个名字本身（很像我们之前见过的 `L.append` 调用）。因为我们实际上并没有在外围作用域中给名字赋值，所以不需要 `nonlocal` 声明。我们将在第 19 章进一步探索函数属性。

重要的是，你会看到 Python 使用的命名约定能确保你作为函数属性赋值的任意名字不会与内部实现相关的名字冲突，使该命名空间等价于一个用户作用域。说到底，函数属性既比 `nonlocal` 更早出现，又提供了类似的功用，在某些角色中反而使 `nonlocal` 技术上显得冗余。注意用外围作用域的可变对象保存状态：顺带一提，也可以不声明名字为 `nonlocal` 就修改外围作用域中的可变（mutable）对象，比如列表。例如，下面这段代码实现了可修改、每调用独立的状态信息，与前一版本大体工作方式相同（不过它不支持从函数外部访问状态信息）：

```
python def outer(start):  
    def inner(label):  
        state[0] += 1 # 聪明的小把戏还是黑暗魔法？  
        print(label, state[0]) # 利用就地可变修改  
    state = [start]  
    return inner
```

这利用了列表的可变性，并且像函数属性一样，依赖于“就地对象修改不会把名字归类为局部变量”这个事实。不过这可能比函数属性或 `nonlocal` 都更加晦涩——这是一种比其他方案更早的技术，在今天似乎介于“晦涩的 hack”与“过时的变通办法”之间。比起用列表加数字下标这样做，你大概还是用命名函数属性更好（但你无法控制别人怎么写代码）。

深度理解

- 核心概念：函数属性方案 = "状态挂在函数对象上"。inner.state 把函数当"轻量对象"用：每个工厂调用产生新函数 → 新函数自带属性字典 → 状态天然每份独立、外部可读。F.state 与 F is G 验证了"独立副本"与"不同对象"。
- 底层实现：函数对象有 dict 属性字典 (func.dict) 。inner.state = start 就是往 inner.dict 里写条目 {'state': 0} ; inner.state += 1 是 LOADATTR (读) → BINARYOP + → STOREATTR (写) ——全是属性读写，不是名字赋值，所以 inner 这个名字的局部属性从未被触碰，nonlocal 不是必须的。F is G 为 False 证明两个函数对象是独立个体，属性各归各。
- 为什么这样设计：函数属性 (Python 2.1 时代就支持) 比 nonlocal (Python 3.0 才加入) 更古老；历史原因+显式性让它至今有用。其关键洞察是"对象属性修改 ≠ 名字赋值"——这条 17.3 节的规则在这里开出了花。
- 解决什么实际问题：需要"状态对外可读/可写"的闭包场景（调试、测试、外部检查），用函数属性比 nonlocal 直观；F.state 一行就能看到当前值，而 nonlocal 版本只能再跑一次函数才知道。
- 初学者误区：①以为 inner.state += 1 需要 nonlocal inner——不需要，这是属性修改不是名字赋值；②把状态初始化写在嵌套 def 之前——会失败，因为那时 inner 函数对象还没创建；③以为"列表 hack" (state = [start]) 是高招——Lutz 直言它"从晦涩 hack 到过时变通"，能用

函数属性就别用下标魔法。

代码分析

```
>>> def outer(start):  
...     def inner(label):  
...         inner.state += 1          # inner  
...         print(label, inner.state) # inner  
...     inner.state = start           # inner  
...     return inner
```

解释器如何处理：①嵌套 `def inner` 执行后，函数对象创建完毕，`inner` 这个名字绑定到它（`outer` 的局部变量）；②`inner.state = start`：往函数对象 `dict` 写入 `{'state': 0}`——注意它位于 `def` 之后，因为在 `def` 执行前函数对象还不存在；③`F('attr1'): inner.state += 1` 展开为读属性（0）→ 加 1 → 写属性（1），打印 `attr1 1`。整个过程中 `inner` 这个名字从未被赋值（`inner.state` 只是取它的属性），所以编译器不把 `inner` 判为局部、也不要求 `non local`——即使 `inner` 恰好就是外围作用域中的名字。

设计思想：这段代码把“对象 vs 名字”的区分用到极致：名字 `inner` 在外围作用域（E 层找到），属性 `state` 在对象上（任意修改）。函数因此成为“自带属性字典的轻量对象”——这是装饰器（第 39 章）中记录函数元数据（如调用次数）的底层技术。

17.25 类：可修改、每调用独立、OOP (Classes: Changeable, Per-Call, OOP)

英文原文

Another standard prescription for changeable state information in Python is to use classes with attributes. Like function attributes, this scheme makes state information access more explicit than the implicit magic of scope lookup rules. In addition, each instance of a class gets a fresh copy of the state information, as a natural byproduct of Python's object model. Classes also support inheritance, multiple behaviors, and other OOP tools above and beyond functions. As an abstract and partial illustration, the following defines two stateful instances of a class:

```
python class Book: ...  
code to define method functions that manage and use state ...  
lp6e = Book() # Make an instance of the class  
lp6e.year = 2024 # Access instance state via attributes  
lp6e.python = 3.12  
lp5e = Book() # Make another instance of the class  
lp5e.year = 2013 # Each instance has its own attributes/state  
lp5e.python = 3.3
```

Because classes support a broader array of tools, they tend to require more code than closure functions, but may be better in more demanding roles. We haven't explored classes in any sort of detail yet, though, so we'll have to cut this section short here. Watch for classes, and their explicit flavor of multiple-copy state information, in Part VI.

中文翻译

另一种在 Python 中处理可变状态信息的标准处方是使用带属性的类。与函数属性一样，这个方案让状态信息的访问比作用域查找规则的隐式魔法更显式。此外，类的每个实例都会得到状态信息的新鲜副本——这是 Python 对象模型的自然副产品。类还支持继承 (inheritance)、多重行为，以及函数之外的其它 OOP 工具。作为一个抽象而部分性的示例，下面定义了一个类的两个有状态实例：

```
python class Book: ...定义管理并使用状态的方法函数的代码...
lp6e = Book() # 制造类的一个实例
lp6e.year = 2024 # 通过属性访问实例状态
lp6e.python = 3.12
lp5e = Book() # 制造类的另一个实例
lp5e.year = 2013 # 每个实例都有自己的属性/状态
lp5e.python = 3.3
```

因为类支持更广泛的工具集，它们往往比闭包函数需要更多代码，但在要求更高的角色中可能更好。不过我们还没有详细探索过类，所以本节只能就此打住。请关注第 VI 部分中的类，以及它们那种显式的多副本状态信息风格。

深度理解

- 核心概念：类方案 = "状态挂在实例上"。每个实例 (lp6e、lp5e) 都是独立的属性包——lp6e.year 与 lp5e.year 互不干扰。这是"多副本状态"最显式、最成体系的表达。
- 底层实现：Book() 调用创建实例对象，每个实例有独立的属性字典 (instance.dict) 。lp6e.year = 2024 写实例字典；方法函数通过 self 参数访问实例属性。与闭包的 cell 相比，实例状态是"命名属性"，天然可读可写可枚举 (vars(lp6e) 一览无余) 。

·为什么这样设计：类是"有状态的调用对象"的完整工业方案——状态（属性）+ 行为（方法）+ 复用（继承）。闭包适合轻量场景，类适合"状态+方法都要多、还要扩展"的场景。Lutz 说类"代码更多但更强"。

·解决什么实际问题：当状态需要携带行为（方法）、需要继承扩展、需要公开接口时，类胜过闭包。第 VI 部分将全面展开。

·初学者误区：①以为闭包与类"二选一旦闭包更好"——轻量状态用闭包、完整体系用类，按需选择；②混淆"实例属性"与"类属性"——实例属性才是 per-object 状态（类属性是所有实例共享的，这是第 29 章的大坑）；③以为 `lp6e.year` 与 `lp5e.year` 共享——不共享，每个实例各一份。

17.26 谁是赢家？ (And the Winner Is...)

英文原文

As a wrap-up, globals, nonlocals, function attributes, and classes all offer changeable state-retention options. Globals support only single-copy shared data, but nonlocals, function attributes, and classes all support multiple-copy changeable state. Of the latter, nonlocals rely on an implicit LEGB lookup, function attributes are manual but explicit and allow state access outside of functions, and classes are a larger solution that comes with OOP's complexities. As usual, the best to

ol for your program depends upon your program's goals. We've seen that `nonlocal` provides changeable state for nested functions with a dedicated statement, and is especially useful for simpler state-retention needs where global variables do not apply and classes may not be warranted. That being said, function attributes can often serve the same roles as `nonlocal`, with arguably less implicit behavior. We'll revisit state-retention options introduced here in Chapter 39 for a more realistic use case—decorators, a tool that by nature involves multilevel state retention. State options have additional selection factors (e.g., performance), which we'll have to leave unexplored here for space (you'll learn how to time code speed in Chapter 21). For now, it's time to explore one more technique, which both confounds the LEGB rule and segues to the next chapter.

中文翻译

作为总结：全局变量、`nonlocal`、函数属性和类都提供了可修改的状态保留方案。全局变量只支持单副本共享数据，而 `nonlocal`、函数属性和类都支持多副本的可修改状态。在后者中，`nonlocal` 依赖隐式的 LEGB 查找，函数属性是手动的但显式、并允许在函数之外访问状态，类则是一个伴随 OOP 复杂性的更大方案。一如既往，你的程序的最佳工具取决于程序的目标。我们已经看到：`nonlocal` 用一个专门的语句为嵌套函数提供可修改状态，在“全局变量用不上、类又杀鸡用牛刀”的较简单状态保留需求中尤其有用。话

虽如此，函数属性往往也能扮演 `nonlocal` 的角色，而且隐式行为可以说更少。我们将在第 39 章用更真实的用例——装饰器（decorators），一种本质上涉及多层状态保留的工具——重访这里介绍的状态保留方案。状态方案还有额外的选择因素（例如性能），限于篇幅我们只能留待将来（你将在第 21 章学到如何测量代码速度）。现在，该探索最后一项技术了——它既扰乱了 LEGB 规则，又为下一章做了铺垫。

深度理解

·核心概念：四方案横向对比的核心结论："多副本"能力是分水岭——只有 `global` 是单副本（共享），其余三者（`nonlocal`、函数属性、类）都是多副本。多副本 vs 共享，是选择的第一判据；显式性 vs 隐式性，是第二判据。

·对比表格速记：`global` = 共享、简单、易耦合；`nonlocal` = 私密、隐式、轻量；函数属性 = 显式、可外查、古老；类 = 体系化、可继承、代码多。

·为什么这样设计：Python 用"一个概念多种实现"回应不同场景——从"整个程序只有一份"（全局）到"每个对象一份"（类），从"零语法"（`global`）到"专有语句"（`nonlocal`）。没有银弹，只有取舍。

·解决什么实际问题：选型指南——简单单例状态用 `global`；闭包内私有可变状态用 `nonlocal`；需要外部可观测状态用函数属性；完整对象体系用类。装饰器（第 39 章）里这四种方案会全部登场。

·初学者误区：①追求"唯一最优解"——没有；②忽略性能

与代码量的权衡（本书没展开，第 21 章会教你计时）；
③把 nonlocal 当"高级"、函数属性当"hack"——两者各有主场，函数属性甚至更古老、更显式。

17.27 作用域与参数默认值 (Scopes and Argument Defaults)

英文原文

In early versions of Python, the enclosing scope references we've used in this chapter failed because nested functions did not do anything about scopes—a reference to an enclosing function's variable would search only the local, global, and built-in scopes. Because it skipped the scopes of enclosing functions, an error would result. To work around this, programmers typically used default argument values to pass in and remember the objects in an enclosing scope. Though a preview of the next chapter's arguments coverage, this also bears on scopes. In the following, a takeoff of an earlier example, a value from the enclosing function's scope is passed into a nested function via an unused argument's default:

```
python def f1(): X = 88 def f2(X=X): # Remember enclosing scope X with defaults print(X + 1) f2() f1() # Prints 89
```

This syntax also works in lambda, which, as we've learned, naturally and normally creates nested-function scopes:

```
python def f1(): X = 88 f2 = lambda X=X: print(X + 1)
f2()
```

Both of these examples still work in all Python releases, and rely on argument defaults. In short, the syntax `arg=val` in a function header means that the argument `arg` will default to the value `val` if no real value is passed to `arg` in a call. In the preceding code, this is used to explicitly assign enclosing scope state to be retained. Specifically, in the modified `f2s` here, the `X=X` means that the argument `X` on the left will default to the value of `X` in the enclosing scope—because the `X` on the right is evaluated before Python steps into the nested function, it still refers to the `X` in `f1`. In effect, each `f2`'s default argument `X` remembers what `X` was in the enclosing `f1`: the object 88. That's fairly subtle, and it depends entirely on the timing of default-value evaluations. It's also an incidental—if not accidental—feature: this works only if a real value is never passed to argument `X` to overwrite the default. In fact, the nested scope lookup rule was added to Python in part to make defaults unnecessary for this role: today, Python automatically remembers any values required from the enclosing scope for use in nested `defs` and `lambdas`. As we've seen, this example today works the same sans defaults:

```
python def f1(): X = 88 def f2(): # Remember enclosing scope X per LEGB rule print(X + 1) # And likewise f or lambda f2()
```

That said, flat is generally better than nested again, and function nesting in some such code makes programs more complex than they need be. The following, for instance, is an equivalent of the prior examples that avoids nesting altogether. Notice the forward reference to `f2` inside `f1` in this code—it's OK to call a function defined after the function that calls it, as long as the second `def` runs before the first function is actually called. Code inside a function's body is never evaluated until the function is later called:

```
python def f1(): X = 88 # Pass x along instead of nesting f2(X) # Forward reference OK
```

```
def f2(X): print(X + 1) # Flat is still often better than nested
```

If you avoid nesting this way, you can almost forget about the nested scopes concept in Python. On the other hand, nested functions can avoid name clashes by localizing the names of functions used nowhere else, and nesting is the basis of closure functions, which support stateful callable objects useful in a variety of roles. When functions are nested for such reasons, the LEGB rule almost makes defaults unnecessary for saving state from an enclosing function's scope—except in the following case.

中文翻译

在早期版本的 Python 中，本章使用的外围作用域引用会失败，因为当时嵌套函数对作用域什么也不做——引用外围函数的变量只会搜索局部、全局和内置作用域。由于它跳过了外围函数的作用域，结果就是报错。为了绕过这个问题，程序员通常使用参数默认值（default argument values）把外围作用域中的对象传入并记住。虽然这是下一章参数内容的预告，但它也与作用域相关。下面是对前面示例的改编：外围函数作用域中的一个值通过一个未使用的参数的默认值传入嵌套函数：python

```
def f1(): X = 88
def f2(X=X):
    # 用默认值记住外围作用域中的 X
    print(X + 1)
f2()
f1()
# 打印 89
```

这个语法在 lambda 中同样有效——我们已经知道，lambda 自然而正常地创建嵌套函数作用域：python

```
def f1(): X = 88
f2 = lambda X=X: print(X + 1)
f2()
```

这两个示例在所有 Python 版本中仍然有效，并且依赖参数默认值。简而言之，函数头部的 `arg=val` 语法意味着：如果在调用中没有向 `arg` 传入真实值，参数 `arg` 就默认为值 `val`。在上面的代码中，这被用来显式地赋值并保留外围作用域状态。具体来说，在修改后的 `f2` 中，`X=X` 意味着左边的参数 `X` 默认取外围作用域中 `X` 的值——因为右边的 `X` 在 Python 进入嵌套函数之前就被求值，它仍然指 `f1` 中的 `X`。实际上，每个 `f2` 的默认参数 `X` 都记住了 `f1` 外围作用域中 `X` 当时的值：对象 88。这相当微妙，而且完全取决于默认值求值的时机（timing）。它还是一个附带（incidental）——甚至可以说是意外——的特性：它只有在永远不会向参数 `X` 传真实值来覆盖默认值时才能工作。事实上，嵌套作用域查找规则被加入 Python，部分原因正是为了让默认值

不再承担这一角色：今天，Python 会自动记住嵌套 `def` 和 `lambda` 需要的外围作用域中的任何值。正如我们所见，同样的例子今天不用默认值也能工作：`python def f1(): X = 88 def f2(): # 按 LEGB 规则记住外围作用域中的 X print(X + 1) # lambda 同理f2()` 话虽如此，“扁平优于嵌套”依然是普遍原则，这类代码中的函数嵌套会让程序比需要的更复杂。例如，下面是前面示例的等价写法，完全避免了嵌套。注意这段代码中 `f1` 内部对 `f2` 的前向引用（forward reference）——调用一个定义在调用它的函数之后的函数是没问题的，只要第二个 `def` 在第一个函数被真正调用之前执行过。函数体内的代码在函数被调用之前永远不会被求值：`python def f1(): X = 88 # 把 X 传下去，而不是嵌套f2(X) # 前向引用没问题def f2(X): print(X + 1) # 扁平往往仍优于嵌套`如果你这样避免嵌套，就几乎可以忘掉 Python 的嵌套作用域概念了。另一方面，嵌套函数可以通过局部化“别处不用的函数名”来避免名字冲突，而且嵌套是闭包函数的基础——闭包函数支持在多种角色中有用的有状态可调用对象。当函数出于这些原因嵌套时，LEGB 规则几乎让“用默认值保存外围函数作用域状态”变得没有必要——除了下面这种情况。

深度理解

·核心概念：历史回眸——`X=X` 默认值技巧是“LEGB 时代之前”的手动闭包方案：利用默认值在函数定义时（而非调用时）求值的时机，把外层变量的当前值“焊”进内层函数的默认参数。如今被 LEGB 的 E 层取代，仅剩一个场景仍必需。

·底层实现：默认值是函数对象的属性（func.defaults 元组）。def f2(X=X) 编译时，右边的 X 在定义时刻求值——那时 f1 的栈帧还活着，取到 88，存入 f2.defaults。之后 f1 退场也没关系：88 已经在函数对象上了。注意这里的默认值不是 cell——是普通参数，每次调用从 defaults 取。而"前向引用"能成立，是因为 def 是可执行语句：模块执行完 def f2 之前 f1 只是"定义好了但没运行"。

·为什么这样设计：默认值求值时机（定义时）是 Python 的既定规则（第 18 章详述，也是"可变默认值陷阱"的来源）。早期 Python 没有 E 层，程序员"用规则解决问题"发明了此技巧；后来语言进步（PEP 227，Python 2.1）把嵌套作用域设为默认，技巧退居历史舞台。

·解决什么实际问题：读懂老代码（很多库用 lambda x=x：模式）需要知道这个技巧；而"循环中生成闭包"的场景（下一节）至今必须用它——这是它仅存的、不可替代的战场。

·初学者误区：①以为 X=X 是"给参数起别名"——不，右边是定义时的求值，左边是参数名；②以为嵌套一定能解决问题——在循环里生成的闭包，LEGB 反而会"找错值"（下一节的经典陷阱）；③以为 def 语句是"执行到才存在"是错的——准确说：def 执行前函数不存在，但定义可以在调用之后（前向引用合法），因为调用发生在运行期、定义发生在模块加载期。

代码分析

```
def f1():
    X = 88
    def f2(X=X):          # X
        print(X + 1)
    f2()
f1()                      # 89
```

解释器如何处理：执行到 `def f2(X=X)` 时：①先求值默认值表达式——右边的 `X` 走 LEGB（此时 `f1` 栈帧活着），取到 88；②创建 `f2` 函数对象，`f2.defaults = (88,)`；③`f2()` 调用时未传参数，`X` 从 `defaults` 取 88，`print(X+1)` 输出 89。对比现代写法 `def f2():`：`X` 走 `LOADDEREF` 从 `cell` 取 88，殊途同归。区别在于：默认值是“定义时的快照”（此后 `f1` 再改 `X` 也不影响 `f2`），`cell` 是“实时引用”（`f1` 之后修改 `cell` 里的值，`f2` 看得到）。

设计思想：“快照 vs 引用”正是默认值技巧与现代闭包的本质差异。下节的循环陷阱正是由于 `cell` 的“实时引用”特性——所有闭包共享同一个被循环改写的 `cell`，快照（默认值）才是解药。

17.28 循环需要默认值，而不是作用域 (Loops Require Defaults, Not Scopes)

英文原文

So why bother learning an outdated scope-reference scheme? Because it's still required in one common case: if a lambda or def is nested in a loop, and the nested function references an enclosing scope variable t

that is changed by that loop, then all functions generated within the loop will have the same value—the value the referenced variable had in the last loop iteration. In such cases, you must still use defaults to save the variable's current value instead. This may seem obscure, but it can come up in practice more often than you may think, especially in code that generates event-handler functions for a number of widgets in a GUI—for instance, handlers for button-clicks for all the buttons in a panel. If these are created in a loop (and they will often be), you need to be careful to save state with defaults, or all your buttons' callbacks may wind up doing the same thing. Here's an illustration of this phenomenon reduced to simple code: the following attempts to build up a list of functions that each remember the current variable `i` from the enclosing scope:

```
python >>> def makeActions(): ... acts = [] ... for i in range(5): # Try to remember each i in 0..4 ... acts.append(lambda x: i x) # But all remember same last i! ... return acts >>> acts = makeActions() >>> acts[0] <function makeActions.<locals>.<lambda> at 0x101ae7ec0>
Interestingly, this can also be coded as a list comprehension, where a lambda can be used as the collection result, and the list comprehension serves as a local scope for its loop variable:
```

```
python >>> acts = [(lambda x: i x) for i in range(5)] >>> acts[0] <function <lambda> at 0x10de6bce0>
Eit
```


her way, though, this doesn't quite work—because the enclosing scope variable `i` is looked up when the nested functions are later called, they all effectively remember the same value: the value the loop variable had on the last loop iteration. That is, when we pass a power argument of 2 to `x` in each of the following calls, we get back 4 to the power of 2 for each function in the list, because `i` is the same in all of them—4:

```
python >>> acts0 # All are 4 2, 4=last i 16
>>> acts1 # This should be 1 2 (1) 16
>>> acts2 # This should be 2 2 (4) 16
>>> acts4 # Only this should be 4 2 (16) 16
```

In this case, we still have to explicitly retain enclosing scope values with default arguments, rather than enclosing scope references. That is, to make this sort of code work, we must pass in the current value of the enclosing scope's variable with a default. Here's the required mod for both the `def` and comprehension versions:

```
python >>> def makeActions(): ... acts = [] ... for i in range(5): # Use defaults instead ... acts.append(lambda x, i=i: i x) # Remember current i ... return acts
>>> acts = [(lambda x, i=i: i x) for i in range(5)]
In either coding, because defaults are evaluated when the nested function is created (not when it's later called), each remembers its own value for i:
```

```
python >>> acts = makeActions() >>> acts0 # 0 2 0
>>> acts1 # 1 2 1
>>> acts2 # 2 2 4
>>> acts4 # 4 2 16
```

Nor are function attributes a fix here: because a function's own name is a variable from the enclosing scope that changes in the loop too, in every function it may reference the last function made by the last loop iteration. We'll omit the gory details here, but keep in mind that any enclosing scope reference changed by a loop may require defaults. This may seem an odd special case, but it reflects Python's implementation of variable scopes, and will become more likely to crop up as you start writing larger programs. This case is also rooted in both scopes and arguments defaults, and to understand the latter in full, we have to move on to the next chapter. NOTE State with argument-defaults: Also on a related note, it's possible to retain state too with mutable argument defaults like lists and dictionaries (e.g., `def f(a=[])`). Because defaults are implemented as objects attached to functions at function creation time, mutable defaults retain state from call to call, rather than being initialized anew on each call. Defaults can also retain mutables from an enclosing function's scope, thereby enabling changeable per-call state information. Depending on whom you ask—and when you ask them—this is either a feat

ure that supports state retention, or a perilous and dark corner of the language to be avoided. Usually, programmers expect defaults initialized with literals like [] to be re-created on every call, and are surprised when they retain prior calls' values. More on this in "Function Gotchas".

中文翻译

那么为什么还要费心学习一套过时的作用域引用方案？因为在一个常见场景中它仍然必不可少：如果 lambda 或 def 嵌套在循环中，并且嵌套函数引用的外围作用域变量被该循环修改，那么循环内生成的所有函数都将具有相同的值——即被引用变量在最后一次循环迭代时的值。在这种情况下，你仍然必须用默认值来保存变量的当前值。这看起来可能很冷门，但它在实践中出现的频率可能超出你的想象，尤其是在“为一组 GUI 组件生成事件处理器函数”的代码中——例如为面板上所有按钮生成点击处理器。如果这些处理器是在循环中创建的（而且它们常常如此），你就必须小心地用默认值保存状态，否则所有按钮的回调最终可能都在做同一件事。下面把这个现象简化为一段简单代码来演示：下面的代码试图构建一个函数列表，让每个函数记住外围作用域中变量 i 的当前值：python >>> def makeActions(): ... acts = [] ... for i in range(5): # 试图记住 0..4 的每一个 i ... acts.append(lambda x: i x) # 但全都记住了最后一个 i! ... return acts >>> acts = makeActions() >>> acts[0] <function makeActions.<locals>.<lambda> at 0x101ae7ec0> 有趣的是，这也可以用列表推导式（list comprehension）编写——lambda 可以作为集合结果，而列表推导

式为它的循环变量充当局部作用域：python >>> acts = [(lambda x: i x) for i in range(5)] >>> acts[0] <function <lambda> at 0x10de6bce0> 但无论哪种写法，这都不能真正工作——因为外围作用域变量 i 是在嵌套函数稍后被调用时才查找的，所以它们实际上都记住了同一个值：循环变量在最后一次循环迭代时的值。也就是说，在下面每次调用中我们向 x 传的幂参数都是 2，但列表中的每个函数都返回 4 的 2 次方，因为它们而言 i 相同——都是 4：python >>> acts0 # 全是 4 2, 4 = 最后的 i 16 >>> acts1 # 本应是 1 2 (1) 16 >>> acts2 # 本应是 2 2 (4) 16 >>> acts4 # 只有这个本应是 4 2 (16) 16 在这种情况下，我们仍然必须用默认参数显式保留外围作用域的值，而不是依赖外围作用域引用。也就是说，要让这类代码工作，我们必须用默认值传入外围作用域变量的当前值。下面是 def 版本和推导式版本所需的修改：python >>> def makeActions(): ... acts = [] ... for i in range(5): # 改用默认值... acts.append(lambda x, i=i: i x) # 记住当前的 i ... return acts >>> acts = [(lambda x, i=i: i x) for i in range(5)] 在两种写法中，因为默认值在嵌套函数创建时（而不是稍后调用时）被求值，每个函数都记住了自己那份 i 的值：python >>> acts = makeActions() >>> acts0 # 0 2 0 >>> acts1 # 1 2 1 >>> acts2 # 2 2 4 >>> acts4 # 4 2 16 函数属性在这里也救不了场：因为函数自身的名字也是外围作用域中随循环变化的变量，每个函数里它引用的都可能是最后一次循环迭代制作的最后一个函数。这里的细节我们就不展开了，但要记住：任何被循环修改的外围作用域引用都可能需要默认值。这看起来像是一个古怪的特例，但它反映了 Python 对变量作用域的实现方式，而且随着你开始编写

更大的程序，它出现的可能性会越来越大。这个情形同时植根于作用域和参数默认值，要全面理解后者，我们必须进入下一章。注意用可变参数默认值保存状态：顺带一提，也可以用可变参数默认值（如列表和字典，例如 `def f(a=[])`）来保留状态。因为默认值是在函数创建时附加到函数对象上的对象，可变默认值会在调用之间保留状态，而不是在每次调用时重新初始化。默认值也可以保留来自外围函数作用域的可变对象，从而实现可修改的每调用状态信息。取决于你问谁——以及你什么时候问——这要么是支持状态保留的特性，要么是语言中一个应当避开的危险黑暗角落。通常，程序员期望用 `[]` 这样的字面量初始化的默认值在每次调用时重新创建，当它们保留了之前调用的值时都会很惊讶。更多内容见“函数的坑”（Function Gotchas）。

深度理解

·核心概念：迟绑定（late binding）陷阱——闭包捕获的是变量（cell），不是值；`i` 只有一个 cell，循环每次迭代都在改写它，所有闭包调用时读到的都是最终值 4。解法是默认值早绑定（early binding）：`i=i` 在创建时把当前值快照进 defaults。

·底层实现：`for i in range(5)` 不创建新作用域（17.5 节），`i` 是 `makeActions` 的一个局部变量；被 `lambda` 引用后升级为 cell。5 次迭代 = 5 次 `STOREDEREF i`，cell 最终值 4。5 个 `lambda` 的 closure 指向同一个 cell——这是它们“同病相怜”的根源。加了 `i=i` 后，每个 `lambda` 的 defaults 分别是 0、1、2、3、4（创建时求值），调用时 `LOADFAST i` 取参数（默认值兜底），与 cell 无关了。

·为什么这样设计：变量捕获（而非值捕获）是词法作用域的标准语义（JavaScript、Lua 等同样如此），它让“闭包跟随变量变化”成为可能（这正是 17.19 节计数器依赖的行为）。代价就是这个循环陷阱——所以需要默认值技巧显式“值捕获”。

·解决什么实际问题：GUI 事件循环里为每个按钮生成回调（`button.onclick(lambda: handler(i))` 全部点击同一个按钮的经典事故）；批量注册处理器、模板化工厂。所有“在循环里造闭包”的代码都要过这道关。

·初学者误区：①以为每个 `lambda` 各有一份 `i`——只有一个 `cell`、一份 `i`；②以为列表推导式版本“绕开了”陷阱——没有，推导式的 `i` 也是同一个循环变量（虽然推导式有局部作用域，但那是对外层不可见，循环内仍共享）；③以为“等到调用时才求值”是想要的——不，这正是病根；④想用函数属性绕过——函数名字本身也是循环中变化的变量，同样中招。

·生活例子：闭包像“看着同一块公告板写作业的学生”——循环每迭代一次，公告板上的数字就改一次，学生交作业（调用）时看到的都是最后写上去的 4。默认值技巧则是“每人发一张小纸条，上面抄着当时的数字”。

代码分析

```
>>> def makeActions():
...     acts = []
...     for i in range(5):          # 0..4 i
...         acts.append(lambda x: i ** x)  # i
...     return acts
>>> acts[0](2)                      # 4 ** 24 = i
16
```

解释器如何处理：①编译 makeActions: i 被 lambda 引用且被 for 赋值 → 升级为 cell 变量；②循环 5 次：每次迭代 STOREDEREF i 改写 cell (0→1→2→3→4)，同时 MAKEFUNCTION 创建 5 个 lambda，每个的 closure 都引用同一个 cell；③makeActions() 返回后，cell 最终值 4；④acts0 调用：LOADDEREF i 读 cell → 4，BINARARYOP → 16。修复版 lambda x, i=i: i x：创建时右侧 i 求值（此时 cell 分别为 0/1/2/3/4），存入各 lambda 的 defaults；调用时 i 作为参数被绑定到默认值，LOADFAST 取参数——输出 0/1/4/16 各归其位。

设计思想：这个陷阱的教训是“闭包捕获的是变量而非值”。理解 cell 机制后，陷阱不再是“玄学”而是必然推论；而 i=i 技巧之所以有效，靠的是“默认值在创建时求值”这条规则——两条规则相遇，问题迎刃而解。

17.29 本章小结 (Chapter Summary)

英文原文

In this chapter, we studied one of two key concepts related to functions: scopes, which determine how var

ables are looked up when used. As we learned, variables are considered local to the function definitions in which they are assigned, unless they are specifically declared to be global or nonlocal. We also explored some more advanced scope concepts here, including nested function scopes and function attributes. Finally, we looked at some general design ideas, such as the need to minimize globals and cross-file changes. In the next chapter, we're going to continue our function tour with the second key function-related concept: argument passing. As you'll find, arguments are passed into a function by assignment, but Python also provides tools that allow functions to be flexible in how items are passed, including the defaults we previewed here. Before we move on, let's take this chapter's quiz to review the scope concepts we've covered here.

中文翻译

本章研究了与函数相关的两大关键概念之一：作用域（scopes）——它决定变量被使用时如何查找。正如我们学到的：变量被视为“赋值它们的函数定义”的局部变量，除非被显式声明为 global 或 nonlocal。我们还探索了一些更高级的作用域概念，包括嵌套函数作用域和函数属性。最后，我们考察了一些通用设计思想，例如尽量减少全局变量和跨文件修改。下一章我们将继续函数之旅，研究第二个与函数相关的关键概念：参数传递（argument passing）。你会发现，参数是通过赋值传入函数的，但 Python 还提供了一些工具，让函数在“项目如何被传递”上拥有灵活性，包括我们这

里预览过的默认值。继续之前，让我们做一下本章的测验，回顾这里覆盖的作用域概念。

17.30 测验：检验你的知识 (Test Your Knowledge: Quiz)

英文原文

1. What is the output of the following code, and why?

```
python >> X = 'Hack' >> def func():  
... print(X) >> func()
```

2. What is the output of this code, and why?

```
python >> X = 'Hack' >> def func():  
... X = 'Py!' >> func() >> print(X)
```

3. What does this code print, and why?

```
python >> X = 'Hack' >> def func():  
... X = 'Py!' ... print(X) >> func() >> print(X)
```

4. What output does this code produce? Why?

```
python >> X = 'Hack' >> def func():  
... global X ... X = 'Py!' >> func() >> print(X)
```

5. What about this code—what's the output, and why?

```
python >> X='Hack'>> def func():
```

```
... X='Py!'... def nested(): ... print(X) ... nested() >> func() >> print(X)
```

6. How about this example: what is its output, and why?

```
python >> def func():
```

```
... X='Py!'... def nested(): ... nonlocal X ... X='Hack'... nested() ... print(X) >> func()
```

7. Name three or more ways to retain state information across calls in a Python function.

中文翻译

1. 下面代码的输出是什么？为什么？
python >>> X='Hack'>>> def func(): ... print(X) >>> func()
2. 这段代码的输出是什么？为什么？
python >>> X='Hack'>>> def func(): ... X='Py!'>>> func() >>> print(X)
3. 这段代码打印什么？为什么？
python >>> X='Hack'>>> def func(): ... X='Py!'... print(X) >>> func() >>> print(X)
4. 这段代码产生什么输出？为什么？
python >>> X='Hack'>>> def func(): ... global X ... X='Py!'>>> func() >>> print(X)
5. 这段代码呢——输出是什么？为什么？
python >>> X='Hack'>>> def func(): ... X='Py!'... def nested(): ... print(X) ... nested() >>> func() >>> print(X)
6. 这个例子呢：它的输出是什么？为什么？
python >>> def func(): ... X='Py!'... def nested(): ... nonlocal X ... X='Hack'... nested() ... print(X) >>> func()
7. 说出三种或更多种在 Python 函数调用之间保留状态信息的方式。

17.31 测验答案 (Test Your Knowledge: Answers)

英文原文

1. The output here is Hack because the function references a global variable in the enclosing module (because it is not assigned in the function, it is considered global). 2. The output here is Hack again because assigning the variable inside the function makes it a local and effectively hides the global of the same name. The print statement finds the variable unchanged in the global (module) scope. 3. It prints Py! on one line and Hack on another, because the reference to the variable within the function finds the assigned local and the reference in the print statement finds the global. 4. This time it just prints Py! because the global declaration forces the variable assigned inside the function to refer to the variable in the enclosing global scope, even though the variable is assigned inside the function. 5. The output in this case is again Py! on one line and Hack on another, because the print statement in the nested function finds the name in the enclosing function's local scope, and the display at the end finds the variable in the global scope. 6. This example prints Hack because the nonlocal statement means that the assignment to X inside the nested function chang

es X in the enclosing function's local scope. Without this statement, this assignment would classify X as local to the nested function, making it a different variable; the code would then print Py! instead. 7. Although the values of local variables go away when a function returns, you can make a Python function retain state information by using shared global variables, nonlocal enclosing scope references within nested functions, or using default argument values. Function attributes also allow state to be attached to the function itself, instead of looked up in scopes. Another alternative, using classes and OOP, sometimes supports state retention better than any of the scope-based techniques because it makes it explicit with attribute assignments; we'll explore this option in Part VI. Changing mutable objects in scopes and defaults works too, but may not be legal in some locales.

中文翻译

1. 这里输出 Hack，因为函数引用了外围模块中的全局变量（因为它没有在函数中被赋值，所以被视为全局的）。2. 这里输出还是 Hack，因为在函数内赋值使该变量成为局部变量，并有效地遮蔽了同名的全局变量。print 语句在全局（模块）作用域中找到的变量未受影响。3. 它打印两行：一行 Py!、一行 Hack，因为函数内对变量的引用找到了赋值的局部变量，而 print 语句中的引用找到了全局变量。4. 这次只打印 Py!，因为 global 声明强制函数内赋值的变量指向外围全局作用域中的那个变量——即使这个变量是在

函数内部被赋值的。5. 这个案例的输出同样是两行：一行 Py!、一行 Hack，因为嵌套函数中的 print 语句在外围函数的局部作用域中找到了名字，而结尾的显示在全局作用域中找到了变量。6. 这个例子打印 Hack，因为 nonlocal 语句意味着嵌套函数内对 X 的赋值修改的是外围函数局部作用域中的 X。如果没有这个语句，该赋值会把 X 归类为嵌套函数自己的局部变量，使它成为不同的变量；那么代码将改为打印 Py!。7. 虽然局部变量的值在函数返回时会消失，但你可以通过以下方式让 Python 函数保留状态信息：使用共享的 global 变量、嵌套函数中的 nonlocal 外围作用域引用，或使用默认参数值。函数属性也允许把状态附加到函数自身，而不是在作用域中查找。另一种替代方案——使用类和 OOP——有时比任何基于作用域的技术都更好地支持状态保留，因为它通过属性赋值把状态显式化；我们将在第 VI 部分探索这个方案。修改作用域和默认值中的可变对象也能工作，但在某些地方可能不合法。

深度理解（七题逐一拆解）

题号	输出	机制	一句话结论
1	Hack	只读 → LEGB 找到 G 层	不赋值，就是全局
2	Hack	函数内赋值 → 局部，遮蔽全局	函数返回后局部消失，全局不变
3	Py! / Hack	函数内读局部，函数外读全局	内外各读各的
4	Py!	global 把赋值重定向到模块	声明后修改的就是全局
5	Py! / Hack	嵌套函数读到 E 层 (func 的局部 Py!)	E 层优先于 G 层
6	Hack	nonlocal 让嵌套函数修改外层局部	改的是同一个变量

7	—	四种方案：global / nonlocal / 默认值 / 函数属性 (+类、可变对象)	状态保留全家桶
---	---	---	---------

重点复盘：第 5 题最容易错——nested 里的 X 走 E 层命中 func 的局部 X = 'Py!'，根本到不了全局的'Hack'。第 6 题则提醒：若去掉 nonlocal，nested 里 X = 'Hack' 会新建一个局部 X，func 打印的仍是'Py!'——nonlocal 是"修改外层"与"新建内层"的分水岭。

17.32 为什么你会关心：定制 open (Why You Will Care: Customizing open)

英文原文

For another example of closures at work, consider changing the built-in open call to a custom version as suggested earlier in this chapter. If the custom version needs to call the original, it must save it before changing it, and retain it for later use—a classic state retention scenario. Moreover, if we wish to support multiple customizations to the same function, globals won't do: we need per-customizer state.

The following, coded in file makeopen.py, is one way to achieve this. It uses a nested scope closure to remember a value for later use, without relying on global variables—which can clash and allow just one value, and without using a class—that may require more co

de than is warranted here:

```
python import builtins def makeopen(id): original = b
uiltins.open

def custom(pargs, kargs): print(f'Custom open call {id
}', pargs, kargs) return original(pargs, kargs) builtins.o
pen = custom
```

To change open for every module in a process, this code reassigns it in the built-in scope to a custom version coded with a nested def, after saving the original in the enclosing scope so the customization can call it later.

This code is partially a preview, as it relies on starred-argument forms to collect and later unpack arbitrary positional and keyword arguments meant for open—a topic coming up in the next chapter. Much of the magic here, though, is scope closures: the custom open found by the LEGB rule retains the original:

```
python >> file='../Chapter14/data.txt'>> F = open(fi
le) # Call built-in open in builtins >> F.read()'Testing
file IO\nLearning Python, 6E\nPython 3.12\n'>> from
makeopen import makeopen # Import open resetter
function >> makeopen('MOD1') # Custom open calls
built-in open >> F = open(file) # Call custom open in
builtins Custom open call MOD1 ('../Chapter14/data.t
xt',) {} >> F.read()'Testing file IO\nLearning Python, 6
E\nPython 3.12\n'
```

Because each customization remembers the former built-in scope version in its own enclosing scope, they can even be nested naturally in ways that global variables cannot support—each call to the `makeopen` closure function remembers its own versions of `id` and `original`, so multiple customizations may be run:

```
python >> makeopen('MOD2') # Nested customizers
work too >> F = open(file) # Because each retains its
own state Custom open call MOD2 ('../Chapter14/data
a.txt',) {} Custom open call MOD1 ('../Chapter14/data.
txt',) {} >> F.read()'Testing file IO\nLearning Python,
6E\nPython 3.12\n'
```

As is, our function simply adds possibly nested call tracing to a built-in function, but the general technique may have other applications. A class-based equivalent to this may require more code because it would need to save the `id` and `original` values explicitly in object attributes—but requires more background knowledge than we yet have; stay tuned for state retention in classes in this book's Part VI.

中文翻译

再看一个闭包实际工作的例子：如本章前面所建议的，考虑把内置的 `open` 调用换成自定义版本。如果自定义版本需要调用原版，就必须在替换之前保存原版，并把它留作以后使用——这是一个经典的状态保留场景。此外，如果我们希望支持对同一个函数的多次定制，全局变量就不行了：我们需要每个定制者各自的状态。下面这段写在 `makeopen.py`

文件中的代码，就是实现这一目标的一种方式。它用嵌套作用域闭包记住一个值供以后使用——既不依赖全局变量（全局变量会冲突、而且只允许一个值），也不用类（类可能需要比这里值得花费的更多代码）：`python`

```
import builtins
def makeopen(id):
    original = builtins.open
    def custom(pargs, kargs):
        print(f'Custom open call {id}', pargs, kargs)
        return original(pargs, kargs)
    builtins.open = custom
```

为了改变进程中每个模块的 `open`，这段代码把内置作用域中的 `open` 重新赋值为一个用嵌套 `def` 编写的自定义版本——在此之前，先把原版保存在外围作用域中，以便定制版本以后调用它。这段代码部分是预览：它依赖星号参数（starred-argument）形式来收集并在之后解包原本打算传给 `open` 的任意位置参数和关键字参数——这是下一章的主题。不过，这里的很多魔法在于作用域闭包：被 LEGB 规则找到的自定义 `open` 保留了原版：

```
python >>> file = '../Chapter14/data.txt'
>>> F = open(file) # 调用 builtins 中的内置 open
>>> F.read()
'Testing file IO\nLearning Python, 6E\nPython 3.12\n'
>>> from makeopen import makeopen # 导入 open 重置函数
>>> makeopen('MOD1') # 自定义 open 调用内置 open
>>> F = open(file) # 调用 builtins 中的自定义 open
Custom open call MOD1 ('../Chapter14/data.txt',) {}
>>> F.read()
'Testing file IO\nLearning Python, 6E\nPython 3.12\n'
```

因为每次定制都在自己的外围作用域中记住先前内置作用域中的版本，它们甚至可以像全局变量无法支持的方式那样自然嵌套——每次调用 `makeopen` 闭包函数都会记住自己版本的 `id` 和 `original`，所以可以运行多次定制：

```
python >>> makeopen('MOD2') # 嵌套定制同样有效
>>> F = ope
```

n(file) # 因为每个都保留自己的状态Custom open call MOD2 ('../Chapter14/data.txt',) {} Custom open call MOD1 ('../Chapter14/data.txt',) {} >>> F.read()'Testing file IO\nLearning Python, 6E\nPython 3.12\n'就目前而言，我们的函数只是给一个内置函数添加了（可能嵌套的）调用追踪，但这项通用技术可能还有别的应用。基于类的等价实现可能需要更多代码，因为它需要在对象属性中显式保存 id 和 original 值——不过这需要比我们目前掌握的更多背景知识；敬请期待本书第 VI 部分中关于类中状态保留的内容。

深度理解

·核心概念：一个“闭环”实战——LEGB 规则 + 闭包 + 状态保留的合体：makeopen 把内置作用域改名换姓，用闭包给每个定制者分发独立的 original/id 状态，还支持无限嵌套定制（这是全局变量做不到的：全局只有一个 open 槽位，链条会断）。

·底层实现：builtins.open = custom 修改的是 builtins 模块字典——所有模块的 open 查询（B 层）都会命中 custom。custom 的 closure 携带两个 cell：original（当时的 builtins.open 函数对象）和 id。第二次 makeopen('MOD2')：它捕获的 original 是第一次的 custom 函数（因为此刻 builtins.open 已被替换）——于是 MOD2 调用 MOD1 的 custom，MOD1 再调用真正的内置 open，形成调用链：输出两行“Custom open call”，内层先、外层后。pargs, kargs（下一章主题）负责把任意参数原样转发给原版。

·为什么这样设计：这是"猴子补丁（monkey patching）"的标准范式——语言通过"内置名字可重新绑定"和"闭包可保存原版"两个特性，让运行时替换内置函数成为可能，且每次替换都留有余地（链式调用）。

·解决什么实际问题：埋点/日志/审计（记录每次文件访问）、权限拦截（限制可打开路径）、测试替身（mock 内置 IO）。"替换后还能调用原版"是这类需求的灵魂。

·初学者误区：①以为 `makeopen('MOD2')` 后 MOD1 的追踪会丢失——不会，MOD1 的 cell 里存的是真内置 `open`，MOD2 存的是 MOD1 的 `custom`，各自链完整；②以为修改 `builtins` 只影响当前模块——影响进程内所有模块（这就是"全局替换"的含义）；③试图用全局变量实现同样效果——`original` 只有一个槽位，第二次定制会把第一次的覆盖掉，链断裂。

代码分析

```
import builtins
def makeopen(id):
    original = builtins.open      # open
    def custom(*pargs, **kargs):
        print(f'Custom open call {id}', pargs, kargs) #
        return original(*pargs, **kargs)             #
    builtins.open = custom        # open
```

解释器如何处理：①`makeopen('MOD1')`：栈帧中 `original` (cell) 捕获当前 `builtins.open` (真正的内置函数)；创建 `custom`，closure 带两个 cell (`id`、`original`)；随后 `builtins.open = custom` 写入 `builtins` 字典；②此后任何模块里写 `open(file)`，LEGB 到 B 层时查到的是 `cu`

stom; ③custom 执行: LOADDEREF id 打印 "MOD1", LOADDEREF original 取到真 open, CALL 转发; ④ makeopen('MOD2') 时 builtins.open 已是 MOD1 的 custom——MOD2 的 original cell 存的是它, 调用链自然嵌套: MOD2 → MOD1 → 内置。

设计思想: "保存原版 → 替换 → 转发"三步是猴子补丁的永恒模板, 而闭包让每一步的"原版"都按定制者隔离存放。这段代码也是下一章 (args/kwargs 参数收集与解包) 的绝佳预告。

脚注 (Footnotes)

脚注 1

英文原文: The scope lookup rule was branded the "LGB rule" in the first edition of this book. The enclosing "E" layer was added later in Python to obviate the task of passing in enclosing scope names explicitly with default arguments—an advanced topic that we'll sample later in this chapter. Since this scope is now addressed by the nonlocal statement, the lookup rule might have been better named "LNGB," but backward compatibility matters in books, too. The present form of this acronym also does not account for the newer obscure scopes of comprehensions and exception handlers, but acronyms longer than fou

r letters tend to defeat their purpose!

中文翻译：作用域查找规则在本书第一版中被称为"LGB 规则"。后来 Python 加入了外围的 "E" 层，以避免用默认参数显式传入外围作用域名字的麻烦——这是本章稍后会示例的高级主题。由于现在 nonlocal 语句已经处理了这一作用域，查找规则或许更应该叫 "LNGB"，但向后兼容在书里也同样重要。这个缩写的现行形式也没有涵盖推导式和异常处理器的较新小众作用域，但超过四个字母的缩写往往会违背它们存在的目的！

脚注 2

英文原文：Multithreading runs function calls in parallel with the rest of a program and is supported by Python's standard-library modules thread, threading, and queue. Because all threaded functions run in the same process, global scopes often serve as one form of shared memory between them (threads may share both names in global scopes, as well as objects in a process's memory space). Threading is commonly used for long-running tasks in GUIs, to implement nonblocking IO, and to utilize CPU capacity. Threading is also well beyond this language book's scope (a property it shares with async functions, which are nevertheless part of Python syntax today, as you'll learn in Chapter 20). See Python's library manual for more details.

中文翻译：多线程（Multithreading）让函数调用与程序其余部分并行运行，由 Python 标准库模块 `thread`、`threading` 和 `queue` 支持。因为所有线程函数都运行在同一进程中，全局作用域常常充当它们之间的一种共享内存形式（线程既可以共享全局作用域中的名字，也可以共享进程内存空间中的对象）。线程常用于 GUI 中的长时间运行任务、实现非阻塞 IO，以及利用 CPU 容量。线程也远远超出了这本语言书的范围（这一特点与 `async` 异步函数相同——不过异步函数如今已经是 Python 语法的一部分，你将在第 20 章学到）。更多细节请参见 Python 库手册。

本章总结

技术拓展（Technical Expansion）

1. 实际项目中的应用场景

- 配置与计数器：函数内修改模块级配置（`global`）、统计函数调用次数（`nonlocal` 或函数属性）——生产代码中最常见的两种用法。
- GUI 事件回调：循环中为按钮生成处理器必须用 `lambda x, i=i:`（或 `functools.partial`）固化当前值，否则“所有按钮做同一件事”（17.28 节的现实版事故）。
- 装饰器与缓存：第 39 章装饰器本质上就是“工厂函数 + 多层状态保留”；`functools.lru_cache` 内部就用了“函数属

性 + 闭包"保存缓存字典。

·猴子补丁与插桩：makeopen 模式广泛用于测试替身（unittest.mock 内部原理）、性能埋点、第三方库行为修正；闭包保存原版是"可回滚替换"的前提。

·调试技巧：f.closure[0].cellcontents 可以直接窥探闭包捕获的状态——排查"闭包为什么记住了这个值"时一行搞定。

·专业工具：dis.dis() 反汇编查看 LOADDEREF/STOREFAST/STOREGLOBAL 指令，直观验证"编译器把名字归到哪层"；inspect 模块可查看 closure、defaults、globals。

2. 与其他语言的区别 (Java/C++)

维度	Python	Java	C/C++
作用域确定时机	编译期按赋值位置（词法）	编译期按声明块（块级）	编译期按声明块
函数嵌套	一等特性（闭包）	需匿名内部类 / lambda (effectively final 限	标准 C 无嵌套函数（GCC 扩展），C++ 可
捕获外部变量	任意变量（cell），读写皆可（nonlocal）	lambda 只能捕获 effectively final 变量，只读	C++ lambda 需显式 [&]/[=] 指定捕获方式
全局变量	模块级，跨文件需 import	静态字段（static），跨类可见	全局变量 + extern 声明
状态保留	nonlocal / 函数属性 / 类	实例字段	静态局部变量（static）、堆对
编译期安全检查	有（nonlocal 绑定检查）	有（final 变量检查）	无（靠程序员自觉）

核心洞察：Python 的独特之处在于——作用域归类发生在编译期（一眼可判），而闭包捕获却是全动态（任何变量皆可捕获、nonlocal 可写）。Java 的 effectively-final

| 限制与 C++ 的显式捕获语法，都是"编译器需要静态证明"的折中；Python 用 cell 对象的一层间接，把这种证明成本几乎归零。

3. Python 发展历史背景

- 1991 年 (Python 0.9)：最初只有 LGB 三个作用域——嵌套函数引用外层变量会直接失败（只能搜 local/global/builtin）。
- 1994 年 (Python 1.x 时代)：程序员发明"默认参数技巧" (lambda x=x:) 手动模拟捕获——这就是 17.27 节老技法的起源。
- 2001 年 (Python 2.1)：PEP 227 引入静态嵌套作用域 (E 层) ——"函数自动记住外围环境"成为语言默认行为，默认参数技巧退居二线。
- 2001 年 (Python 2.2)：类作用域细节调整；此前的类行为与 LEGB 的关系一直是兼容性重灾区。
- 2008 年 (Python 3.0)：PEP 3104 引入 nonlocal 语句——闭包状态从"只读"升级为"可写"，Python 3 与 Python 2 的关键分歧之一 (Python 2 没有 nonlocal，只能用可变对象或函数属性曲线救国)。
- 2000-2008 年间：列表推导式的循环变量泄漏问题 (Python 2) 成为著名痛点，Python 3 中推导式获得独立局部作用域 (17.5 节)，循环陷阱只剩"闭包共享 cell"这一个形态。

历史教训：作用域机制二十年的演进主线是"从手动到自动

、从只读到可写": 默认参数技巧 (手动) → E 层 (自动读) → nonlocal (自动写)。每次演进都保持向后兼容 (老技巧至今仍有效), 这是 Python 工程哲学的典型样本。

4. 高级开发者需要掌握的相关知识

· cell 与 free variable 机制: 闭包不是魔法——closure 元组里的 cell 对象是"名字与值的间接层"; 理解它才能解释"循环陷阱"与"计数器独立副本"这两个看似矛盾的现象。

· 字节码层面: LOADFAST (局部) / LOADDEREF (cell) / LOADGLOBAL (模块) / LOADATTR (属性) 四类指令的分工, 是"名字 vs 对象 vs 属性"三层模型的最直观表达; 用 dis 模块亲自验证。

· inspect 与运行时自省: inspect.getclosurevars(f) 一步列出闭包捕获的自由变量; f.defaults 查看默认值快照; 调试"闭包为什么这样"时是主力工具。

· functools.partial: 部分求值 (partial application) 的官方工具, 是"工厂函数 + 默认值"思想的库级实现, GUI 回调、事件处理器首选。

· 类与闭包的交汇: 装饰器 (第 39 章)、属性 (@property)、上下文管理器 (第 34 章) 都是"状态保留"家族成员; 学完 Part VI 后应能比较"闭包状态 vs 实例状态"的取舍。

· 闭包与内存: 闭包会让 cell 持有大对象存活——"函数返回后对象不释放"是内存分析 (tracemalloc) 时要留意的

点；except as 变量自动删除的动机（17.5 节）与此同源。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5) 。作用域是 Python 的"第二内脏"（第一是对象模型）——从第 18 章的参数、第 20 章的迭代器/生成器（yield 后的状态保留）、第 29 章的类方法到第 39 章的装饰器，全部建立在本章之上。LEGB 与闭包是面试高频题，也是读任何真实项目源码的必备能力。

·应该掌握到什么程度：

·必会：能默写 LEGB 规则并解释四层含义；区分"读全局免声明、改全局需 global"；能独立复现并解释 17.28 节循环陷阱及 `i=i` 解法；说清 nonlocal 与 global 的差异及 nonlocal 的编译期绑定检查；闭包的 cell 机制（closure）。

·应知：四种状态保留方案（global/nonlocal/函数属性/类）的取舍；`X = 99` vs `X += 1` 触发的 UnboundLocalError 原因；推导式循环变量与 for 循环变量的作用域差异；except as 变量的生命周期；默认值求值时机（定义时）及其与闭包的交互。

·不要求：现在就精通类作用域细节（第 29 章）、装饰器多层状态（第 39 章）、多线程共享内存（库手册）；不必背诵字节码指令集。

·后续学习内容：

- 立即：第 18 章"参数"——默认值求值时机（本章技巧的地基）、args/kwargs（makeopen 的前置）。
- 近程：第 19 章递归与函数细节（closure 相关内省）、第 20 章迭代器与生成器（生成器函数体天然"状态机"，yield 保存执行现场）。
- 进阶：Part VI 类与继承（"类被作用域查找跳过"的现实后果）、第 34 章异常（except as 变量删除）、第 39 章装饰器（四种状态方案的最终汇演）。
- 实践建议：①用 dis.dis 反汇编 maker 工厂函数与 inner，亲眼确认 LOADDEREF/STOREDEREF；②做"按钮陷阱"实验：先写 17.28 的错误版，观察全部输出 16，再改用 i=i 版，体会"快照 vs 引用"；③动手实现一个 makeopen 式的"调用追踪器"，用两层嵌套验证链式捕获；④把 F.closure[0].cellcontents 打出来，亲眼看看闭包里的状态。